IFC4x3 Alignment Geometry Implementation Guide

An implementation guide for developers adding IFC4x3 alignment geometry support to infrastructure software applications.

Richard Brice, PE

Foreword

`IfcAlignment`, and the related geometric concepts, are new to the IFC4x3 specification. By virtue of expanding the IFC specification into the infrastructure domain, an entirely new audience is summoned into the buildingSmart community. I am one of those new arrivals.

The IFC specification is very technical and is written with an expectation that the reader has a high level of knowledge and experience with IFC. Being a technical specification, there is no hand-holding for the novice. Industry guidance documents are generally not available to assist those new to the infrastructure and alignment aspects of IFC4x3.

I set out to add alignment geometry support to the IfcOpenShell Toolkit. Being a bridge engineer, I am well versed in highway alignment geometry. I was completely overwhelmed and befuddled with alignment representations in the IFC specifications. Through a long and persistent struggle, I was able to figure out most all of what a developer needs to implement IFC alignment geometry in a software application.

This implementation guide captures what I have learned about IFC4x3 alignment geometry and provides guidance as to how a software developer may go about creating an implementation. I hope that other developers find this guide useful, because in the end, IFC is about interoperability. We all need a common understanding of the infrastructure alignment concepts and geometry to successfully exchange information.

Table of Contents

Notation Scalars and Parameters Arc-Length and Distance

Symbol	Definition
s	Arc-length parameter along the parent curve
s_0	Arc-length at the start of the trim; equals `SegmentStart`
L	Segment length; equals `SegmentLength`
ℓ	Horizontal distance from the segment start; evaluation variable in vertical and cant sections
d	Distance measured along the horizontal `IfcCompositeCurve`
d_0	Distance along at the trim start; $d_0 = x(s_0)$
h_l	Horizontal length of a vertical segment; equals `HorizontalLength`

Angles

Symbol	Definition
$\theta(s)$	Bearing angle at arc-length s in the horizontal plane
$\theta(\ell)$	Tangent angle at horizontal distance ℓ; grade angle $\tan^{-1}(g(\ell))$ in the vertical plane; slope of deviating elevation $\tan^{-1}(D'(\ell))$ in the cant plane
θ_0	Tangent angle at the trim start s_0
θ_p	Tangent angle of `IfcCurveSegment.Placement.RefDirection`
θ_v	Grade direction angle; $\theta_v = \tan^{-1}(dy_v/dx_v)$
ϕ	Cross-slope angle; angle from the transverse y-axis to the `Axis` vector
$\phi_s,\ \phi_e$	Cross-slope angle at the start and end of a cant segment
ϕ_p	Cross-slope angle of `IfcCurveSegment.Placement.Axis`
Δ	Sweep angle of a circular arc

Curvature and Radius

Symbol	Definition
$\kappa(s)$	Curvature at arc-length s; $\kappa = 1/R$
κ_s	Curvature at the segment start; $\kappa_s = 1/R_s$
κ_e	Curvature at the segment end; $\kappa_e = 1/R_e$
R	Radius of curvature
R_s	Radius at the segment start; equals `StartRadius`
R_e	Radius at the segment end; equals `EndRadius`
f	Cumulative change in curvature over the segment; $f = L(1/R_e - 1/R_s)$

Grade

Symbol	Definition
$g(s)$	Gradient (slope) at arc-length s
g_s	Gradient at the segment start; equals `StartGradient`
g_e	Gradient at the segment end; equals `EndGradient`

Position and Elevation

Symbol	Definition
$x(s),\ y(s)$	Coordinates of the parent curve at arc-length s
$x_0,\ y_0$	Parent curve position at the trim start s_0
$x_p,\ y_p$	Placement location; equals `IfcCurveSegment.Placement.Location`
z	Elevation (vertical y-coordinate mapped to 3D z)
z_0	Elevation at the trim start; equals `StartHeight`
$C_x,\ C_y$	Center coordinates of a circular arc parent curve
c	Chord length from the trim start to an evaluation point

Cant (Deviating Elevation)

Symbol	Definition
$D(s)$	Deviating elevation at arc-length s; vertical offset of the track centreline from the gradient curve
D_0	Deviating elevation at the trim start; $D_0 = D(s_0)$
$D_s,\ D_e$	Deviating elevation at the segment start and end; $D_s = (D_{sl} + D_{sr})/2$
$D_{sl},\ D_{sr}$	Left and right cant elevation at the segment start; equals `StartCantLeft`, `StartCantRight`
$D_{el},\ D_{er}$	Left and right cant elevation at the segment end; equals `EndCantLeft`, `EndCantRight`
D_p	Deviating elevation component of `IfcCurveSegment.Placement.Location`
ΔD	Change in deviating elevation over the segment; $\Delta D = D_e - D_s$
D_{rh}	Rail head distance; centre-to-centre distance between railheads; equals `RailHeadDistance`

Spiral Curve Coefficients

Symbol	Definition
A	Clothoid constant
u	Clothoid unit parameterization parameter; $u = s\ /\ \lvert A\sqrt{\pi}\rvert$
A_i	Dimensional polynomial coefficient (with units) for spiral and parabolic curves
a_i	Normalized (dimensionless) polynomial coefficient prior to scaling
$A_{i1},\ A_{i2}$	Coefficient A_i for the first and second halves of a Helmert segment

---

Vectors

Symbol	Definition
$dx,\ dy$	Horizontal tangent direction components; $dx = \cos\theta$, $dy = \sin\theta$
$dx_p,\ dy_p$	Tangent direction components at the placement; $dx_p = \cos\theta_p$, $dy_p = \sin\theta_p$
$dx_v,\ dy_v$	Grade direction components from M_v; $dx_v = \cos\theta_v$, $dy_v = \sin\theta_v$
$\mathbf{RefDir}_p$	Reference direction of `IfcCurveSegment.Placement`; equals `Placement.RefDirection`
$\mathbf{Axis}_p$	Axis direction of `IfcCurveSegment.Placement`; equals `Placement.Axis`
$\mathbf{X}_p,\ \mathbf{Y}_p,\ \mathbf{Z}_p$	Orthonormal basis vectors of the placement frame

---

Matrices

All matrices are 4×4 homogeneous transformation matrices. Column 4 carries position; columns 1–3 carry the frame orientation.

Symbol	Definition
M_{CSP}	Curve segment placement matrix; constructed from `IfcCurveSegment.Placement`; maps the trimmed segment into the alignment coordinate system
M_N	Normalization matrix; translates the trim-start point to the origin and rotates the tangent to align with the positive x-direction
M_{PC}	Parent curve matrix at the evaluation point; encodes $x(s)$, $y(s)$, and $\theta(s)$
M_h	Horizontal placement matrix; $M_h = M_{CSP}\, M_N\, M_{PC}$
M_v	Vertical placement matrix; $M_v = M_{CSP}\, M_N\, M_{PC}$ in the (distance-along, elevation) plane
M_v'	Modified vertical matrix; rows 2 and 3 of M_v swapped, distance-along component zeroed, for multiplication with M_h
M_v''	Orientation-only form of M_v'; column 4 set to $(0, 0, 0, 1)^T$ for use in the cant 3D composition
M_c	Cant placement matrix; $M_c = M_{CSP}\, M_N\, M_{PC}$ in the (distance-along, deviating elevation) plane
M_c'	Modified cant matrix; deviating elevation moved from row 2 to row 3, distance-along component zeroed, for multiplication
M_c''	Orientation-only form of M_c'; column 4 set to $(0, 0, 0, 1)^T$ for use in the cant 3D composition
M_{3D}	Full 3D placement matrix combining horizontal and vertical; $M_{3D} = M_h\, M_v'$
$M_{3D,cant}$	Full 3D placement matrix combining horizontal, vertical, and cant; $M_{3D,cant} = M_h\, M_v''\, M_c''$
I	4×4 identity matrix

Section 1 - IFC Alignment Concepts 1.0 Introduction

IFC 4.3 ADD 2 finalized the adoption of alignments for infrastructure works. Since the concept of alignment is relatively new to IFC, few details and examples are provided in the available literature. This guide aims to document the relevant concepts and mathematics a software developer might need to implement alignment-based geometry.

If you are unfamilar with infrastructure alignment geometry, information is available in a multitude of highway engineering, rail engineering, and surveying texts. A concise primer is available in the US Federal Highway Adminstration (FHWA), Bridge Geometry Manual, Chapters 1 - 5 and Appendix B.

The IFC specification identifies concept templates as the mechanism for mapping semantic alignment representations to their geometric counterparts. The concept templates — covering aggregation to project, alignment layout nesting, and the four alignment geometry variants (horizontal; horizontal+vertical; horizontal+vertical+cant; segments) — provide the minimum information required to properly structure alignment semantic and geometric entities an IFC model. The *partial* concept templates that cover individual curve segment geometry types, however, are sparse: they show class relationships but provide no mathematical equations or parameter mappings. This guide fills that gap. It assumes the reader has a basic working knowledge of IFC — familiarity with the schema structure, entity relationships, and how IFC files are organized — and focuses specifically on the alignment geometry that IFC 4.3 ADD 2 introduced.

1.1 Horizontal Alignment

A horizontal alignment typically consists of straight lines called tangents (or tangent runs) that are connected by curves. The curves are usually circular arcs. Easement or transition curves in the form of spirals can be used to provide gradual transitions between tangents and circular curves. The horizontal alignment is a plan view curvilinear path in a Cartesian coordinate system aligned with East and North. Positions along an alignment, measured along the plan view projection of the 3D alignment curve, are typically denoted with stations.

1.2 Vertical Alignment

A vertical alignment (often referred to as the vertical profile by civil engineers) consists of straight sections of grade lines connected by vertical curves. The vertical curves are typically parabolas, though sometimes circular arcs or clothoid curves are used. A vertical alignment is defined along the curvilinear path of a horizontal alignment in a 2D "Distance Along, Elevation" coordinate system. Combined, the horizontal and vertical alignments define a 3D curve. This combination of two 2D curves is sometimes referred to as a 2.5D or 2D+1D geometry. The IFC specification notes that contemporary alignment design almost always implements this 2.5D approach, with precision varying based on management priorities, design era, and available software tools.

1.3 Cant Alignment

When a rail vehicle traverses a horizontal curve, centrifugal force acts laterally on passengers and cargo. Tilting the track cross-section — raising the outer (high) rail above the inner (low) rail — converts part of that lateral force into a downward component, improving ride comfort and permitting higher operating speeds through curves. This tilt is called **cant** (or *superelevation* in road engineering, though the two are geometrically analogous).

Cant is measured as the height difference between the two rail heads, perpendicular to the track. The low (inner-curve) rail is the conventional pivot for rotation; the high (outer-curve) rail rises by the cant amount.

A cant alignment is structured similarly to the horizontal and vertical alignments: **constant-cant zones** — corresponding to the circular arc sections of the horizontal alignment, where cant is held at a fixed value — are connected by **cant transitions** that ramp over the length of the horizontal transition spiral. Transitions may be linear or may follow the same non-linear curve families used for horizontal spirals.

Like the vertical alignment, the cant alignment is defined in a two-dimensional profile: distance along the horizontal alignment on one axis, cant value on the other. The cant profile encodes not only a deviating elevation but also the cross slope of the rail head, fully defining the rotated cross-section. This profile is then layered onto the combined horizontal and vertical alignment to produce a full 3D alignment curve incorporating cross-sectional banking.

1.4 Semantic Definition (Business Logic)

The semantic definition of alignment allows for the alignment to be described as close as possible to the terminology and concepts used in a business context. Examples include horizontal, vertical and cant layouts, stationing, and anchor points for domain specific properties such as design speed, cant deficiency, superelevation transitions, and widenings.

Specialized applications can analyze alignments using the semantic information. Alignments can be evaluated against design criteria such as speed requirements, sight distances, and maximum gradient, to name a few.

Importantly, the semantic and geometric representations are independent and can be used and exchanged separately. A proper IFC file may contain only the semantic definition without any geometric representation, which is common in early design stages when geometry has not yet been computed.

An `IfcAlignment` may be defined with a horizontal layout only, a horizontal and vertical layout, or a full horizontal, vertical, and cant layout. Each layout type is composed through an `IfcRelNests` relationship: `IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant` are nested within the `IfcAlignment` as applicable. Each layout is in turn composed of one or more `IfcAlignmentSegment` entities, also through `IfcRelNests`. The `IfcAlignmentSegment` models the segment design parameters in a subtype of `IfcAlignmentParameterSegment`: `IfcAlignmentHorizontalSegment`, `IfcAlignmentVerticalSegment`, or `IfcAlignmentCantSegment`, depending on the layout. The semantic definition model is shown in Figure 1.4-1.

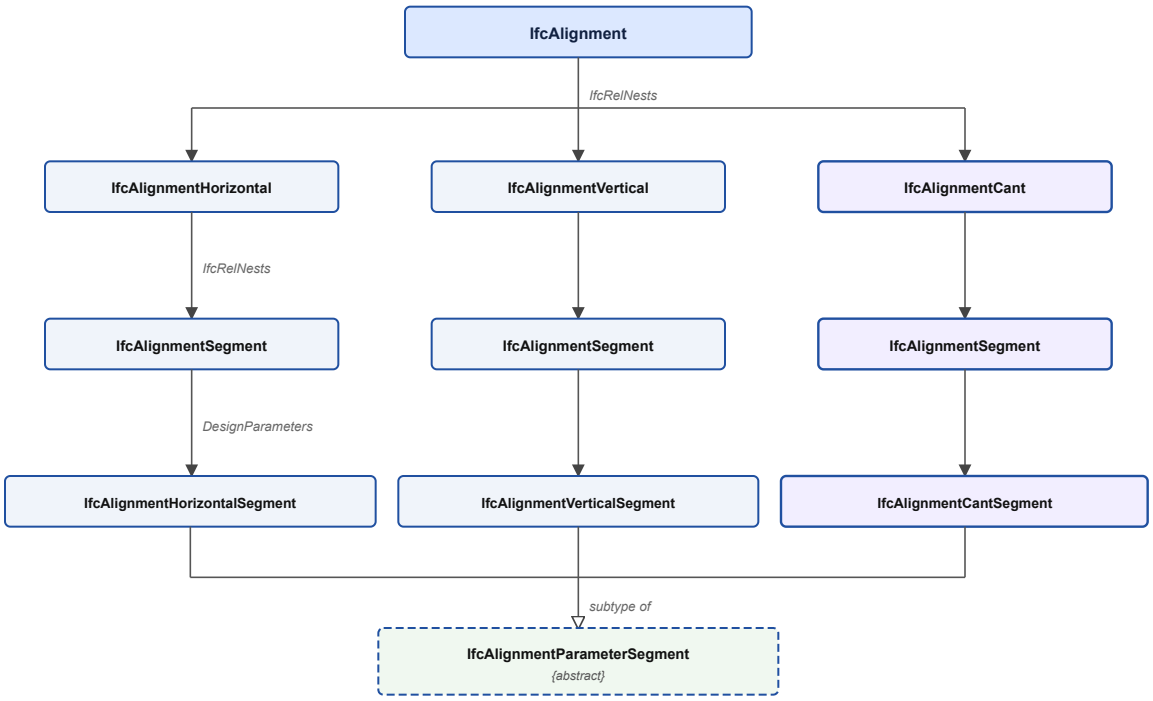


Figure 1.4-1 Semantic alignment model

`IfcRelNests.RelatedObjects` is an ordered collection, so the layout sub-objects have a defined sequence within the nest. The IFC specification does not state this order explicitly, but it can be inferred from the diagrams in the `IfcAlignment` documentation (§5.4.3.1.5): `IfcAlignmentHorizontal` precedes `IfcAlignmentVertical`, which precedes `IfcAlignmentCant`.

The ordering of `IfcAlignmentSegment` instances within each layout's nest is similarly unspecified by the schema. For `IfcAlignmentHorizontalSegment` the only logical order is start to end. For `IfcAlignmentVerticalSegment` and `IfcAlignmentCantSegment`, positional order could theoretically be derived from the `StartDistAlong` attribute regardless of list order, but relying on this is unnecessarily complex. Segments should always be ordered start to end in `IfcRelNests.RelatedObjects`.

An interesting caveat of the alignment layout entities (`IfcAlignmentHorizontal` , `IfcAlignmentVertical` , and `IfcAlignmentCant`) is that the last `IfcAlignmentSegment.DesignParameters` must be zero length. If a geometric representation is provided, then its corresponding last segment must also be zero length. For a continuous composition of segments, the end of one segment is at the same location as the start of the next segment. The zero-length segment is intended to provide the end point of the last segment.

Aggregation to project. `IfcAlignment` is a mandatory participant in the IFC spatial structure. Every alignment must be aggregated to `IfcProject` — directly, or indirectly through a parent alignment — via an `IfcRelAggregates` relationship. An alignment not connected to the project is invalid.

Referencing to site. Because an alignment typically traverses multiple administrative or physical sites, `IfcAlignment` associates with `IfcSite` using `IfcRelReferencedInSpatialStructure` rather than `IfcRelContainedInSpatialStructure` . The referenced-in relationship is non-exclusive: a single alignment may be referenced by every `IfcSite` it crosses. This is the key distinction between *containment* (one element, one spatial zone) and *referencing* (one element, many zones).

1.5 Geometric Representation 1.5.1 Overview

Section 4.1.7.1.1 of the IFC 4x3 specification indicates that the valid representations of `IfcAlignment` are:

- `IfcCompositeCurve` as a 2D horizontal alignment (represented by its horizontal alignment segments), without a vertical layout.
- `IfcGradientCurve` as a 3D horizontal and vertical alignment (represented by their alignment segments), without a cant layout.
- `IfcSegmentedReferenceCurve` as a 3D curve defined relative to an IfcGradientCurve to incorporate the application of cant.
- `IfcOffsetCurveByDistances` as a 2D or 3D curve defined relative to an `IfcGradientCurve` or another `IfcOffsetCurveByDistances` .
- `IfcPolyline` or `IfcIndexedPolyCurve` as a 3D alignment by a 3D polyline representation (such as coming from a survey).
- `IfcPolyline` or `IfcIndexedPolyCurve` as a 2D horizontal alignment by a 2D polyline representation (such as in very early planning phases or as a map representation).

The `RepresentationIdentifier` and `RepresentationType` values required by the IFC concept templates for each alignment geometry variant are listed in Table 1.5.1-1.

Alignment variant	**Curve entity**	**`RepresentationIdentifier`**	**`RepresentationType`**
Horizontal only	`IfcCompositeCurve`	`'Axis'`	`'Curve2D'`
Horizontal + Vertical	`IfcCompositeCurve` (plan view)	`'FootPrint'`	`'Curve2D'`
Horizontal + Vertical	`IfcGradientCurve` (3D)	`'Axis'`	`'Curve3D'`
Horizontal + Vertical + Cant	`IfcCompositeCurve` (plan view)	`'FootPrint'`	`'Curve2D'`
Horizontal + Vertical + Cant	`IfcSegmentedReferenceCurve` (3D)	`'Axis'`	`'Curve3D'`

Table 1.5.1-1 — Required RepresentationIdentifier and RepresentationType for each alignment geometry variant

`IfcGradientCurve` and `IfcSegmentReferenceCurve` inherit from `IfcCompositeCurve` . These curves consist of a sequence of segments defined by `IfcCurveSegment` . The mathematical computations for `IfcCurveSegment` geometry are the primary focus of this document.

`IfcPolyline` and `IfcIndexedPolyCurve` produce approximate, discrete geometry rather than exact parametric curves. They arise in survey-derived alignments and early planning contexts. Their use, limitations, and incompatibility with the full semantic alignment definition are discussed in Section 6.

`IfcOffsetCurveByDistances` is treated in a dedication section.

1.5.2 Understanding Geometric Representation of Alignment

The semantic definition of an alignment and its geometric representation carry overlapping information — both describe segment types, lengths, and radii — but they serve different consumers. The semantic representation encodes design intent in domain vocabulary: a horizontal segment typed as `CLOTHOID` with a `StartRadius` of infinity, an `EndRadius` of 300 m, and a `SegmentLength` of 100 m tells a design application *what* the engineer specified and enables evaluation against standards for minimum radius, design speed, and sight distance. The geometric representation encodes the computed mathematical form: the exact start coordinates, the tangent bearing at each point, and parametric equations that yield position and direction at any arc-length along the curve — the form that a rendering engine, clash-detection tool, or quantity-takeoff calculation requires. Because the two representations serve different consumers, an IFC file may legally contain only the semantic definition, only the geometric representation, or both.

The geometric representation of `IfcAlignment` consists of one or more `IfcShapeRepresentation` instances: a plan-view 2D curve and, where vertical or cant geometry is present, a 3D curve. These are illustrated in Figure 1.5.2-1.

Geometrically, the horizontal alignment is defined by an `IfcCompositeCurve` and the vertical alignment is defined by an `IfcGradientCurve`. The horizontal alignment is the basis curve for the vertical alignment.

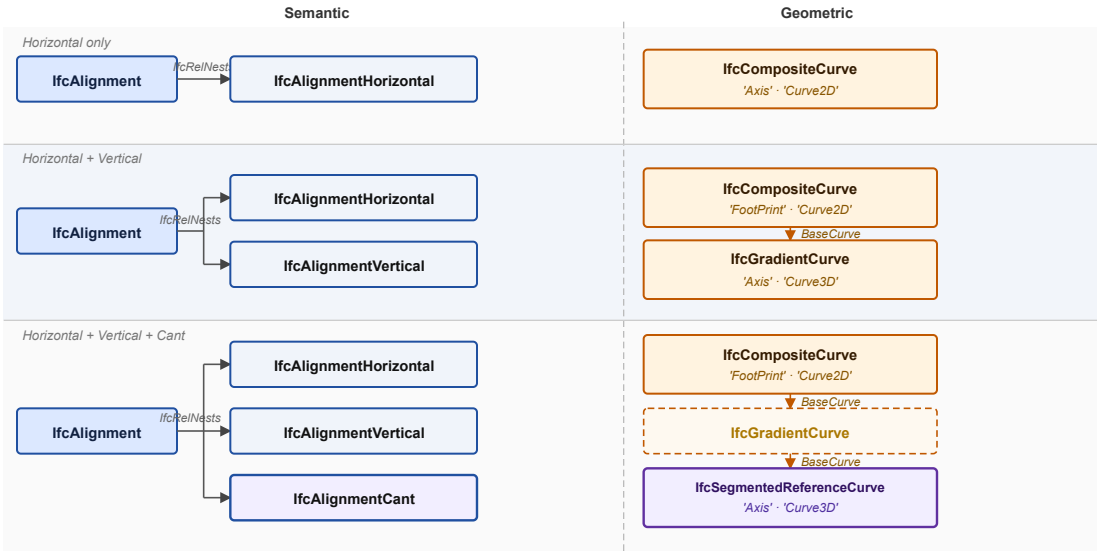


Figure 1.5.2-1 Geometric representation variants for the three alignment constructs (H, H+V, H+V+C). The dashed `IfcGradientCurve` box in the H+V+C row appears as a `BaseCurve` intermediate only — it is not itself a shape representation.

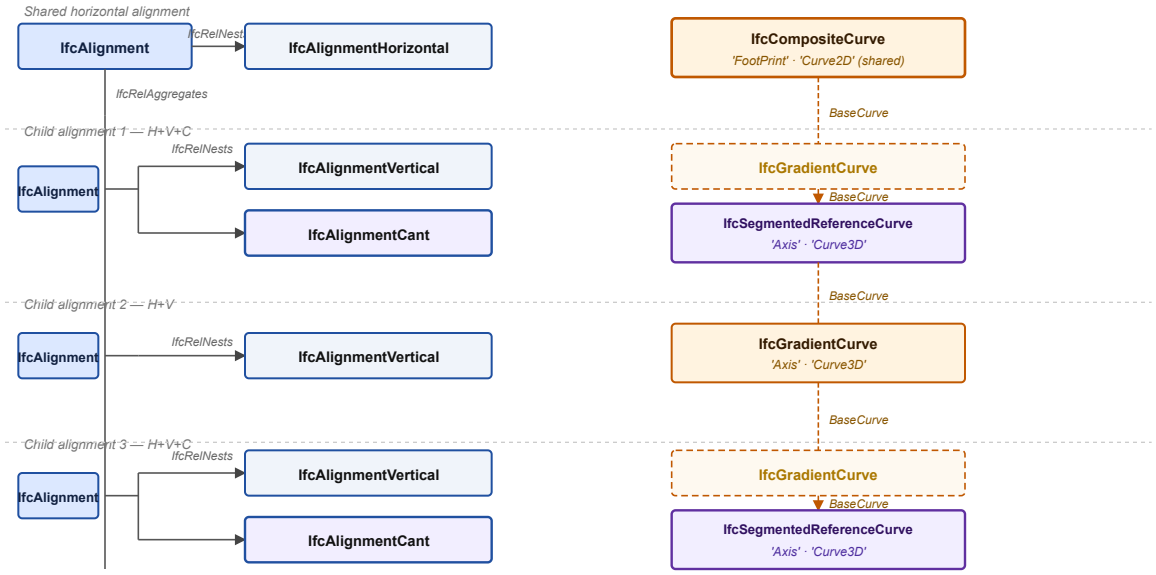


Figure 1.5.2-2 Reuse of a shared `IfcCompositeCurve` across multiple child alignments. Each child `IfcAlignment` nests only its own vertical and cant layouts; the horizontal `IfcCompositeCurve` is referenced as `BaseCurve` by each child's gradient or segmented reference curve.

The geometric elements are modeled with `IfcCurveSegment`. `IfcCurveSegment` cuts a segment from a parent curve and places it in the alignment coordinate system. A horizontal circular arc is modeled with an `IfcCurveSegment` that cuts an arc from an `IfcCircle`. Figure 1.5.2-3 shows an alignment consisting of a tangent run (red) and a horizontal curve (green). The line and curve segments are cut from their `IfcLine` and `IfcCircle` parent curves, respectively. The process of defining a parent curve, cutting a segment from that curve, and placing the cut segment into the alignment is repeated to define the entire horizontal alignment. All the horizontal alignment `IfcCurveSegment` objects are then combined to create an `IfcCompositeCurve` to complete the plan view geometric representation of the horizontal alignment.

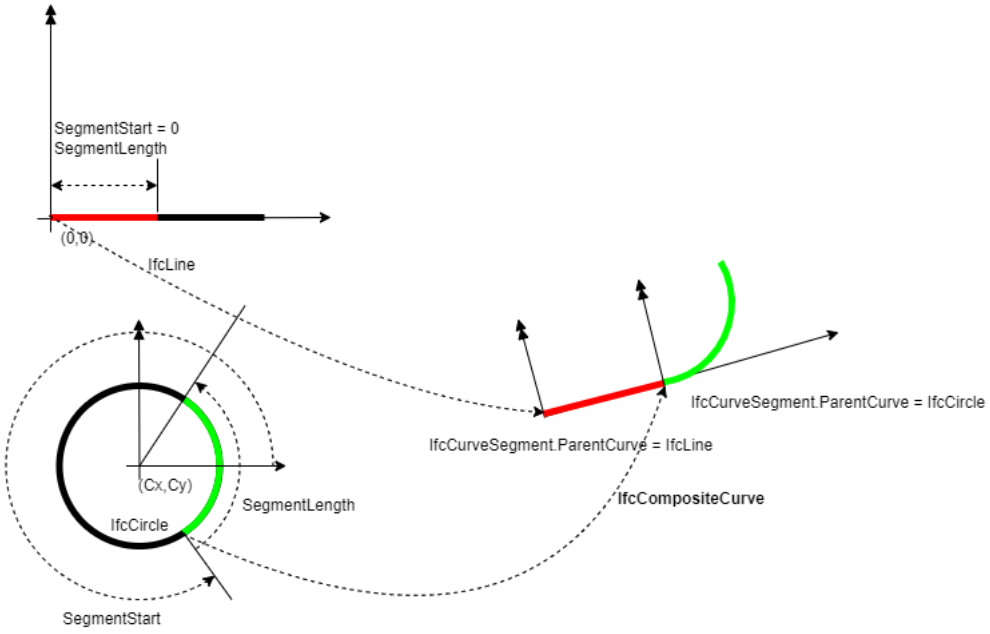


Figure 1.5.2-3 Illustration of parent curves and their placement with `IfcCurveSegment.Placement`

A similar process is used to create the vertical alignment. `IfcCurveSegment` trim geometry from parent curves and combine end to start to create an `IfcGradientCurve`. `IfcGradientCurve` is a sub-type of `IfcCompositeCurve` with the additional attribute `BaseCurve`. The segments of the gradient curve are positions in a 2-dimensional plane in a "distance along the horizontal alignment, elevation" coordinate system. The `IfcGradientCurve` uses the horizontal alignment `IfcCompositeCurve` as its base curve.

`IfcSegmentedReferenceCurve` is also a sub-type of `IfcCompositeCurve`. It defines a deviating elevation and rail head cross slope along the base curve. The base curve is typically an `IfcGradientCurve`, though the IFC specification would permit it to be any `IfcCompositeCurve`. The "distance along" coordinate is along the gradient curve's base curve, which is the horizontal alignment curve.

1.5.2.1 Curve segment geometry

When defining curve segment geometry with an `IfcCurveSegment`, the location and orientation of the parent curve are not particularly important. The `IfcCurveSegment.Placement` defines the location of the start point of the trimmed curve as well as the orientation of the tangent at the start of the trimmed curve. In the example above, the `IfcLine` parent curve is starts at (0,0) and progresses in horizontal direction, but the tangent segment of the alignment starts an specific (X,Y) position and is oriented in a North-East direction. The `IfcCurveSegment.Placement` attribute establishes this position and orientation.

1.5.2.2 Vertical and Cant Curve Extents

For `IfcGradientCurve` and `IfcSegmentedReferenceCurve` the segments do not need to start or end at the same location of their base curve. These curves can be shorter or longer than the horizontal alignment. This is illustrated in Figure 1.5.2.2-1 where the blue curve is the full horizontal curve and the orange curve is the portion of the horizontal curve coinciding with the vertical curve. The `IfcGradientCurve` (#770) has the `IfcCompositeCurve` (#657) as its base curve. The placement of the first `IfcCurveSegment` (#771) in the `IfcGradientCurve` is defined by `IfcAxis2Placement2D` (#777) with Location as `IfcCartesianPoint` (#778). The first segment is located at a distance of 250.0036 along the horizontal `IfcCompositeCurve` at an elevation of 145.3129 m. The IFC specification does not provide guidance on how to define complete 3D geometry in the regions of the horizontal alignment that do not overlap with the vertical or cant curves.

```
#657=IFCCOMPOSITECURVE((#658,#671,#684,#697,#710,#723,#736,#749,#762),.F.)
#770=IFCGRADIENTCURVE((#771,#784,#796,#809,#821,#834,#846,#859,#871),.F.,#657,#887)
#771 = IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE., #777, FCNONNEGATIVELENGTHMEASURE(0.),
IFCNONNEGATIVELENGTHMEASURE(40.0002408172751), #780);
#777 = IFCAXIS2PLACEMENT2D(#778, #779);
#778 = IFCCARTESIANPOINT((250.003634293681, 145.312908277025));
```

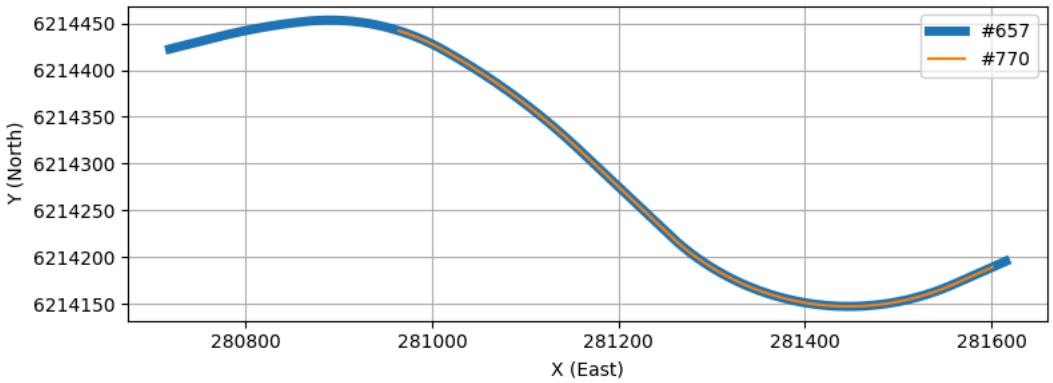


Figure 1.5.2.2-1 — Example of `IfcGradientCurve` with a start location and length different than the `IfcCompositeCurve` BaseCurve.

1.6 Reference Implementation and Segment Mapping:

Sections 2.0, 3.0, and 4.0 discuss the mapping of the semantic definition of alignment segments (`IfcAlignmentSegment` subtypes) to their corresponding geometric representation (`IfcCurveSegment.ParentCurve`). In general, the mapping formulas are given without derivation and have been developed from the reference implementation, EnrichIFC4x3, published at IFC-Rail-Unit-Test-Reference-Code/EnrichIFC4x3/EnrichIFC4x3/business2geometry at master · bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code (github.com).

The cant segment mappings in the EnrichIfc4x3 reference implementation are incorrect. These errors are documented in Section 4, which presents corrected formulas and explains why the reference implementation is incorrect.

1.7 Units of Measure

Unless otherwise specified, the following unit conventions apply throughout this guide:

Quantity	**Unit**
Length, distance, elevation, offset, cant	meter (m)
Spiral and polynomial coefficients (A, A_0, A_1, ...)	meter (m)
Curvature ($\kappa = 1/R$)	reciprocal meter (m^{-1})
Angles (bearings, grade angles, cross-slope angles)	radian (rad)
Gradient (vertical slope)	dimensionless ratio (m/m)

Table 1.7-1 — Unit conventions used throughout this guide

Gradient is expressed as a decimal rise-over-run value — 0.02 represents a 2% grade — not as a percentage or angle.

Section 2 - Horizontal Alignments 2.0 Introduction

The geometric representation of a horizontal alignment is accomplished with an `IfcCompositeCurve`. The composite curve consists of a sequence of `IfcCurveSegment` entities whose geometry is defined by a parent curve. This section defines the mathematical relationships and equations for each parent curve type and the algorithm for evaluating points on those curves.

Table 2.0-1 maps each `IfcAlignmentHorizontalSegment.PredefinedType` to its corresponding parent curve type.

Business Logic (`IfcAlignmentHorizontaSegment.PredefinedType`)	**Geometric Representation (`IfcCurveSegment.ParentCurve`)**
LINE	`IfcLine`
CIRCULARARC	`IfcCircle`
CLOTHOID	`IfcClothoid`
CUBIC	`IfcPolynomialCurve`
HELMERTCURVE	`IfcSecondOrderPolynomialSpiral`
BLOSSCURVE	`IfcThirdOrderPolynomialSpiral`
COSINECURVE	`IfcCosineSpiral`
SINECURVE	`IfcSineSpiral`
VIENNESEBEND	`IfcSeventhOrderPolynomialSpiral`

Table 2.0-1 — Mapping of business logic to geometric representation for horizontal alignment

2.1 General

Each curve is parameterized by arc-length s, where $s = 0$ at the start of the parent curve. The start and end radii R_s and R_e are taken from `IfcAlignmentHorizontalSegment.StartRadius` and `EndRadius`; a value of zero indicates infinite radius (zero curvature, i.e. a straight line). The segment length L is `IfcAlignmentHorizontalSegment.SegmentLength`. The tangent angle $\theta(s)$ is measured from the positive x-axis; its cosine and sine form the `RefDirection` of the curve at s.

The x and y coordinates of all horizontal parent curves are defined by the integrals:

$$x(s) = \int \cos\theta(s)\, ds \qquad y(s) = \int \sin\theta(s)\, ds$$

2.2 Curve Segment Evaluation Algorithm

This algorithm evaluates the 2D position and tangent direction of a point on an `IfcCurveSegment` in the alignment coordinate system. Let s_0 = `SegmentStart` and s = the arc-length parameter of the point to evaluate, where $s_0 \le s < s_0 + \texttt{SegmentLength}$.

Step 1 — Form the curve segment placement matrix M_{CSP}

M_{CSP} places the trimmed segment into the alignment coordinate system. It is constructed directly from `IfcCurveSegment.Placement`, where (x_p, y_p) is the `Location` and θ_p is the bearing of the `RefDirection`.

$$M_{CSP} = \begin{matrix} \cos\theta_p & -\sin\theta_p & 0 & x_p \\ \sin\theta_p & \cos\theta_p & 0 & y_p \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Compute the position (x_0, y_0) and tangent angle θ_0 of the parent curve at s_0. This establishes the frame of the parent curve at the point where trimming begins.

M_N simultaneously translates the trim-start point to the origin and rotates so that the tangent at s_0 aligns with the positive x-direction.

$$M_N = \begin{matrix} \cos\theta_0 & \sin\theta_0 & 0 & -x_0\cos\theta_0 - y_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & x_0\sin\theta_0 - y_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate and map each point

For the point at arc-length s, compute the parent curve position $(x(s), y(s))$ and tangent angle $\theta(s)$ and form:

$$M_{PC} = \begin{matrix} \cos(\theta(s)) & -\sin(\theta(s)) & 0 & x(s) \\ \sin(\theta(s)) & \cos(\theta(s)) & 0 & y(s) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Apply the normalization and placement in sequence:

$$M_h = M_{CSP} M_N M_{PC}$$

The position of the point in the alignment coordinate system is the fourth column of M_h, and its tangent direction is the first column. Numerical examples of this algorithm are given in Sections 2.3 through 2.11.

2.3 Line

A tangent run is geometrically represented with a segment trimmed from an `IfcLine` parent curve.

2.3.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \theta$$

$$\kappa(t) = 0$$

The point on line equation can be parameterized with a **unit parameterization** where the parameter u ranges from 0 to 1 over the full extent of the curve, independent of its physical length.

$$\lambda(u) = C + L\left(\int_0^{u=1} \cos\left(\theta(t)\right) dt\ x,\ \int_0^{u=1} \sin\left(\theta(t)\right) dt\ y\right)$$

It can also be parameterized with an **arc length parameterization** where the parameter u equals the distance along the line in units of length, so $u = s$.

$$\lambda(u) = C + \left(\int_0^{u=L} \cos\left(\theta(t)\right) dt\ x,\ \int_0^{u=L} \sin\left(\theta(t)\right) dt\ y\right)$$

Other parent curve types can also be parameterized by unit value or arc length. However, the polynomial spirals do not have a well defined unit value parameterization. For this reason, the IFC specification mandates that all parameterization for alignment geometry be by arc length. For `IfcCurveSegment`, the `SegmentStart` and `SegmentLength` attributes **must** be of type `IfcLengthMeasure`. See Section 8.9.3.28 IfcCurveSegment, Informal Proposition 1.

This matters because `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength` are of type `IfcCurveMeasureSelect`, which can be either `IfcParameterValue` (a dimensionless scalar based on unit value) or `IfcLengthMeasure` (a physical length).

2.3.2 Semantic Definition to Geometry Mapping

Mapping of the semantic definition of the linear segment to the geometric definition is described with the following example.

Given a horizontal alignment segment is a line segment starting at point (500,2500), bearing in the direction S 57 E (5.70829654085293 radian) and has a length of 1956.785654.

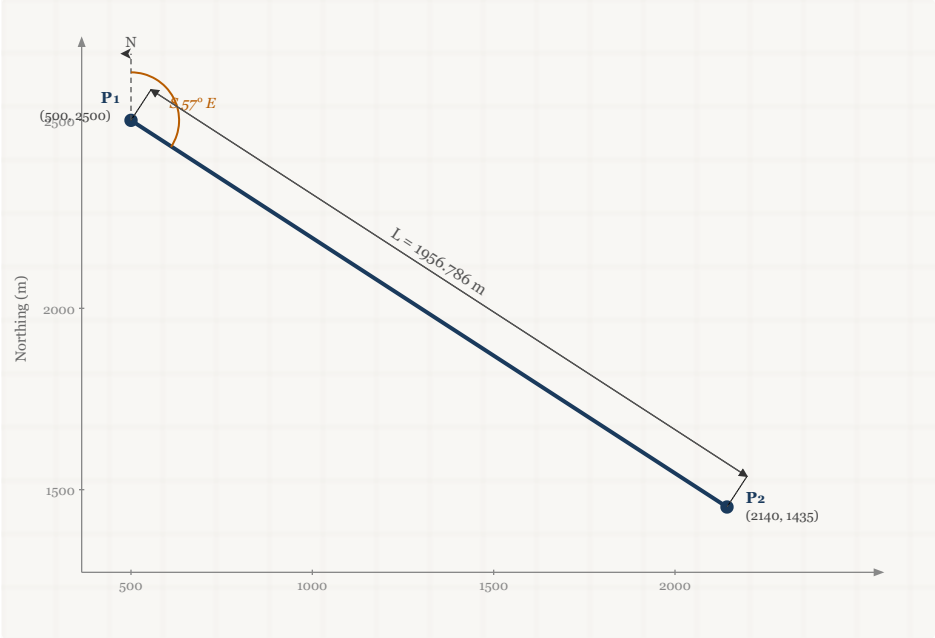


Figure 2.3.2-1 Tangent segment

This semantic definition of this segment is:

```
#31=IFCCARTESIANPOINT((500.,2500.));
#32=IFCALIGNMENTHORIZONTALSEGMENT($,$,#31,5.70829654085293,0.,0.,1956.785654,$,.LINE.);
```

The geometric representation is an `IfcLine`. The line can be defined in different ways and is generally not important, as will be shown in the example below. `IfcLine` is an infinitely long line that passes through a specified point at some direction in the X-Y plane. A valid, but unnecessisarly complex parent curve is show in Figure 2.3.2.2-1. The `IfcLine` passes through poin t(1000,-800) and is direct at 135° from the x-axis. The `IfcCurveSegment` trim starts 10 m from (1000,-800) and the trimmed segment has a lenght of 1956.796 m.

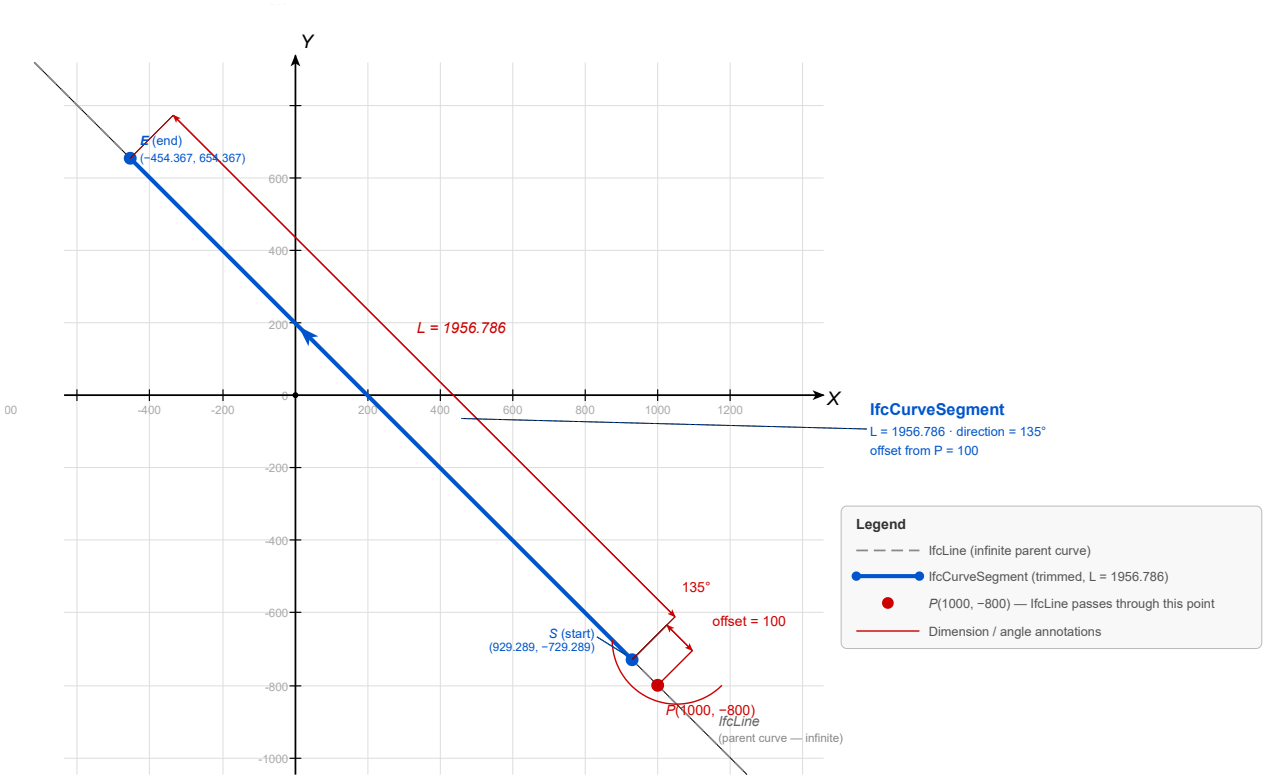


Figure: IfcLine with Trimmed IfcCurveSegment — Arbitrary Placement

Figure 2.3.2-2 IfcLine at arbitrary placement

Figure 2.3.2-2 represents a valid parent curve defiition and implementations must be prepared to properly interpret this parect curve geometry. This guide defines general algorithms to accomodate this geometry.

In practice, it is far easier to define the parent curve passing through point (0,0) and in the direction of the X-axis. The trimming can begin at the origin as shown in Figure 2.3.2-3.

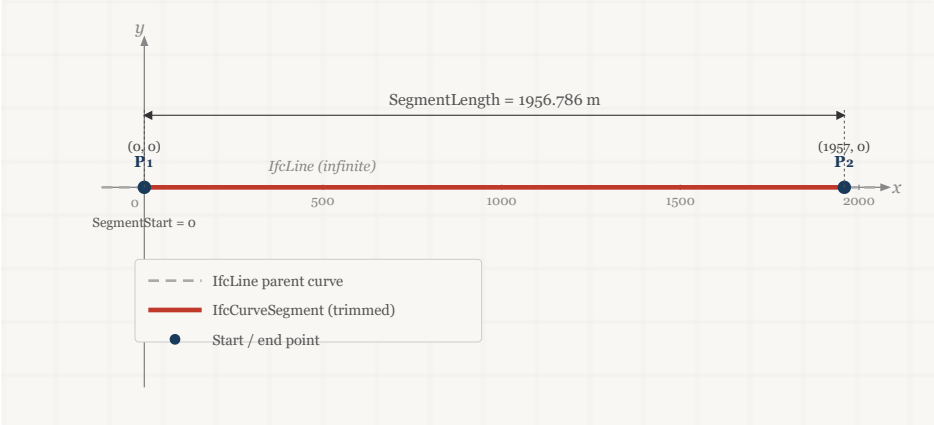


Figure 2.3.2-3 IfcLine parent curve at origin

The parent curve definition is:

```
#51=IFCCARTESIANPOINT((0.,0.));
#52=IFCDIRECTION((1.,0.));
#53=IFCVECTOR(#52,1.);
#54=IFCLINE(#51,#53);
```

`IfcCurveSegment` defines the portion of the line used and how it is placed and oriented in the X-Y plane.

`IfcCurveSegment.SegmentStart` defines where the parent curve trimming begins. It can be anywhere along the line. It is easiest to begin the trimming at the origin but could be anywhere. The `SegmentStart` attribute is 0.0 in this case.

`IfcCurveSegment.SegmentLength` defines where the parent curve trimming ends relative to `SegmentStart`. This is the length of the curve we want to trim. The `SegmentLength` attribute is 1956.785654 for this example.

The `SegmentStart` and `SegmentLength` attributes trims a portion of the `IfcLine` parent curve, which is oriented in the direction (1,0) with origin at (0,0).

The trimmed portion of the curve needs to be placed and oriented into the horizontal alignment. The tangent segment begins at (500,2500) and runs in the direction 5.70829654085293 radian.

From the segment direction,

$$dy = \sin(5.70829654085293) = -0.54374144087698$$

$$dx = \cos(5.70829654085293) = 0.839252789970355$$

The segment placement is

```
#31=IFCCARTESIANPOINT((500.,2500.));
#49=IFCDIRECTION((0.839252789970355,-0.54374144087698));
#50=IFCAXIS2PLACEMENT2D(#31,#49);
```

The `IfcCurveSegment` is be defined as:

```
#55=IFCCURVESEGMENT(.CONTSAMEGRADIENT.,#50,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(1956.785654),#54);
```

2.3.3 Compute Point on Curve

Compute the position matrix for a point 1500 m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

From the `IfcCurveSegment.Placement`:

$$x = 500, y = 2500$$

$$dx = 0.839252789970355,\ dy = -0.54374144087698$$

$$M_{CSP} = \begin{matrix} 0.839252789970355 & 0.54374144087698 & 0 & 500 \\ -0.54374144087698 & 0.839252789970355 & 0 & 2500 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = [I]$.

Step 3 — Evaluate and map each point

Evaluate the parent curve at $u = 100$

$$\lambda(u) = C + \left(\int_0^{u=L} \cos\left(\theta(t)\right) dt \; x, \; \int_0^{u=L} \sin\left(\theta(t)\right) dt \; y \right)$$

$$\theta(100) = \arctan\left(\frac{0}{1}\right) = 0$$

$$x = 0 + \cos(0) \int_0^{100} dt = 0 + 1(1500 \; m - 0 \; m) = 1500 \; m$$

$$y = 0 + \sin(0) \int_0^{100} dt = 0 + 0(1500 \; m - 0 \; m) = 0 \; m$$

Though it is much easier to use the following calculation

$$x(u) = p_x + u(dx)$$

$$y(u) = p_y + u(dy)$$

$$x = 0 + 1(1500) = 1500$$

$$y = 0 - 0(1500) = 0$$

$$M_{PC} = \begin{matrix} 1 & 0 & 0 & 1500 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

The resulting position matrix is

$$M_h = M_{CSP} M_N M_{PC}$$

$$M_h = \begin{matrix} 0.839252789970355 & 0.54374144087698 & 0 & 500 \\ -0.54374144087698 & 0.839252789970355 & 0 & 2500 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} [I] \begin{matrix} 1 & 0 & 0 & 1500 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.4 Circular Arc

Circular alignment curves are geometrically represented by an arc trimmed from an `IfcCircle` parent curve.

2.4.1 Parent Curve Parametric Equations

The curve tangent angle, curvature, and point on the curve are given by the following equations

$$\theta(s) = \frac{s}{R}$$

$$\kappa(s) = \frac{1}{R}$$

$$x(s) = \int \cos\left(\theta(s)\right) ds$$

$$y(s) = \int \sin\left(\theta(s)\right) \; ds$$

2.4.2 Semantic Definition to Geometry Mapping

`IfcCircle` is defined by its center (`IfcCircle.Position`) and radius (`IfcCircle.Radius`).

Consider a horizontal curve segment that starts at (0,0), has a radius of 300 m, and an arc length of 100 m. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 300., 300., 100., $, .CIRCULARARC.);
```

The parent curve can be defined as follows:

```
#45 = IFCCIRCLE(#46, 300.);
#46 = IFCAXIS2PLACEMENT2D(#47, #48);
#47 = IFCCARTESIANPOINT((0., 300.));
#48 = IFCDIRECTION((0., -1.));
```

This is a circle centered at point (0,300) with the local X-axis aligned with the negative global Y-axis as shown in Figure 2.4.2-1.

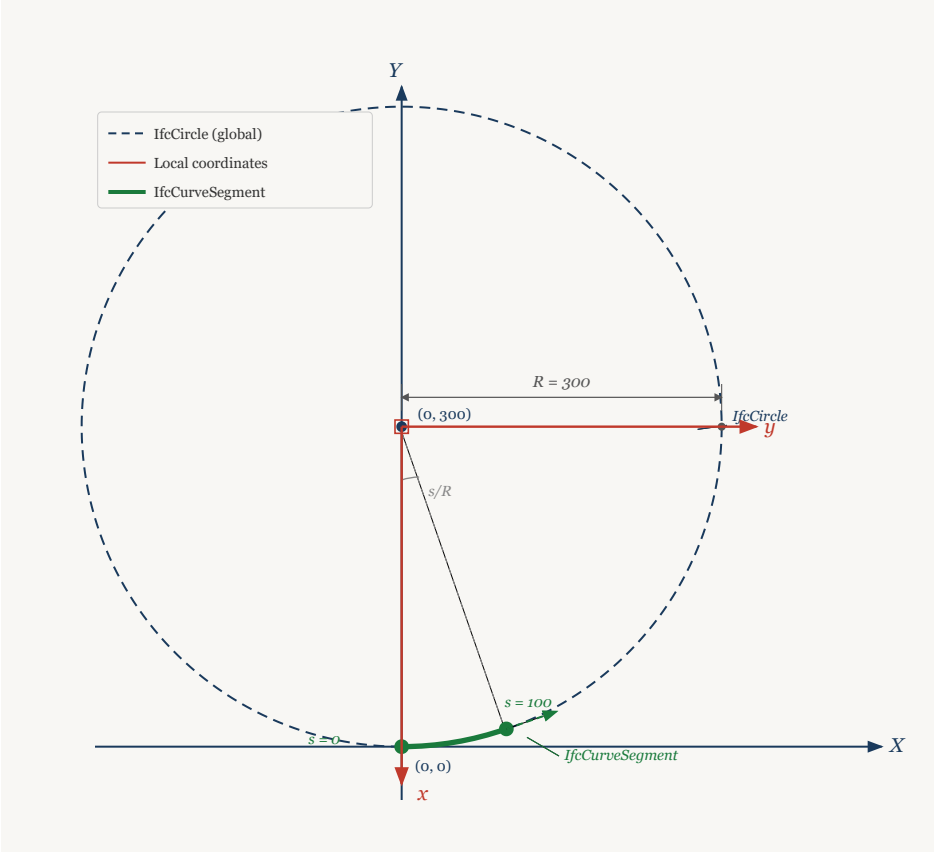


Figure 2.4.2-1 - Curve segment trimmed from an IfcCircle parent curve

The parent curve is placed such that a trim starting at 0.0 is at (0,0) and the tangent is in the direction (1,0).

The curve segment is defined by its placement at a segment trimmed from the parent curve starting at 0.0 for a length of 100.0 along the curve.

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
```

2.4.3 Compute Point on Curve

Compute the placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

$M_{CSP} = [I]$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

The trim begins where the local x-axis of the circle intersects the circumfrance. The circle is centered at (0,300) with radius = 300 and the local x-axis is in the direction of the global y-axis. This puts the trim start point at $x_0 = 0, y_0 = 0$ and the tangent direction (1,0) with $\theta_0 = 0$

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = [I]$.

Step 3 — Evaluate and map each point

Compute point on parent curve at $u = 50$

$$\theta(s) = \frac{s}{R} = \frac{50}{300}$$

$$d_x = \cos(\theta(s)) = \cos(\frac{50}{300}) = 0.98614323$$

$$d_y = \sin(\theta(s)) = \sin(\frac{50}{300}) = 0.16589613$$

$$x(s) = \int \cos\left(\theta(s)\right) ds = \int_0^{50} \cos\left(\frac{s}{300}\right) ds = 300 \sin\left(\frac{50}{300}\right) - 300 \sin\left(\frac{0}{300}\right) = 49.76883981$$

$$y(s) = \int \sin\left(\theta(s)\right)\, ds = \int_0^{50} \sin\left(\frac{s}{300}\right)\, ds = -300 \cos\left(\frac{50}{300}\right) - \left(-300 \cos\left(\frac{0}{300}\right)\right) = 4.1570305$$

$$M_{PC} = \begin{matrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

The resulting position matrix is

$$M_h = M_{CSP} M_N M_{PC}$$

$$M_h = [I]\,[I] \begin{matrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.5 Clothoid

Spiral transition curves are geometrically represented by a segment trimmed from one of the many spiral types in IFC. `IfcClothoid` geometry is used for clothoid curves.

2.5.1 Parent Curve Parametric Equations

The curve tangent angle, curvature, and point on the curve are given by the following equations

$$\theta(s) = \frac{\pi}{2} \frac{A}{|A|} s^2$$

$$\kappa(s) = \frac{A}{|A^3|} s$$

$$x(u) = A\sqrt{\pi}\int_0^u \cos\left(\frac{\pi}{2}\frac{A}{|A|}t^2\right)\,dt$$

$$y(u) = A\sqrt{\pi}\int_0^u \sin\left(\frac{\pi}{2}\frac{A}{|A|}t^2\right)\,dt$$

IFC provides a unit parametrization of clothoid curve.

$$u = \frac{s}{A\sqrt{\pi}}$$

with $-\infty < u < \infty$. When $\theta = \pi/2$, $u = 1.0$.

Care must be taken when evaluating clothoids because `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength` are defined with arc length. The details are illustreated in the example calculations.

2.5.2 Semantic Definition to Geometry Mapping

Consider a horizontal clothoid segment that starts at (0,0) with a start direction of (1.0,0.0). The radii at the start and end of the segment are 300 and 1000, respectively. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 300., 1000., 100., $, .CLOTHOID.);
```

From the semantic definition, compute the clothoid constant, A

$$R_s = 300,\ R_e = 1000,\ L = 100$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{1000} - \frac{100}{300} = -0.23333$$

$$A = \frac{L}{\sqrt{|f|}}\frac{f}{|f|} = \frac{100}{\sqrt{|-0.23333|}}\frac{-0.23333}{|-0.23333|} = -207.0196678$$

Place the parent curve at (0,0) with a tangent direction of (1,0)

```
#45 = IFCCLOTHOID(#46, -207.019667802706);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

The curve segment starts where the radius is 300. A positive radius indicates a counter-clockwise curve which is a curve towards the left. The distance from the origin where this radius occurs can be computed from the curvature equation. The curvature at the start is 1/300 and the clothoid constant is -207.0196678. Solve for distance from origin.

$$\kappa(s) = \frac{A}{|A^3|}s$$

$$\frac{1}{300} = \frac{-207.0196678}{-207.0196678^3}s$$

$$s = -142.8571429$$

Figure 2.5.2-1 shows the parent curve clothoid with the segment having $\kappa_s = \frac{1}{300}$ and $\kappa_e = \frac{1}{1000}$ highlighted. This is the segment that `IfcCurveSegment` must trim from the parent curve. The negative clothoid constant results in the parent curve in quadrants 2 and 4. The curvature is positive in quadrant 2. For this reason, the `SegmentStart` attribute is negative. From ths starting point, the trimming progresses to a larger radius, which is towards the origin. For this reason `SegmentLength` is a positive value. If the trimming were to progress to a smaller radius, `SegmentLength` would be a negative value. The `SegmentStart` and `SegmentLength` attributes can be positive and negative values in any combination. This is true for all parent curve types.

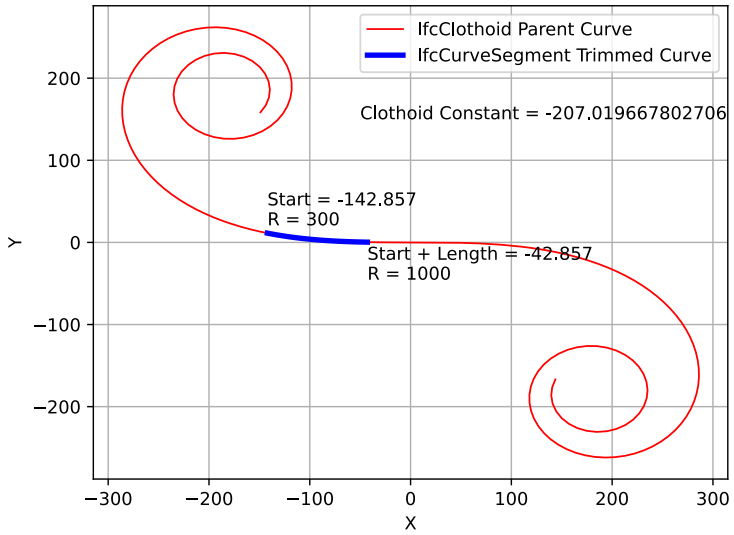


Figure 2.5.2-1 - `IfcCurveSegment` trimmed from an `IfcClothoid` parent curve

Place the curve segment at (0,0) with the tangent in the direction (1,0).

```
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
```

Define the curve segment

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(-142.857142857143), IFCLENGTHMEASURE(100.), #45);
```

2.5.3 Compute Point on Curve

Compute the placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix. This is not true in all cases.

$$M_{CSP} = [I]$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Start by computing the point and curve tangent at the start of the parent curve trim.

Recall that the clothoid equations are in terms of a unit parameterization. Compute the parametric position on the curve.

$$u = \frac{-142.8571428}{-207.0196678\sqrt{\pi}} = -0.3893278$$

Compute the point on curve tangent direction

$$\theta_0(-0.3893278) = \frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}(-0.3893278)^2 = -0.238095237$$

and compute the point on the curve

$$x_0(-0.3893278) = -207.0196678\sqrt{\pi}\int_0^{-0.3893278}\cos\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}t^2\right)\,dt = -142.04941746210602\ m$$

$$y_0(-0.3893278) = -207.0196678\sqrt{\pi}\int_0^{-0.3893278}\sin\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}t^2\right)\,dt = 11.292042785713347\ m$$

$$\cos(-0.238095237) = 0.971788979$$

$$\sin(-0.238095237) = -0.235852028$$

$$-(-142.04941746210602\ m)\cos(-0.238095237) - (11.292042785713347\ m)\sin(-0.238095237) = 140.705309648\ m$$

$$(-142.04941746210602\ m)\sin(-0.238095237) - (11.292042785713347\ m)\cos(-0.238095237) = 22.529160405\ m?$$

$$M_N = \begin{matrix} \cos\theta_0 & \sin\theta_0 & 0 & -x_0\cos\theta_0 - y_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & x_0\sin\theta_0 - y_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} = \begin{matrix} 0.971788979 & -0.235852028 & 0 & 140.705309648 \\ 0.235852028 & 0.971788979 & 0 & 22.529160405 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate and map each point

Compute point and curve tangent at 50 m from the start,

$$s = -142.8571428 + 50 = -92.8571428$$

$$u = \frac{-92.8571428}{-207.0196678\sqrt{\pi}} = -0.25306307$$

$$x(-0.25306307) = -207.0196678\sqrt{\pi}\int_0^{-0.25306307}\cos\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}s^2\right)\,ds = -92.763220844871512$$

$$y(-0.25306307) = -207.0196678\sqrt{\pi}\int_0^{-0.25306307}\sin\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}s^2\right)\,ds = 3.1114126254443812$$

$$\theta(-0.25306307) = \frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}(-0.25306307)^2 = -0.100595238$$

$$dx = \cos(-0.100595238) = 0.994944564$$

$$dy = \sin(-0.100595238) = -0.100425663$$

In matrix form

$$M_{PC} = \begin{matrix} 0.994944564 & 0.100425663 & 0 & -92.763220844871512 \\ -0.100425663 & 0.994944564 & 0 & 3.1114126254443812 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP}M_NM_{PC} = [I] \begin{matrix} 0.971788979 & -0.235852028 & 0 & 140.705309648 \\ 0.235852028 & 0.971788979 & 0 & 22.529160405 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} \begin{matrix} 0.994944564 & 0.100425663 & 0 & -92.76322084 \\ -0.100425663 & 0.994944564 & 0 & 3.111412625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.99056175921247780 & -0.13706714116038599 & 0 & 49.825200930152405 \\ 0.13706714116038599 & 0.99056175921247780 & 0 & 3.6744032279728032 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.6 Cubic Transition Curve

`IfcPolynomalCurve` is the geometric type for cubic transition spirals.

2.6.1 Parent Curve Parametric Equations

The approach for calculating the coordinates of a horizontal cubic curve is unique compared to the other curve types. For a horizontal alignment, the coordinate point and curve tangent angle is generally defined by parametric equations where the parameter is the distance along the curve. A cubic curve is represented with the `IfcPolynomialCurve` parent curve type. When `IfcPolynomialCurve` is used as a cubic transition curve, the x coefficients are [0,1] and the y coefficients are [0,0,0, A_3] resulting in a function of the form $y(x) = A_3 x^3$. See 4.2.2.1.5 Cubic Transition Segment.

The distance along a curve is defined by

$$d = \int \sqrt{(y'(x))^2 + 1}\, dx$$

$$y'(x) = 3A_3 x^2$$

$$d = \int \sqrt{9A_3^2 x^4 + 1}\, dx$$

This equation is solved for x and then y can be computed. This can be accomplished with numerical methods.

1. For a distance u along the curve, find x for $d - u = 0$, within a tolerance consistent with the modeled elements. See Section 12 for a discussion on tolerances.
2. Compute $y(x) = A_3 x^3$
3. Compute curve tangent slope as $\frac{dy}{dx} = 3A_3 x^2$

2.6.2 Semantic Definition to Geometry Mapping

Consider a horizontal cubic transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $,.CUBIC.);
```

Compute the polynomial curve coefficients

$$A_0 = A_1 = A_2 = 0$$

$$A_3 = \frac{1}{6R_e L} - \frac{1}{6R_s L} = \frac{1}{6(300\ m)(100\ m)} - \frac{1}{6(\infty)(100\ m)} = 5.55555 \cdot 10^{-6}\ m^{-2}$$

The geometric representation is

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCPOLYNOMIALCURVE(#46, (0., 1.), (0., 0., 0., 5.55555555555556E-6), $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

—-

:warning:

There is a flaw in the IFC Specification. IfcAlignmentHorizontalSegment indicates the cubic formula is $y = \frac{x^3}{6RL}$ which means the `IfcPolynomialCurve.CoefficentY[3]` attribute must have unit of $Length^{-2}$. This is a direct contradiction to IfcPolynomialCurve which clearly states the coefficent are real values (i.e. scalar values) with `IfcReal`.

Why is this a problem? The calculation of a point on the polynomal curve is different depending on how the coefficients are defined.

The parametric form of the polynomial curve is $\lambda(u) = (x(u), y(u), z(u))$. The polynomial curve represents real geometry so the resulting values, x, y, and z must have units of $Length$.

When u is parametrized as a Length measure, as required by IfcCurveSegment, the parametric terms of the polynomal equation are:

$$x(u) = \sum_{i=0}^{l} a_i u^i$$

$$y(u) = \sum_{j=0}^{m} b_j u^j$$

$$z(u) = \sum_{k=0}^{n} c_k u^k$$

For x, y, and z to have values in $Length$ measure, the implicit unit of measure of the coefficients must be:

$Length^{(1-i)}$ for a_i

$Length^{(1-j)}$ for b_j

$Length^{(1-k)}$ for c_k

When u is parametrized as a scalar, as required by IfcPolynomialCurve, the parametric terms of the polynomial equation are:

$$x(u) = L \sum_{i=0}^{l} a_i u^i$$

$$y(u) = L \sum_{j=0}^{m} b_j u^j$$

$$z(u) = L \sum_{k=0}^{n} c_k u^k$$

The parameter u and the coefficients are scalar so they must be multipled with the curve length L to result values of Length.

The equations for x, y, and z differ depending on the parameterization.

Concept Template 4.2.2.1.5 Cubic Transition Segment and IfcAlignmentHorizontalSegment provide clear direction (not withstanding typographical errors) that `IfcPolynomalCurve` is to be evaluated as $y = CoefficientsY[3]x^3$ which dictates that $CoefficientsY[3]$ must be in units of $Length^{-2}$ and must be encoded in the `IfcPolynomialCurve.CoefficientY` attribute as an `IfcReal`.

An offical Implementation Agreement doesn't seem to exist, however, this interpetation is consistent with discussions on GitHub.

:warning:

—-

2.6.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = [I]$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = [I]$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

From the `IfcPolynomalCurve` definition,

X Coefficients = ($0m^1$, $1m^0$)

Y Coefficients = ($0m^1$, $0m^0$, $0m^{-1}$, $5.55555 \cdot 10^{-6}m^{-2}$)

$$y(x) = \left(5.55555 \cdot 10^{-6}m^{-2}\right)x^3$$

Find x such that

$$|d - u| = \int_0^x \left(\sqrt{\left(y'(x)\right)^2 + 1}\right)dx - u \ < 10^{-4} \approx 0$$

$$y'(x) = 3\left(5.55555 \cdot 10^{-6}m^{-2}\right)x^2$$

Solve numerically

$$x = 99.72593255m$$

Check solution

$$d = \int_0^{99.72593255\ m} \sqrt{\left(3 \cdot (5.55555 \cdot 10^{-6}m^{-2})(x\ m)^2\right)^2 + 1}dx = 100\ m$$

Compute y

$$y(x) = (5.55555 \cdot 10^{-6}m^{-2})(99.72593255\ m)^3 = 5.5100\ m$$

The tangent vector at $u = 100\ m$ along the curve is

$$y'(99.72593255\ m) = 3\left(5.55555 \cdot 10^{-6}m^{-2}\right)(99.72593255\ m)^2 = 0.165753$$

Normalizing the tangent vector

$$dx = \frac{1}{\sqrt{1^2 + 0.615753^2}} = 0.986539$$

$$dy = \frac{0.165753}{\sqrt{1^2 + 0.615753^2}} = 0.1635219$$

The resulting matrix is

$$M_{PC} = \begin{matrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP}\ M_N\ M_{PC} = [I]\ [I] \begin{matrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.7 Helmert Transition Curve

The Helmert curve is unique in that it is semantically defined as a single layout segment (`IfcAlignmentHorizontalSegment`) but geometrically defined with two curve segments (`IfcCurveSegment`), each representing half of the transition curve. The parent curve for a helmert transition curve is `IfcSecondOrderPolynomialSpiral` .

2.7.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \frac{t^3}{3A_2^3} + \frac{A_1}{2\ A_1^3}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{1}{A_2^3}t^2 + \frac{A_1}{A_1^3}t + \frac{1}{A_0}$$

The polynomial coefficietions carry a second subscript to indicate first half, 1, and second half, 2. For example, A_{21} is coefficient A_2 for the first half and A_{02} is coefficient A_0 for the second half.

First Half

$$a_{01} = \frac{L}{R_s},\ A_{01} = \frac{L}{|a_0|}\ \frac{a_0}{|a_0|}$$

$$a_{11} = 0,\ A_{11} = 0$$

$$a_{21} = 2f,\ A_{21} = \frac{L}{\sqrt[3]{|a_2|}}\ \frac{a_2}{|a_2|}$$

Second Half

$$a_{02} = -f + \frac{L}{R_s},\ A_{02} = \frac{L}{|a_0|}\ \frac{a_0}{|a_0|}$$

$$a_{12} = 4f,\ A_{12} = \frac{L}{\sqrt{|a_1|}}\ \frac{a_1}{|a_1|}$$

$$a_{22} = -2f,\ A_{22} = \frac{L}{\sqrt[3]{|a_2|}}\ \frac{a_2}{|a_2|}$$

2.7.2 Semantic Definition to Geometry Mapping

Consider a horizontal Helmert transition curve segment that starts at (0,0) with a start direction of (1,0). The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $, .HELMERTCURVE.);
```

Compute the curve parameters

$$L = 100\ m$$

$$R_s = \infty$$

$$R_e = 300\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

First Half

$$a_{01} = \frac{100}{\infty} = 0,\ A_0 = 0$$

$$a_{00} = 0,\ A_{11} = 0$$

$$a_{21} = 2(0.33333) = 0.66667,\ A_2 = \frac{100\ m}{\sqrt[3]{|0.66667|}} \frac{0.66667}{|0.66667|} = 114.4714255\ m$$

The geometric representation of the first half is

```
#36 = IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(50.), #45);
#37 = IFCLOCALPLACEMENT($, #38);
#38 = IFCAXIS2PLACEMENT3D(#39, $, $);
#39 = IFCCARTESIANPOINT((0., 0., 0.));
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCSECONDORDERPOLYNOMIALSPIRAL(#46, 114.471424255333, $, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

> Note: when a polynomial coefficient is zero, the correspond term is unused and coded as `$` in the IFC entity

Second Half

$$a_{02} = -0.33333 + \frac{100}{\infty} = -0.33333,\ A_0 = \frac{100\ m}{|-0.33333|} \frac{-0.33333}{|-0.33333|} = -300\ m$$

$$a_{12} = 4(0.33333) = 1.33333,\ A_1 = \frac{100\ m}{\sqrt{|1.33333|}} \frac{1.33333}{|1.33333|} = 86.602540378\ m$$

$$a_{22} = -2(0.33333) = -0.66667,\ A_2 = \frac{100\ m}{\sqrt[3]{|-0.66667|}} \frac{-0.66667}{|-0.66667|} = -114.4714255\ m$$

Figure 2.7.2-1 shows the first and second half parent curves, without any adjustments.

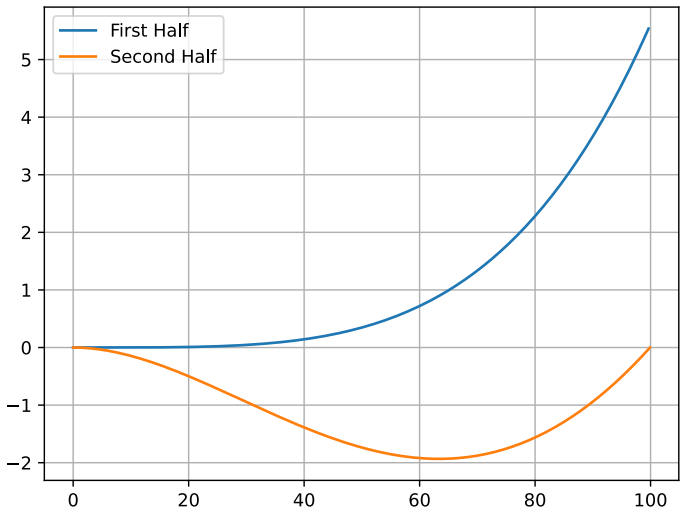


Figure 2.7.2-1 First and second half of parent curves, without placement adjustments

Two-level placement for the second half

`IfcCurveSegment` evaluation normalizes the parent curve at `SegmentStart`, then applies the curve segment `Placement`. For the second half, `SegmentStart = L/2`, so the second half parent curve's value at $t = L/2$ is the normalization origin. The second half parent curve's own `Position` attribute is therefore set so that the point at $L/2$ exactly matches the endpoint of the first half.

With this alignment the normalization cancels the offset, and the curve segment `Placement` supplies only the world-space join position.

Level	Attribute	Purpose
`IfcSecondOrderPolynomialSpiral.Position`	$(x_p,\ y_p,\ \theta_p)$	Shift/rotate the raw spiral so its value at $t = L/2$ equals (x_1, y_1, θ_1)
`IfcCurveSegment.Placement`	World-space join point	Place the normalized second-half curve at the correct position in the composite curve

Table 2.7.2-1 — Two-level placement for the second Helmert half

Finding the parent curve Position

Evaluate the raw second half spiral (with `Position` at origin) at $t = L/2$ to obtain (x_2, y_2, θ_2). The rigid-body transform that maps this to (x_1, y_1, θ_1) is

$$\theta_p = \theta_1 - \theta_2 = 0.0277777777 - (-0.0277777777) = 0.05555555$$

$$x_p = x_1 - x_2\cos\theta_p + y_2\sin\theta_p = 49.9972 - 49.9664\cos(0.0555555) - 1.73556\sin(0.0555555) = 0.011503\ m$$

$$y_p = y_1 - x_2\sin\theta_p - y_2\cos\theta_p = 0.347204 - 49.9664\sin(0.055555) + 1.73556\cos(0.0555555) = -0.6943693\ m$$

$$dx_p = \cos(\theta_p) = \cos(0.055555555) = 0.998457$$

$$dy_p = \sin(\theta_p) = \sin(0.055555555) = 0.055527$$

Figure 2.7.2-2 shows the same first and second half parent curves, with the origin of the second half curve translated (x_p, y_p) and rotated so that the curve tangent is (dx_p, dy_p)

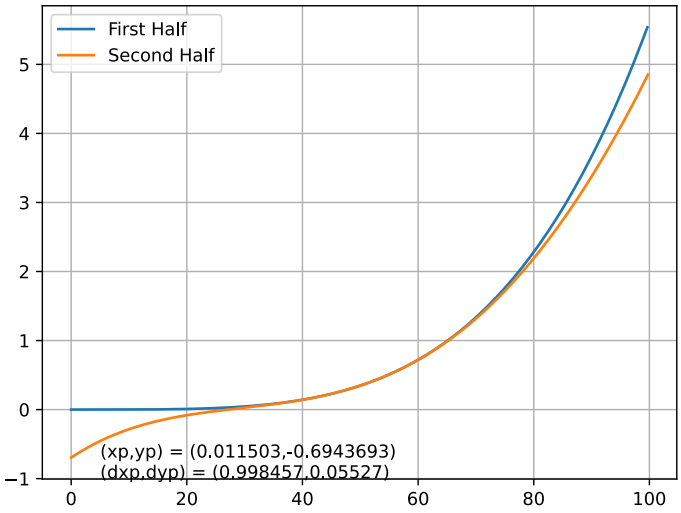


Figure 2.7.2-2 - First and second half parent curves with second half curve positioned

Finding the curve segment Placement

The `Placement` is the world-space position and direction at the join point, obtained by rotating (x_1, y_1) by the overall segment start direction θ_s and translating by the start point (x_s, y_s):

$$x_{join} = x_s + x_1 \cos\theta_s - y_1 \sin\theta_s$$

$$y_{join} = y_s + x_1 \sin\theta_s + y_1 \cos\theta_s$$

$$\theta_{join} = \theta_s + \theta_1$$

For the example with $\theta_s = 0$:

$$x_{join} = 0.0 + 49.99724\cos(0.0) - 0.347204\sin(0.0) = 49.99724 \text{ m}$$

$$y_{join} = 0.0 + 49.99724\sin(0.0) + 0.347204\cos(0.0) = 0.347204 \text{ m}$$

$$\theta_{join} = 0.0 + 0.02777777 = 0.02777777 \text{ rad}$$

The `RefDirection` of the curve segment `Placement` is $(\cos\theta_{join}, \sin\theta_{join})$:

$$\cos\theta_{join} = \cos(0.02777777) = 0.999614$$

$$\sin\theta_{join} = \sin(0.02777777) = 0.027774$$

The geometric representation of the second half

```
#48 = IFCCURVESEGMENT(.CONTINUOUS., #49, IFCLENGTHMEASURE(50.), IFCLENGTHMEASURE(50.), #52);
#49 = IFCAXIS2PLACEMENT2D(#50, #51);
#50 = IFCCARTESIANPOINT((49.9972443634885, 0.347204361427475));
#51 = IFCDIRECTION((0.999614222337484, 0.0277742056705088));
#52 = IFCSECONDORDERPOLYNOMIALSPIRAL(#53, -114.471424255333, 86.6025403784439, -300.);
#53 = IFCAXIS2PLACEMENT2D(#54, #55);
#54 = IFCCARTESIANPOINT((0.0115725277555399, -0.694297741112199));
#55 = IFCDIRECTION((0.998457186998745, 0.0555269820047339));
```

Figure 2.7.2-3 shows the trimmed and positioned first and second half parent curves resulting in the full Helmert transition curve.

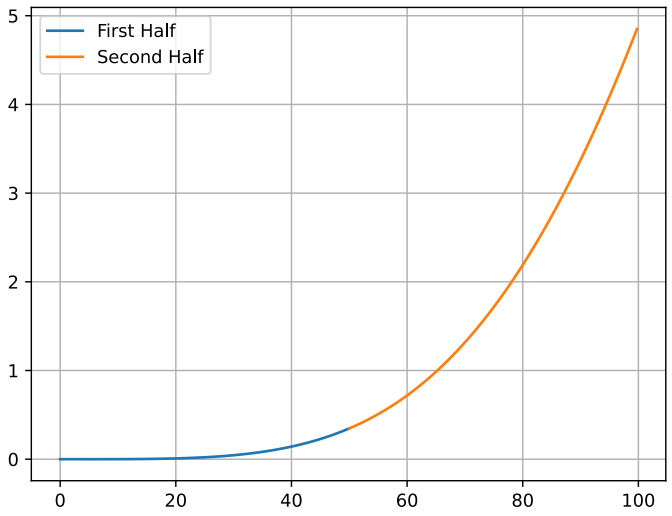


Figure 2.7.2-3 - Final Helmert transition curve

2.7.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$ (the end of the full Helmert segment).

The point falls in the second half ($50 < u \leq 100$). The parent curve parameter is $t = \text{SegmentStart} + (u - 50) = 50 + 50 = 100$.

Step 1 — Form the curve segment placement matrix M_{CSP}

The curve segment `Placement` for the second half is at $(x_{join}, y_{join}) = (49.9972,\ 0.347204)$ with direction $\theta_{join} = 0.027778$ rad:

$$M_{CSP} = \begin{matrix} 0.999614 & -0.027774 & 0 & 49.9972 \\ 0.027774 & 0.999614 & 0 & 0.347204 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the second half parent curve at SegmentStart and form the normalization matrix M_N

The second half parent curve has `Position` at $(x_p, y_p) = (0.011503,\ -0.694370)$ with direction $\theta_p = 0.055556$ rad. This was chosen so that the parent curve at $t = L/2 = 50$ yields $(x_1, y_1, \theta_1) = (49.9972,\ 0.347204,\ 0.027778)$ — the endpoint of the first half.

M_N maps (x_1, y_1, θ_1) to the origin with the x-axis tangent direction:

$$M_N = \begin{matrix} \cos\theta_1 & \sin\theta_1 & 0 & -x_1\cos\theta_1 - y_1\sin\theta_1 \\ -\sin\theta_1 & \cos\theta_1 & 0 & x_1\sin\theta_1 - y_1\cos\theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_N = \begin{matrix} 0.999614 & 0.027774 & 0 & -50.0 \\ -0.027774 & 0.999614 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate and map each point

Evaluate the second half parent curve at $t = 100$. The raw spiral (before applying `Position`) uses $(A_{0,2},\ A_{1,2},\ A_{2,2}) = (-300,\ 86.6025,\ -114.4714)$:

$$\theta_{raw}(t) = \frac{t^3}{3(-114.4714)^3} + \frac{86.6025}{2\ \ 86.6025^3}t^2 + \frac{t}{-300}$$

$$\theta_{raw}(100) = -0.22222 + 0.66666 - 0.33333 = 0.11111 \text{ rad}$$

$$x_{raw} = \int_0^{100} \cos\theta_{raw}(t)\,dt = 99.8942272 \text{ m}$$

$$y_{raw} = \int_0^{100} \sin\theta_{raw}(t)\,dt = -0.00146765266 \text{ m}$$

Apply the parent curve `Position` (x_p, y_p, θ_p):

$$x_{pos} = x_p + x_{raw}\cos\theta_p - y_{raw}\sin\theta_p = 0.011503 + 99.8942272(0.998457) - (-0.00146765266)(0.055527) = 99.7517 \text{ m}$$

$$y_{pos} = y_p + x_{raw}\sin\theta_p + y_{raw}\cos\theta_p = -0.694370 + 99.8942272(0.055527) + (-0.00146765266)(0.998457) = 4.8510 \text{ m}$$

$$\theta_{pos}(100) = \theta_p + \theta_{raw}(100) = 0.055556 + 0.11111 = 0.16\overline{6} \text{ rad}$$

Note: $\theta_{pos}(100) = \frac{1}{6} = \frac{L}{2R_c} = \frac{100}{2\times300}$, confirming the total tangent angle change matches a Clothoid with the same endpoints.

Form M_{PC} from the parent curve point at $t = 100$:

$$M_{PC} = \begin{matrix} \cos\theta_{pos} & -\sin\theta_{pos} & 0 & x_{pos} \\ \sin\theta_{pos} & \cos\theta_{pos} & 0 & y_{pos} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} = \begin{matrix} 0.98615 & -0.16590 & 0 & 99.8942272 \\ 0.16590 & 0.98615 & 0 & 4.8510 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Apply normalization and curve segment placement:

$$M_h = M_{CSP}\, M_N\, M_{PC}$$

$$M_h = \begin{matrix} 0.999614 & -0.027774 & 0 & 49.9972 & 0.999614 & 0.027774 & 0 & -50.0 & 0.98615 & -0.16590 & 0 & 99.8942272 \\ 0.027774 & 0.999614 & 0 & 0.347204 & -0.027774 & 0.999614 & 0 & 0 & 0.16590 & 0.98615 & 0 & 4.8510 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.98614323 & -0.16589613 & 0. & 99.75176295 \\ 0.16589613 & 0.98614323 & 0. & 4.85106157 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.8 Bloss Transition Curve

`IfcThirdOrderPolynomialSpiral` is the geometric type for Bloss transition spirals.

2.8.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \frac{A_3}{4\ A_3^5}t^4 + \frac{1}{3A_2^3}t^3 + \frac{A_1}{2\ A_1^3}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{A_3}{A_3^5}t^3 + \frac{t^2}{A_2^3} + \frac{A_1}{2\ A_1^3}t + \frac{1}{A_0}$$

2.8.2 Semantic Definition to Geometry Mapping

Consider a horizontal Bloss transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $, .BLOSSCURVE.);
```

Compute the polynomial spiral parameters

$$L = 100\ m$$

$$R_s = \infty$$

$$R_e = 300\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

$$A_1 = 0$$

$$a_2 = 3f = 3(0.33333) = 1$$

$$A_2 = \frac{L}{\sqrt[3]{|a_2|}}\frac{a_2}{|a_2|} = \frac{100}{\sqrt[3]{|1|}}\frac{1}{|1|} = 100\ m$$

$$a_3 = -2f = -2(0.33333) = -0.66667$$

$$A_3 = \frac{L}{\sqrt[4]{|a_3|}}\frac{a_3}{|a_3|} = \frac{100\ m}{\sqrt[4]{|-0.66667|}}\frac{-0.66667}{|-0.66667|} = -110.668192\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCTHIRDORDERPOLYNOMIALSPIRAL(#46, -110.668191970032, 100., $, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.8.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 50$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = [I]$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = [I]$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$x = \int_0^{50} \cos\theta(s)\, ds = 49.9962109$$

$$y = \int_0^{50} \sin\theta(s)\, ds = 0.416638875$$

$$\theta(50) = \frac{-110.668192}{4\ -110.668192^5}(50)^4 + \frac{1}{3(100)^3}(50)^3 = 0.03125$$

$$dx = \cos(0.03125) = 0.999512$$

$$dy = \sin(0.03125) = 0.031245$$

In matrix form

$$M_{PC} = \begin{matrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = [I]\ [I] \begin{matrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.9 Cosine Transition Curve

`IfcCosineSpiral` is the geometric type for cosine transition spirals.

2.9.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\kappa(t) = \frac{1}{A_0} + \frac{1}{A_1}\cos\left(\frac{\pi}{L}t\right)$$

$$\theta(t) = \frac{1}{A_0}t + \frac{L}{\pi A_1}\sin\left(\frac{\pi}{L}t\right)$$

2.9.2 Semantic Definition to Geometry Mapping

Consider a horizontal cosine transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $, .COSINECURVE.);
```

Compute the cosine spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

Constant Term:

$$a_0 = 0.5f + \frac{L}{R_s} = 0.5(0.33333) + \frac{100}{\infty} = 0.16667$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = \frac{100\ m}{|0.16667|}\ \frac{0.16667}{|0.16667|} = 600\ m$$

Cosine Term:

$$a_1 = -0.5f = -0.5(0.33333) = -0.16667$$

$$A_1 = \frac{L}{|a_1|}\frac{a_1}{|a_1|} = \frac{100}{|-0.16667|}\ \frac{-0.16667}{|-0.16667|} = -600\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCCOSINESPIRAL(#46, -600., 600.);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.9.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = [I]$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = [I]$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(t) = \frac{1}{600}t - \frac{100}{600\pi}\sin\left(\frac{\pi}{100}t\right)$$

$$x = \int_0^{100} \cos\theta(t)\, dt = 99.7485\ m$$

$$y = \int_0^{100} \sin\theta(t)\, dt = 4.9458\ m$$

$$\theta(100) = \frac{1}{600}100 - \frac{100}{600\pi}\sin\left(\frac{\pi}{100}100\right) = \frac{1}{6} = 0.1666667$$

$$dx = \cos\left(\frac{1}{6}\right) = 0.98614$$

$$dy = \sin\left(\frac{1}{6}\right) = 0.16589$$

In matrix form

$$M_{PC} = \begin{matrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP}M_N M_{PC} = I\ I \begin{matrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.10 Sine Transition Curve

`IfcSineSpiral` is the geometric type for sine transition spirals.

2.10.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\kappa(t) = \frac{1}{A_0} + \frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 t + \frac{1}{A_2}\sin\left(\frac{2\pi}{L}t\right)$$

$$\theta(t) = \frac{1}{A_0}t + \frac{1}{2}\left(\frac{A_1}{|A_1|}\right)\left(\frac{t}{A_1}\right)^2 - \frac{L}{2\pi A_2}\left(\cos\left(\frac{2\pi}{L}t\right) - 1\right)$$

$$x = \int_0^{L} \cos\theta(t)\, dt$$

$$y = \int_0^{L} \sin\theta(t)\, dt$$

2.10.2 Semantic Definition to Geometry Mapping

Consider a horizontal sine transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $, .SINECURVE.);
```

Compute the sine spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

Constant Term:

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_o = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

Linear Term:

$$a_1 = f = 0.33333$$

$$A_1 = \frac{L}{\sqrt{|a_1|}}\frac{a_1}{|a_1|} = \frac{100}{\sqrt{|0.33333|}}\frac{0.33333}{|0.33333|} = 173.2050808\ m$$

Sine Term:

$$a_2 = -\frac{1}{2\pi}f = -\frac{1}{2\pi}(0.33333) = -0.053051647$$

$$A_2 = \frac{L}{|a_2|}\frac{a_2}{|a_2|} = \frac{100}{|-0.053051647|}\frac{-0.053051647}{|-0.053051647|} = -1884.955592\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCSINESPIRAL(#46, -1884.95559215388, 173.205080756888, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.10.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = [I]$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = [I]$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(t) = \frac{1}{2}\left(\frac{t}{173.2050808}\right)^2 + \frac{100}{2\pi(1884.955592)}\left(\cos\left(\frac{2\pi}{100}t\right) - 1\right)$$

$$x = \int_0^{100} \cos\theta(t)\, dt = 99.75698\ m$$

$$y = \int_0^{100} \sin\theta(t)\, dt = 4.70132\ m$$

$$\theta(100) = \frac{1}{2}\left(\frac{100}{173.2050808}\right)^2 + \frac{100}{2\pi(1884.955592)}\left(\cos\left(\frac{2\pi}{100}100\right) - 1\right) = \frac{1}{6} = 0.1666667$$

$$dx = \cos\left(\frac{1}{6}\right) = 0.98614$$

$$dy = \sin\left(\frac{1}{6}\right) = 0.16589$$

In matrix form

$$M_{PC} = \begin{matrix} 0.98614 & -0.16589 & 0 & 99.75698 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = [I]\ [I] \begin{matrix} 0.98614 & -0.16589 & 0 & 99.75698 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.98614 & -0.16589 & 0 & 99.75698 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.11 Viennese Transition Curve

[todo: need a description of a viennese bend and its application in railroad. also need to note the unique feature that the cant parameters factor into the horizontal geometry. state the parent curve type in this description]

Parent curve type: `IfcSeventhOrderPolynomialSpiral`

2.11.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(s) = \frac{A_7}{8\ A_7^9}s^8 + \frac{1}{7A_6^7}s^7 + \frac{A_5}{6\ A_5^7}s^6 + \frac{1}{5A_4^5}s^5 + \frac{A_3}{4\ A_3^5}s^4 + \frac{1}{3A_2^3}s^3 + \frac{A_1}{2\ A_1^3}s^2 + \frac{1}{A_0}s$$

$$\kappa(s) = \frac{A_7}{A_7^9}s^7 + \frac{1}{A_6^7}s^6 + \frac{A_5}{A_5^7}s^5 + \frac{1}{A_4^5}s^4 + \frac{A_3}{A_3^5}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{A_1^3}s + \frac{1}{A_0}$$

2.11.2 Semantic Definition to Geometry Mapping

Consider a horizontal Viennese Bend transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length is 100. The Gravity Center Line Height is 1.8 (this optional parameter is required

for the Viennese Bend). The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., 1.8, .VIENNESEBEND.);
```

Viennese Bend transition curves are unique in that the horizontal geometry of the curve depends on the cant. For the example, the cant segment horizontal length is 100, the start cant is 0.0 and the end cant is 0 at the left rail and 0.1 at the right rail. The semantic definition is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 0., 0.1, .VIENNESEBEND.);
```

h_{cg} = Gravity Center Line Height = `IfcAlignmentHorizontalSegment.GravityCenterLineHeight`

d_{rh} = Rail Head Distance = `IfcAlignmentCant.RaliHeadDistance`

D_{sl} = `IfcAlignmentCantSegment.StartCantLeft`

D_{el} = `IfcAlignmentCantSegment.EndCantLeft`

D_{sr} = `IfcAlignmentCantSegment.StartCantRight`

D_{er} = `IfcAlignmentCantSegment.EndCantRight`

$\beta_s = \frac{(D_{sr} - D_{sl})}{d_{rh}}$, Start Cross Slope

$\beta_e = \frac{(D_{er} - D_{el})}{d_{rh}}$, End Cross Slope

$cf = -420. \left(\frac{h_{cg}}{L}\right) (\beta_e - \beta_s)$, Cant Factor

Compute the polynomial spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

$$\beta_s = \frac{(0. - 0.)}{1.5} = 0.0$$

$$\beta_e = \frac{0.1 - 0.0}{1.5} = 0.066667$$

$$cf = -420 \left(\frac{1.8}{100}\right)(0.066667 - 0.) = -0.504$$

Constant term

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

Linear term

$$a_1 = 0$$

$$A_1 = 0$$

Quadratic term

$$a_2 = cf = -0.504$$

$$A_2 = \frac{L}{\sqrt[3]{|a_2|}} \frac{a_2}{|a_2|} = \frac{100}{\sqrt[3]{|-0.504|}} \frac{-0.504}{|-0.504|} = -125.6579069\ m$$

Cubic term

$$a_3 = -4cf = -4(-0.504) = 2.016$$

$$A_3 = \frac{L}{\sqrt[4]{|a_3|}} \frac{a_3}{|a_3|} = \frac{100}{\sqrt[4]{|2.016|}} \frac{2.016}{|2.016|} = 83.92229813\ m$$

Quartic term

$$a_4 = 5cf + 35f = 5(-0.504) + 35(0.33333) = 9.1466655$$

$$A_4 = \frac{L}{\sqrt[5]{|a_4|}} \frac{a_4}{|a_4|} = \frac{100}{\sqrt[5]{|9.1466655|}} \frac{9.1466655}{|9.1466655|} = 64.231406\ m$$

Quintic term

$$a_5 = -2cf - 84f = -2(-0.504) - 84(0.33333) = -26.9919999$$

$$A_5 = \frac{L}{\sqrt[6]{|a_5|}} \frac{a_5}{|a_5|} = \frac{100}{\sqrt[6]{|-26.9919999|}} \frac{-26.9919999}{|-26.9919999|} = -57.7378785\ m$$

Sextic term

$$a_6 = 70f = 70(0.33333) = 23.33333333$$

$$A_6 = \frac{L}{\sqrt[7]{|a_6|}} \frac{a_6}{|a_6|} = \frac{100}{\sqrt[7]{|23.33333333|}} \frac{23.33333333}{|23.33333333|} = 63.76388134\ m$$

Septic term

$$a_7 = -20f = -20(0.33333) = -6.66666666$$

$$A_7 = \frac{L}{\sqrt[8]{|a_7|}} \frac{a_7}{|a_7|} = \frac{100}{\sqrt[8]{|-6.66666666|}} \frac{-6.66666666}{|-6.66666666|} = -78.8880838\ m$$

```
#66 = IFCCURVESEGMENT(.CONTINUOUS., #72, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #75);
#72 = IFCAXIS2PLACEMENT2D(#73, #74);
#73 = IFCCARTESIANPOINT((0., 0.));
#74 = IFCDIRECTION((1., 0.));
#75 = IFCSEVENTHORDERPOLYNOMIALSPIRAL(#76, -78.8880838459446, 63.7638813456506, -57.7378785242934, 64.2314061308743, 83.922298125931, -125.657906854
#76 = IFCAXIS2PLACEMENT2D(#77, $);
#77 = IFCCARTESIANPOINT((0., 0.));
```

2.11.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = [I]$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = [I]$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(s) = \frac{A_7}{8\ A_7^9}s^8 + \frac{1}{7A_6^7}s^7 + \frac{A_5}{6\ A_5^7}s^6 + \frac{1}{5A_4^5}s^5 + \frac{A_3}{4\ A_3^5}s^4 + \frac{1}{3A_2^3}s^3 + \frac{A_1}{2\ A_1^3}s^2 + \frac{1}{A_0}s$$

$$x = \int_0^{100} \cos\theta(s)\ ds = 99.76319823871657$$

$$y = \int_0^{100} \sin\theta(s)\ ds = 4.499916129045404$$

$$\theta(100) = \frac{-78.8880838}{8\ |(-78.8880838)^9|}100^8 + \frac{1}{7(63.76388134)^7}100^7 + \frac{-57.7378785}{6\ |(-57.7378785)^7|}100^6 + \frac{1}{5(64.231406)^5}100^5 + \frac{83.92229813}{4\ |(83.92229813)^5|}$$

$$dx = \cos\left(0.1666666667\right) = 0.9861432315629198$$

$$dy = \sin\left(0.1666666667\right) = 0.16589613269344608$$

In matrix form

$$M_{PC} = \begin{matrix} 0.9861432315629198 & -0.16589613269344608 & 0 & 99.76319823871657 \\ 0.16589613269344608 & 0.9861432315629198 & 0 & 4.499916129045404 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP}M_N M_{PC} = [I]\ [I] \begin{matrix} 0.9861432315629198 & -0.16589613269344608 & 0 & 99.76319823871657 \\ 0.16589613269344608 & 0.9861432315629198 & 0 & 4.499916129045404 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_h = \begin{matrix} 0.9861432315629198 & -0.16589613269344608 & 0 & 99.76319823871657 \\ 0.16589613269344608 & 0.9861432315629198 & 0 & 4.499916129045404 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

2.12 Zero Length Segment

IFC Concept Templates 4.1.4.4.1.1, 4.1.4.4.1.2, 4.1.7.1.1.1, 4.1.7.1.1.2, 4.1.7.1.1.3, and 4.1.7.1.1.4 require that the final `IfcAlignmentSegment` nested within `IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant` has a `SegmentLength` of zero. A corresponding zero-length `IfcCurveSegment` is required in each associated geometric representation (`IfcCompositeCurve`, `IfcGradientCurve`, or `IfcSegmentedReferenceCurve`).

The specification imposes no constraint on which curve type backs the zero-length geometric segment. As best practice, use `.LINE.` (`IfcAlignmentHorizontalSegment`), `.CONSTANTGRADIENT.` (`IfcAlignmentVerticalSegment`), or `.CONSTANTCANT.` (`IfcAlignmentCantSegment`) as the `PredefinedType` of the semantic closing segment, and an `IfcLine` as the parent curve of the corresponding `IfcCurveSegment`. The placement of both the semantic and geometric closing segments must match the endpoint and end direction of the preceding segment.

2.12.1 Semantic Closing Segment

The `StartPoint` and `StartDirection` of the semantic segment are taken from the end state of the preceding segment, and `SegmentLength` is set to zero.

```
// Last non-zero length segment
#88=IFCCARTESIANPOINT((7790.93212831259,4006.73076454875));
#89=IFCALIGNMENTHORIZONTALSEGMENT($,$,#88,-1.23849516741382,0.,0.,2112.2850844257,$,.LINE.);

// Final segment, zero length.
#91=IFCCARTESIANPOINT((8480.,2010.));
#92=IFCALIGNMENTHORIZONTALSEGMENT($,$,#91,-1.23849516741382,0.,0.,0.,$,.LINE.);
```

2.12.2 Geometric Closing Segment

The zero-length `IfcCurveSegment` is placed at the endpoint of the alignment with its axis aligned to the end tangent direction. The parent curve is an `IfcLine` whose point and direction vector match that placement. `SegmentStart` and `SegmentLength` are both zero. Because the segment has zero length, the choice of `IfcLine` here has no geometric effect — it represents a point at the end of the alignment.

```
// Last non-zero length segment
#169=IFCCARTESIANPOINT((0.,0.));
#170=IFCDIRECTION((1.,0.));
#171=IFCVECTOR(#170,1.);
#172=IFCLINE(#169,#171);
#173=IFCDIRECTION((0.326219162729524,-0.945294164727599));
#174=IFCAXIS2PLACEMENT2D(#88,#173);
#175=IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE.,#174,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(2112.2850844257),#172);

// Final segment, zero length
#176=IFCCARTESIANPOINT((0.,0.));
#177=IFCDIRECTION((1.,0.));
#178=IFCVECTOR(#177,1.);
#179=IFCLINE(#176,#178);
#180=IFCDIRECTION((0.326219162729524,-0.945294164727599));
#181=IFCAXIS2PLACEMENT2D(#91,#180);
#182=IFCCURVESEGMENT(.DISCONTINUOUS.,#181,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(0.),#179);
```

Section 3 - Vertical Alignments 3.0 Introduction

A vertical alignment defines the elevation profile of a route as a function of distance along its horizontal path. Combined with the horizontal alignment, it establishes the three-dimensional geometry of a road, railway, or other linear infrastructure element.

3.1 General

The geometric representation of a vertical alignment is accomplished with `IfcGradientCurve`. This defines the vertical alignment in a "distance along, elevation" coordinate system. Distance along is measured along `IfcGradientCurve.BaseCurve` = `IfcCompositeCurve`.

Table 3.1-1 maps each `IfcAlignmentVerticalSegment.PredefinedType` to its corresponding parent curve type.

Business Logic (`IfcAlignmentVerticalSegment.PredefinedType`)	**Geometric Representation (`IfcCurveSegment.ParentCurve`)**
CONSTANTGRADIENT	`IfcLine`
CIRCULARARC	`IfcCircle`
CLOTHOID	`IfcClothoid`
PARABOLICARC	`IfcPolynomialCurve`

Table 3.1-1 — Mapping of business logic to geometric representation for vertical alignment

3.2 Curve Segment Evaluation Algorithm

Vertical segments are evaluated in a two-dimensional "distance along, elevation" coordinate system. In this system $x(s)$ is the distance measured along the horizontal `IfcCompositeCurve` and $y(s)$ is the elevation. The grade angle is $\theta(\ell) = \tan^{-1}(g(\ell))$ where $g(\ell)$ is the gradient.

Steps 1–3 follow the identical procedure described in Section 2.2 for horizontal segments, substituting distance along for the horizontal x-coordinate and elevation for the horizontal y-coordinate. Let s_0 = `IfcAlignmentVerticalSegment.StartDistAlong`.

Step 1 — Form the curve segment placement matrix M_{CSP}

M_{CSP} is constructed from `IfcCurveSegment.Placement`, where (d_p, z_p) is the `Location` (distance along, elevation) and θ_p is the grade angle of the `RefDirection`.

$$M_{CSP} = \begin{matrix} \cos\theta_p & -\sin\theta_p & 0 & d_p \\ \sin\theta_p & \cos\theta_p & 0 & z_p \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Compute the distance along $d_0 = x(s_0)$, elevation $z_0 = y(s_0) = \text{IfcAlignmentVerticalSegment.StartHeight}$, and grade angle $\theta_0 = \theta(s_0)$.

M_N simultaneously translates the trim-start point to the origin and rotates so that the tangent at s_0 aligns with the positive x-direction.

$$M_N = \begin{matrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate and map each point in the vertical plane

For the point at distance along s, compute $x(s)$, $y(s)$, and $\theta(\ell)$:

$$M_{PC} = \begin{matrix} \cos\theta(\ell) & -\sin\theta(\ell) & 0 & x(s) \\ \sin\theta(\ell) & \cos\theta(\ell) & 0 & y(s) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_v = M_{CSP} M_N M_{PC}$$

Column 4 of M_v contains the distance along d and elevation z of the evaluated point. Column 1 contains $(dx_v, dy_v) = (\cos\theta_v, \sin\theta_v)$, the grade direction at that point. Step 4 is performed immediately for this point before moving to the next point.

Step 4 — Combine with the horizontal alignment point to produce the 3D placement matrix

The vertical result lives in the (distance along, elevation) plane and must be merged with the horizontal alignment to form a 3D position and orientation. The "distance along" axis corresponds to the curve tangent of the horizontal alignment and the vertical y is elevation (Z in 3D).

Evaluate the horizontal placement matrix M_h at distance d along the `IfcCompositeCurve`. This yields the horizontal 2D position (x_h, y_h) and the horizontal alignment tangent direction $(dx_h, dy_h) = (\cos\theta_h, \sin\theta_h)$.

M_v is a two-dimensional matrix whose rows index the vertical frame: row 1 = distance-along, row 2 = elevation, row 3 = out-of-plane. Three modifications produce M'_v, the form required for multiplication with M_h.

Column 4 — The distance-along value d is replaced with zero. The horizontal position already comes from M_h; retaining d would displace the point a second time along the horizontal tangent.

Rows 2 and 3 — M_h's third row $(0,\ 0,\ 1,\ 0)$ passes row 3 of the right operand through unchanged into row 3 of the product. Because row 3 of M_{3D} must carry the elevation (Z) component of every vector, elevation must occupy row 3 of M'_v — not row 2 as in M_v. The row semantics are therefore swapped: in M'_v, row 2 is the cross-track (lateral) component and row 3 is the elevation (Z) component.

Columns 2 and 3 — The row reordering reindexes the column values and exchanges the columns. M_v column 2, the in-plane normal $(-dy_v,\ dx_v,\ 0)$ in [d-along, elevation, out-of-plane] ordering, becomes $(-dy_v,\ 0,\ dx_v)$ in [d-along, cross-track, elevation] ordering — the 3D up direction — and moves to column 3. M_v column 3, the out-of-plane unit direction $(0,\ 0,\ 1)$, becomes $(0,\ 1,\ 0)$ — the cross-track direction — and moves to column 2.

$$M'_v = \begin{matrix} dx_v & 0 & -dy_v & 0 \\ 0 & 1 & 0 & 0 \\ dy_v & 0 & dx_v & z \\ 0 & 0 & 0 & 1 \end{matrix}$$

The columns of M'_v define directions in the 3D frame local to the alignment:

- **Column 1** $(dx_v,\ 0,\ dy_v)$: the grade direction — dx_v scales the contribution along the horizontal tangent; dy_v is the elevation rate of change (Z component of the 3D tangent).
- **Column 2** $(0,\ 1,\ 0)$: the cross-track direction, always lateral to the alignment and always horizontal.
- **Column 3** $(-dy_v,\ 0,\ dx_v)$: the "up" direction in the vertical plane of the alignment, orthogonal to the grade direction.
- **Column 4** $(0,\ 0,\ z)$: the elevation offset with no horizontal displacement.

The 3D placement matrix is:

$$M_{3D} = M_h \cdot M'_v$$

Multiplying out, the columns of M_{3D} are:

- **Column 1** (RefDirection / 3D tangent): $(dx_h \cdot dx_v,\ dy_h \cdot dx_v,\ dy_v)$ — the horizontal tangent scaled by $\cos\theta_v$, with $\sin\theta_v$ as the Z component.
- **Column 2** (Y / cross-track): $(-dy_h,\ dx_h,\ 0)$ — the horizontal normal direction, unchanged from M_h.
- **Column 3** (Axis / up): $(-dx_h \cdot dy_v,\ \ -dy_h \cdot dy_v,\ dx_v)$ — orthogonal to both the 3D tangent and the cross-track direction.
- **Column 4** (Location): $(x_h,\ y_h,\ z)$ — the full 3D position.

3.3 Constant Gradient

A constant gradient is geometrically represented with a segment trimmed from an `IfcLine` parent curve.

3.3.1 Parent Curve Parametric Equations

$$x(s) = p_x + s$$
$$y(s) = p_y + s(dy/dx)$$

Note that these equations are different from the equations for horizontal.

3.3.2 Semantic Definition to Geometry Mapping

Mapping of the semantic definition of the linear segment to the geometric definition is described with the following example.

Consider a vertical gradient at an uphill slope of 0.5 starting at point (0,10). The horizontal projection of the segment length is 100.

```
#44 = IFCALIGNMENTVERTICALSEGMENT($, $, 0., 100., 10., 0.5, 0., $, .CONSTANTGRADIENT.);
```

Define the direction of the `IfcLine` so it matches the vertical segment

$$dx = \cos\left(\tan^{-1} 0.5\right) = 0.894427191$$
$$dy = \sin\left(\tan^{-1} 0.5\right) = 0.44713595$$

Define the `IfcLine` as passing through point (0,0)

```
#80 = IFCLINE(#81, #82);
#81 = IFCCARTESIANPOINT((0., 0.));
#82 = IFCVECTOR(#83, 1.);
#83 = IFCDIRECTION((0.894427190999916, 0.447213595499958));
```

Place the curve segment at (0,10) with a tangent direction (0.894427190999916, 0.447213595499958).

```
#77 = IFCAXIS2PLACEMENT2D(#78, #79);
#78 = IFCCARTESIANPOINT((0., 10.));
#79 = IFCDIRECTION((0.894427190999916, 0.447213595499958));
```

`IfcCurveSegment.SegmentLength` is the length measured along the trimmed curve. For a horizontal length of 100 m, the length along the curve is $100/0.894427190999916 = 111.803398874989$ m

The curve segment is defined as

```
#71 = IFCCURVESEGMENT(.CONTINUOUS., #77, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(111.803398874989), #80);
```

3.3.3 Compute Point on Curve

Compute the vertical placement matrix for the point at horizontal distance $\ell = 50$ m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

From `IfcCurveSegment.Placement`: $(d_p,\ z_p) = (0,\ 10)$, $\theta_p = 0.463647609$ rad:

$cos\theta_p = cos(0.463647609) = 0.894427191 \quad sin\theta_p = sin(0.463647609) = 0.447213595$

$$M_{CSP} = \begin{matrix} 0.894427191 & -0.447213595 & 0 & 0 \\ 0.447213595 & 0.894427191 & 0 & 10 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

The trim begins at $s_0 = \text{SegmentStart} = 0$. For the `IfcLine` through the origin with direction $(dx,\ dy) = (0.894427191,\ 0.447213595)$:

$$d_0 = x(0) = 0 \quad z_0 = y(0) = 0 \quad \theta_0 = tan^{-1}\left(\frac{dy}{dx}\right) = tan^{-1}\left(\frac{0.894427191}{0.447213595}\right) = 0.463647609 \text{ rad}$$

$$M_N = \begin{matrix} 0.894427191 & 0.447213595 & 0 & 0 \\ -0.447213595 & 0.894427191 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate and map each point in the vertical plane

The horizontal distance $\ell = 50$ m is not the arc-length parameter. Convert to arc-length s along the `IfcLine`:

$$s = s_0 + \frac{\ell}{dx} = 0 + \frac{50}{0.894427191} = 55.9016994 \text{ m}$$

Evaluate the parent curve at $s = 55.9016994$:

$$d = x(55.9016994) = 55.9016994 \times 0.894427191 = 50.000 \text{ m}$$

$$z = y(55.9016994) = 55.9016994 \times 0.447213595 = 25.000 \text{ m}$$

$$\theta = 0.463647609 \text{ rad} \quad \text{(constant for a line)}$$

$$M_{PC} = \begin{matrix} 0.894427191 & -0.447213595 & 0 & 50.000 \\ 0.447213595 & 0.894427191 & 0 & 25.000 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

The vertical placement matrix is:

$$M_v = M_{CSP}\ M_N\ M_{PC} = \begin{matrix} 0.894427191 & -0.447213595 & 0 & 0 & 0.894427191 & 0.447213595 & 0 & 0 & 0.894427191 & -0.4472 \\ 0.447213595 & 0.894427191 & 0 & 10 & -0.447213595 & 0.894427191 & 0 & 0 & 0.447213595 & 0.89442 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \end{matrix}$$

$$M_v = \begin{matrix} 0.894427191 & -0.447213595 & 0 & 50.000 \\ 0.447213595 & 0.894427191 & 0 & 35.000 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

3.4 Circular Arc

A circular vertical curve is geometrically represented with a segment trimmed from an `IfcCircle` parent curve.

3.4.1 Parent Curve Parametric Equations 3.4.2 Semantic Definition to Geometry Mapping

Vertical circular arcs are tricky. The "distance along" dimension is horizontal while the `IfcCurveSegment` trimming parameters `SegmentStart` and `SegmentLength` are measured along the `IfcCircle`.

The following procedure maps the semantic parameters of a vertical circular arc to its geometric definition.

Given the following semantic definition of a vertical circular arc, create the geometric definition.

```
#44 = IFCALIGNMENTVERTICALSEGMENT($, $, 0., 100., 10., -5.E-1, -1., 384.773458895502, .CIRCULARARC.);
```

Determine the tangent slope angle at the start and end of the segment.

$g_{start} = -0.5$

$\theta_{start} = tan^{-1}(g_{start})$

$\theta_{start} = tan^{-1}(-0.5) = -0.463648$

$g_{end} = -1$

$\theta_{end} = tan^{-1}(g_{end})$

$\theta_{end} = tan^{-1}(-1) = -0.785398$

Compute the radius of the circle.

`IfcAlignmentVerticalSegment.RadiusOfCurvature` is optional. If provided, it should be consistent with the `HorizontalLength`, `StartGradient` and `EndGradient`, but is not guaranteed. For this reason, the radius is computed from the required `HorizontalLength`, `StartGradient` and `EndGradient` attributes. Note that in computing the radius, it is taken to be an absolute value because `IfcCircle.Radius` is a `IfcPositiveLengthMeasure`.

$R = \frac{h_l}{sin(\theta_{start}) - sin(\theta_{end})}$

$h_l = \text{IfcAlignmentVerticalSegment.HorizontalLength} = 100.$

$R = \frac{328.083989501312}{sin(-0.785398) - sin(-0.463648)} = 384.773458895502$

Compute the arc-length trimming parameters `SegmentStart` and `SegmentLength`.

For an `IfcCircle` with CCW parametrization and `RefDirection` = (1, 0), the arc-length parameter at a point where the grade angle is θ depends on which half of the circle the vertical curve occupies.

If $\theta_{start} < \theta_{end}$ — sagging curve (lower half of circle):

$$\text{SegmentStart} = R\left(\theta_{start} + \tfrac{3}{2}\pi\right)$$

If $\theta_{start} > \theta_{end}$ — cresting curve (upper half of circle):

$$\text{SegmentStart} = R\left(\theta_{start} + \tfrac{1}{2}\pi\right)$$

In both cases:

$$\text{SegmentLength} = R(\theta_{end} - \theta_{start})$$

For a cresting curve $\theta_{end} < \theta_{start}$, so `SegmentLength` is negative — the segment is traversed clockwise.

For this example (cresting):

$\text{SegmentStart} = R\left(\theta_{start} + \frac{\pi}{2}\right) = (384.773458895502)(-0.463647609 + \frac{\pi}{2}) = 426.001441657352$

$\text{SegmentLength} = R(\theta_{end} - \theta_{start}) = (384.773458895502)(-0.785398 - (-0.463647609)) = -123.801073716741$

Determine the placement of the trimmed curve

X = `IfcAlignmentVerticalSegment.StartDistAlong` = 0.0

Y = `IfcAlignmentVerticalSegment.StartHeight` = 10.0

$$C_x = Rd_y = 384.773458895502(-0.447213595) = -172.075922$$

$$C_y = -Rd_x = -384.773458895502(0.894427191) = -344.151844$$

$dx = cos(\theta_{start}) = cos(-0.463647609) = 0.894427191$

$dy = sin(\theta_{start}) = sin(-0.463647609) = -0.447213595$

The geometric representation is

```
#71 = IFCCURVESEGMENT(.CONTINUOUS., #77, IFCLENGTHMEASURE(426.001441657352), IFCLENGTHMEASURE(-123.801073716741), #80);
#77 = IFCAXIS2PLACEMENT2D(#78, #79);
#78 = IFCCARTESIANPOINT((0., 10.));
#79 = IFCDIRECTION((8.94427190999916E-1, -4.47213595499958E-1));
#80 = IFCCIRCLE(#81, 384.773458895502);
#81 = IFCAXIS2PLACEMENT2D(#82, #83);
#82 = IFCCARTESIANPOINT((-172.075922005613, -344.151844011225));
#83 = IFCDIRECTION((1., 0.));
```

3.4.3 Compute Point on Curve

Compute the vertical placement matrix for the point at horizontal distance $\ell = 50$ m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

From `IfcCurveSegment.Placement`: $(d_p,\ z_p) = (0., 10.), dx_p = 0.894427190999916,\ dy_p = -0.447213595499958$

$$M_{CSP} = \begin{matrix} 0.894427190999916 & 0.447213595499958 & 0 & 0 \\ -0.447213595499958 & 0.894427190999916 & 0 & 10. \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

$$\Delta_0 = tan^{-1}\left(\tfrac{-344.151844011225}{-172.075922005613}\right) = 1.1071487177940900$$

$$x_0 = C_x + R\cos(\Delta_0) = -172.075922 + 384.773458895502\cos(1.1071487177940900) = 0$$

$$y_0 = C_y + R\sin(\Delta_0) = -344.151844 + 384.773458895502\sin(1.1071487177940900) = 0$$

$$\theta_0 = \Delta_0 - \tfrac{\pi}{2} = -0.463647609$$

$dx_0 = \cos(\theta_0) = 0.89442719099991$

$dy_0 = \sin(\theta_0) = -0.447213595499958$

$$M_N = \begin{matrix} 0.89442719099991 & -0.447213595499958 & 0 & 0 \\ 0.447213595499958 & 0.89442719099991 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate and map each point in the vertical plane

Compute the vertical placement matrix for the point at horizontal distance $\ell = 50\ m$ from the start of the curve segment.

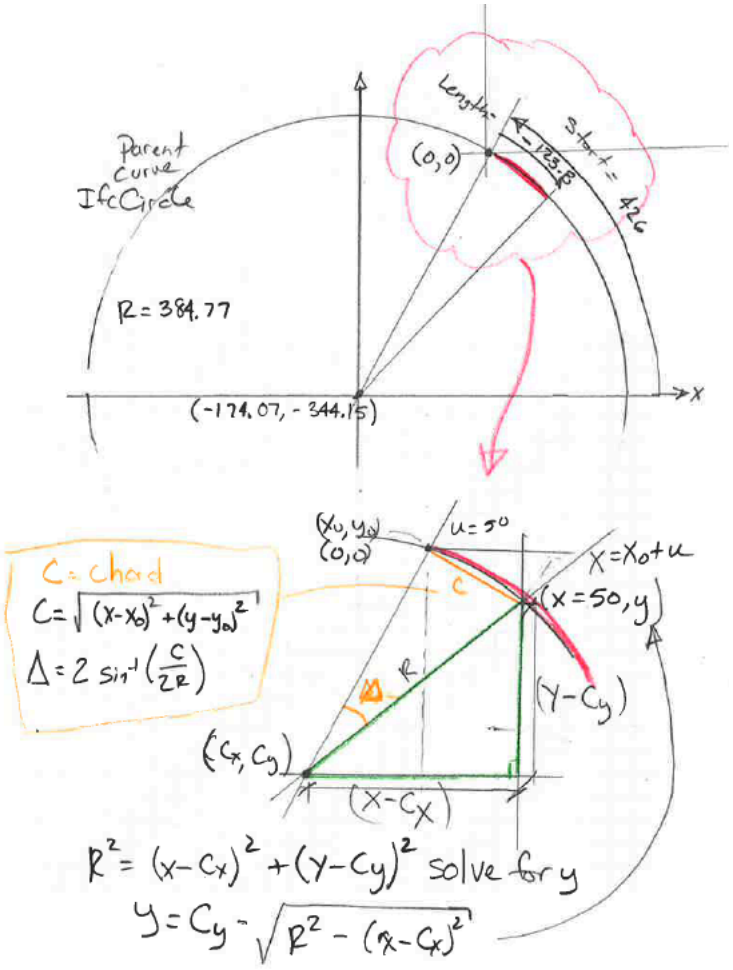
The center point of the trimmed circular arc is

$C_x = -172.075922005613,\ C_y = -344.151844011225$

From Step 2

$x_0 = 0,\ y_0 = 0,\ \theta_0 = -0.463647609$

Compute the chord length between the start and end of the trimmed segment. This is accomplished by locating the point on the parent curve that corresponds to a horizontal distance of $\ell = 50\ m$. From Figure 3.4.3-1 the point on the parent curve is located using the green triangle. Once the point on the parent curve is known, a chord distance between the start of the trim and the point can be computed. From here a sweep angle Δ is computed and then the tangent direction at the point.



Location point on parent curve corresponding to $\ell = 50$

$$x = x_0 + \ell = 50\ m$$

$$y = C_y - \frac{L}{|L|}\sqrt{R^2 - (x - C_x)^2} = -344.151844011225 - \frac{-123.801073716741}{|-123.801073716741|}\sqrt{(384.773458895502)^2 + (50 - (-172.075922005613))^2} =$$

Compute the chord length from the trim start to the point

$$c = \sqrt{(x - x_0)^2 + (y - y_0)^2} = \sqrt{(50 - (0))^2 + (-19.933926737614911 - 10)^2} = 58.275552077461235$$

Angle subtented by the trimmed segment

$$\Delta = 2\sin^{-1}\left(\frac{c}{2R}\right) = 2\sin^{-1}\left(\frac{58.275552077461235}{2 \cdot 384.773458895502}\right) = 0.15159931828379261$$

Arc-length of the trimmed segment = $R\Delta = (384.773458895502)(0.15159931828379261) = 58.331394062255001$

Tangent angle at $\ell = 50\ m$

$\Delta_{pc} = \theta_0 - \Delta = 1.1071487177940900 - 0.15159931828379261 = 0.95554939951029738$

Compute point on trimmed segment at ℓ.

$$x_{pc} = C_x + Rcos(\Delta_{pc}) = (-172.075922005613) + 384.77345889550202\cos(0.95554939951029738) = 50$$

$$y_{pc} = C_y + Rsin(\Delta_{pc}) = (-344.15184401122502) + 384.77345889550202\sin(0.95554939951029738) = -29.933926737614570$$

$$dx_{pc} = -\frac{L}{|L|}\sin(\Delta_{pc}) = -\frac{-123.801073716741}{|-123.801073716741|}\sin(0.95554939951029738) = 0.81663095520043849$$

$$dy_{pc} = \frac{L}{|L|}\cos(\Delta_{pc}) = \frac{-123.801073716741}{|-123.801073716741|}\cos(0.95554939951029738) = -0.57716018834325311$$

$$M_{PC} = \begin{matrix} 0.81663095520043849 & 0.57716018834325311 & 0 & 50 \\ -0.57716018834325311 & 0.81663095520043849 & 0 & -29.933926737614570 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

The vertical placement matrix is:

$$M_v = M_{CSP}\ M_N\ M_{PC} = \begin{matrix} 0.894427190999916 & 0.447213595499958 & 0 & 0 & 0.89442719099991 & -0.447213595499958 & 0 \\ -0.447213595499958 & 0.894427190999916 & 0 & 10. & 0.447213595499958 & 0.89442719099991 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{matrix}$$

$$M_v = \begin{matrix} 0.81663096 & 0.57716019 & 0. & 50. \\ -0.57716019 & 0.81663096 & 0. & -19.93392674 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

3.5 Clothoid [todo - finish this]

The reference implementation at IFC-Rail-Unit-Test-Reference-Code/EnrichIFC4x3/EnrichIFC4x3/business2geometry at master · bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code (github.com), does not seem to correctly handle vertical alignments with clothoids. The examples provided are incomplete. The vertical alignments have a total length of zero and the resulting clothoid constants are zero.

An example of an example file generated from the reference implementation is https://github.com/bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code/blob/master/alignment_testset/IFC-WithGeneratedGeometry/GENERATED__VerticalAlignment_Clothoid_100.0_10.0_0.5_0.0_1_Meter.ifc.

```
#41 = IFCALIGNMENTVERTICAL('1FNFyDAJeHwv87wDZHIYI1', $, $, $, $, #89, $);
#42 = IFCALIGNMENTSEGMENT('1FNFyHAJeHwuDtwDZHIYI2', #3, $, $, $, #72, #75, #44);
#43 = IFCRELNESTS('4CGecNrjCHwxOSbERtTLTf', $, $, $, #41, (#42));
#44 = IFCALIGNMENTVERTICALSEGMENT($, $, 0., 100., 10., 5.E-1, 0., $, .CLOTHOID.);

#70 = IFCGRADIENTCURVE((#71), .F., #45, #86);
#71 = IFCCURVESEGMENT(.CONTINUOUS., #77, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(0.), #80);
#77 = IFCAXIS2PLACEMENT2D(#78, #79);
#78 = IFCCARTESIANPOINT((0., 10.));
#79 = IFCDIRECTION((8.94427190999916E-1, 4.47213595499958E-1));
#80 = IFCCLOTHOID(#81, 0.);
#81 = IFCAXIS2PLACEMENT2D(#82, $);
#82 = IFCCARTESIANPOINT((0., 0.));
```

Due to a lack of information, vertical clothoid transition curves have not yet been addressed.

3.6 Parabolic Arc

The most common transition curve in a vertical profile is a parabola. The geometric representation is `IfcPolynomialCurve`. Mapping of the semantic definition to the geometric definition can be a bit tricky.

3.6.1 Parent Curve Parametric Equations

The general form of a parabola is

$$y(x) = A_2x^2 + A_1x + A_0$$

where:

A_2 = (end gradient - start gradient)/(2 * horizontal length)

A_1 = start gradient

A_0 = start height

The gradient of the curve is

$$y'(x) = 2A_2x + A_1$$

3.6.2 Semantic Definition to Geometry Mapping

Consider a 1600 m parabolic vertical cover that starts 1200 m along the horizontal alignment. The entry grade is 1.75% and the exit grade is -1%. The elevation at the start of the curve is 121 m.

The semantic definition is

```
#289=IFCALIGNMENTVERTICALSEGMENT($,$,1200.,1600.,121.,0.017500000000000002,-0.01,$,.PARABOLICARC.);
```

Compute the polynomai curve coefficients

$A_0 = 121.\ m$

$A_1 = 0.0175$

$A_2 = \frac{-0.01-0.0175}{2 \cdot 1600.} = -8.59375 \cdot 10^{-6}\ m^{-1}$

It is easiest to place the parent curve at the origin and orient it with the global coordinate system. The parent curve is defined as

```
#291=IFCCARTESIANPOINT((0.,0.));
#292=IFCDIRECTION((1.,0.));
#293=IFCAXIS2PLACEMENT2D(#291,#292);
#294=IFCPOLYNOMIALCURVE(#293,(0.,1.),(121.,0.017500000000000002,-8.5937500000000005E-06),$);
```

> Note 1: Even though vertical is typically z, alignment is 2.5D geometry and the coordinate system of gradient curve is "Distance along Horizontal", "Elevation" which is a 2D curve in the plane of the horizontal curve. When the `IfcGradientCurve` and `IfcCompositeCurve` are combined to get a 3D point, the elevation is then mapped to Z. See example in Section 3.7 below.

> Note 2: The coefficients A_0, A_1, and A_2 must have the following unit of measure, consistent with the project units:
>
> $A_0 = Length^1$
>
> $A_1 = Length^0$
>
> $A_2 = Length^{-1}$
>
> The coefficients of `IfcPolynomialCurve` expect real numbers without explicit unit of measure. This is a problem with the IFC Specification. See the discussion of `IfcAlignmentHorizontalSegment` and `IfcPolynomialCurve` for Cubic Transition Curve in Section 2.0 - Horizontal Alignments. Implicit units of measure are required for the polynomial coefficients.

The polynomial curve is trimmed using `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength`. These parameters are measured along the length of the curve. The horizontal projection of the segment length, from `IfcAlignmentVerticalSegment.HorizontalLength` is $h_l = 1600$. `SegmentLength` will be slightly longer because it is the distance along the curve.

The distance along a curve is

$s(x) = \int (\sqrt{(y')^2 + 1})dx$

The length along the parabolic curve is then:

$s(x) = \int \sqrt{4A_2^2x^2 + 4A_2A_1x + A_1^2 + 1}dx$

The solution to this integral is:

$s(x) = \int \left(\sqrt{ax^2 + bx + c}\right)dx = \frac{b+2ax}{4a}\sqrt{ax^2 + bx + c} + \frac{4ac-b^2}{8a^{\frac{3}{2}}}ln\ \ 2ax + b + 2\sqrt{a(ax^2 + bx + c)}$

The length along the curve is $L_c = s(h_l) - s(0.0)$.

Let $\quad a = 4A_2^2 \quad b = 4A_2A_1 \quad c = A_1^2 + 1$

Compute the coefficients a, b, c, substitute curve length equation and solve. The curve length is

$L_c = 1600.0616641340894$

The placement of the trimmed curve segment is noteworthy. From the `IfcAlignmentVerticalSegment` attributes, the parabolic segment of the vertical alignment starts at 1200 from the beginning of the horizontal alignment and is at an elevation of 121. The RefDirection vector at the start of the curve is needed as well.

The gradient is $y'(x = 0) = 0.0175$

$\theta_p = tan^{-1}(0.0175) = 0.017498213869856595$

$dx = cos(0.017498213869856595) = 0.017497320927833689$

$dy = sin(0.017498213869856595) = 0.99984691016192495$

The geometric representation is

```
#291=IFCCARTESIANPOINT((0.,0.));
#292=IFCDIRECTION((1.,0.));
#293=IFCAXIS2PLACEMENT2D(#291,#292);
#294=IFCPOLYNOMIALCURVE(#293,(0.,1.),(121.,0.017500000000000002,-8.5937500000000005E-06),$);
#295=IFCCARTESIANPOINT((1200.,121.));
#296=IFCDIRECTION((0.99984691016192495,0.017497320927833689));
#297=IFCAXIS2PLACEMENT2D(#295,#296);
#298=IFCCURVESEGMENT(.CONTSAMEGRADIENT.,#297,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(1600.0616641340894),#294);
```

3.6.3 Compute Point on Curve

Compute the vertical placement matrix for the point at horizontal distance 1500 m from the start of the alignment. This vertical alignment segments starts at 1200 from the start of the alignment. The evaluation point is $\ell = 1500 - 1200 = 300$ from the start of the segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

From `IfcCurveSegment.Placement`: $(d_p,\ z_p) = (1200,\ 121)$, $dx_p = 0.99984691016192495$, $dy_p = 0.017497320927833689$

$$M_{CSP} = \begin{matrix} 0.99984691016192495 & -0.017497320927833689 & 0 & 1200 \\ 0.017497320927833689 & 0.99984691016192495 & 0 & 121 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0,\ y_0 = 0,\ \theta_0 = 0$.

Since $x_0 = 0,\ y_0 = 0,\ \theta_0 = 0,\ M_N = [I]$

Step 3 — Evaluate and map each point in the vertical plane

Evaluate the parent curve at $x = 300$

$y(300) = -8.59375 \cdot 10^{-6}300^2 + 0.0175(300) + 121 = 125.4765625$

$y'(300) = 2(-8.59375 \cdot 10^{-6})300 + 0.0175 = 0.01234375$

$dx = \frac{1}{\sqrt{(0.01234375)^2+1}} = 0.999923825$

$dy = \frac{0.01234375}{\sqrt{(0.01234375)^2+1}} = 0.01234281$

$$M_{PC} = \begin{matrix} 0.999923825 & -0.01234281 & 0 & 300 \\ 0.01234281 & 0.999923825 & 0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

The vertical placement matrix is:

$$M_v = M_{CSP}\ M_N\ M_{PC} = \begin{matrix} 0.99984691016192495 & -0.017497320927833689 & 0 & 1200 \\ 0.017497320927833689 & 0.99984691016192495 & 0 & 121 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} [I] \begin{matrix} 0.999923825 & -0.01234281 & 0 & \\ 0.01234281 & 0.999923825 & 0 & 12 \\ 0 & 0 & 1 & \\ 0 & 0 & 0 & \end{matrix}$$

$$M_v = \begin{matrix} 0.9999998299568322 & -0.0005831691921746294 & 0.0 & 1500.0 \\ 0.0005831691921746294 & 0.9999998299568322 & 0.0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

3.7 Combined 3D

To complete evaluation of a point on the alignment, the 2D vertical is combined with the 2D horizontal resulting in a point in 3D space.

From line example in Section 2.3, the horizontal alignment point at 1500 m is

$$M_h = \begin{matrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

From the parabolic arc example in Section 3.6, the corresponding vertical alignment point is

$$M_v = \begin{matrix} 0.9999998299568322 & -0.0005831691921746294 & 0.0 & 1500.0 \\ 0.0005831691921746294 & 0.9999998299568322 & 0.0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Combine with the horizontal alignment point to produce the 3D placement matrix

Modify the M_v matrix into M'_v to put the vertical point into the same reference frame as the horizontal. Swap rows and columns 2 and 3. Set the distance along to 0.0.

$$M'_v = \begin{matrix} dx_v & 0 & -dy_v & 0 \\ 0 & 1 & 0 & 0 \\ dy_v & 0 & dx_v & z \\ 0 & 0 & 0 & 1 \end{matrix} = \begin{matrix} 0.9999998299568322 & 0.0 & -0.0005831691921746294 & 0.0 \\ 0 & 1 & 0 & 0 \\ 0.0005831691921746294 & 0 & 0.9999998299568322 & 125.4765625 \\ 0 & 0 & 0 & 1 \end{matrix}$$

The 3D placement matrix is:

$$M_{3D} = M_h \cdot M'_v$$

$$M_{3D} = \begin{matrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 & 0.9999998299568322 & 0.0 & -0.0005831691921746294 & 0.0 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0.0005831691921746294 & 0 & 0.9999998299568322 & 125.476562 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{matrix}$$

$$M_{3D} = \begin{matrix} 0.8391888595725846 & 0.5437414408769801 & -0.010358737485349092 & 1758.8791849555332 \\ -0.5437000211676682 & 0.8392527899703555 & 0.006711297136288405 & 1684.38783868453 \\ 0.012342809710188737 & 0.0 & 0.999923824622885 & 125.4765625 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{matrix}$$

Section 4 - Cant Alignments 4.0 Introduction

Cant (also called superelevation) is the transverse inclination of a railway track cross-section: the elevation difference between the two railheads when one rail is raised above the other. By tilting the track into a curve, cant directs a component of gravitational force inward, partially offsetting the centrifugal acceleration experienced by vehicles negotiating the curve. The result is improved ride quality, reduced wheel–rail lateral forces, and the ability to operate at higher speeds through curves without exceeding comfort or safety limits.

Cant is measured as a length — the height difference between railheads — rather than as an angle. The conversion between the two depends on track gauge. `IfcAlignmentCant.RailHeadDistance` supplies the centre-to-centre distance between railheads, which is the lever arm for this conversion.

4.1 General

An `IfcSegmentedReferenceCurve` describes the cross slope of superelevated rail lines as a track banks through curves. When the point of rotation is about one of the railheads, the alignment elevation must deviate to accomodate the cross slope.

An `IfcSegmentedReferenceCurve` also describes this deviation in elevation. As a subtype of `IfcCompositeCurve`, an `IfcSegmentedReferenceCurve` consists of an end to start collection of `IfcCurveSegment`. An `IfcSegmentedReferenceCurve` also has a `BasisCurve` which is typically an `IfcGradientCurve`.

Table 4.1-1 maps each `IfcAlignmentCantSegment.PredefinedType` to its corresponding parent curve type.

Business Logic (`IfcAlignmentCantSegment.PredefinedType`)	**Geometric Representation (`IfcCurveSegment.ParentCurve`)**
CONSTANTCANT	`IfcLine`
LINEARTRANSITION	`IfcClothoid`
HELMERTCURVE	`IfcSecondOrderPolynomialSpiral`
BLOSSCURVE	`IfcThirdOrderPolynomialSpiral`
COSINECURVE	`IfcCosineSpiral`
SINECURVE	`IfcSineSpiral`
VIENNESEBEND	`IfcSeventhOrderPolynomialSpiral`

Table 4.1-1 — Mapping of business logic to geometric representation for cant alignment

4.1.1 Cant Transition

Cant transitions occur when the segment start elevation differs from the segment start elevation of the next segment. The left section in Figure 4.1-1 shows the cross section of a rail line. At the start of the segment, right rail is elevated above the left rail. The y-ordinate of `IfcCurveSegment.Placement.Location` indicates the deviating elevation at the centerline between rails, which is one-half the cant value at the right rail. The y-ordinate of `IfcCurveSegment.Placement.Location` for the next segment indicates a value of 0.0 meaning that the cant transitioned from the starting value to zero over the length of the segment. This transition is the rotation of the right rail about the left rail, and the corresponding change in deviating elevation at the centerline over the length of the segment. The cant transition is shown in the right section of Figure 4.1.1-1 as a linear transition. `IfcCurveSegment.ParentCurve` defines how the transition occurs. Figure 4.1.1-2 shows a non-linear transition of the centerline and right rail deviating elevations. Notice that the deviating elevation of the left rail is constant at zero because it is the point of rotation.

When the segment start elevation is the same as the start of the next segment, the deviating elevation along the length of the segment is constant. This occurs when centrifugal forces are constant or zero, in constant radius curvature and straight sections of the railway, respectively.

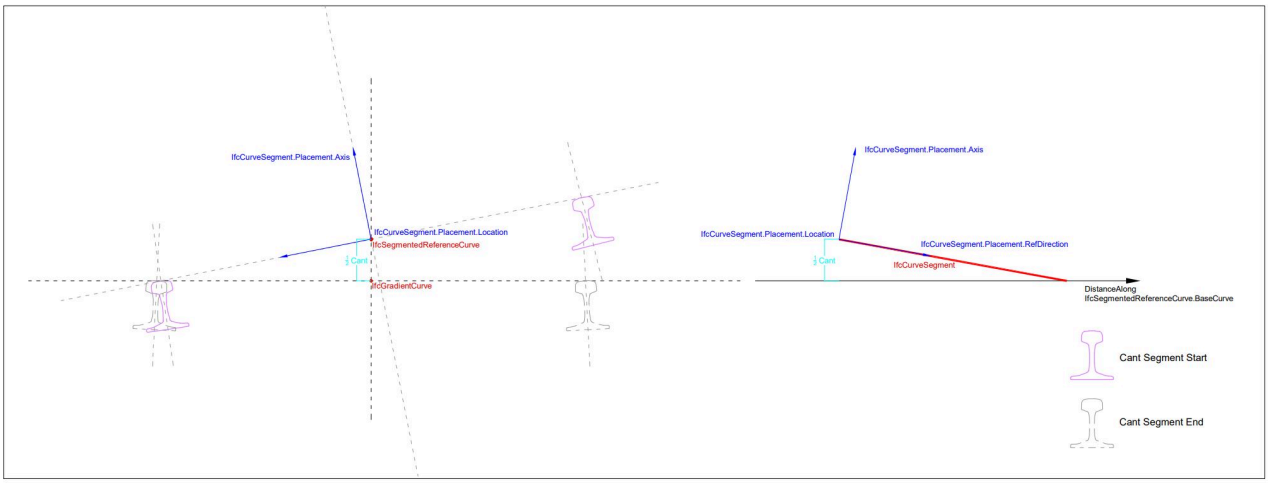


Figure 4.1.1-1 - cross section and elevation of a cant segment (source: bSI)

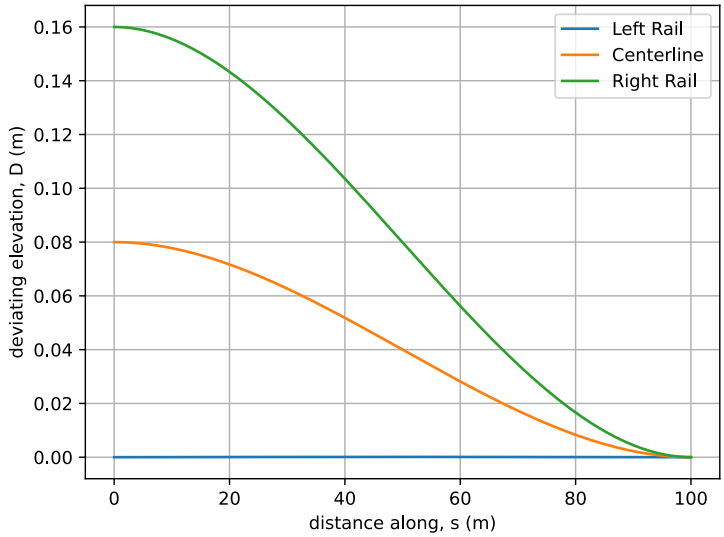


Figure 4.1.1-2 - Deviating elevation of rails

The railhead cross slope angle is defined by the `IfcCurveSegment.Placement.Axis` attribute. The `Axis` direction is generally upwards. Figure 4.1.1-3 shows the cross slope angle for the transitions in Figure 4.1.1-2. The direction perpendicular to the plane of the railhead is about 1.46 rad at the start of the segment and increases to about 1.57 as the rotation decreases. When the left and right rails are at the same elevation, The cross slope angle is measured from the y-axis is $\frac{\pi}{2}$.

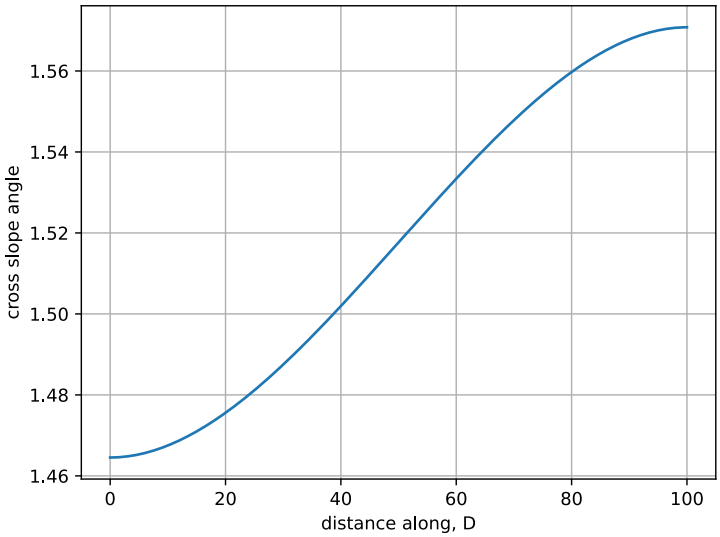


Figure 4.1.1-3 - Rail head cross slope

4.1.2 Coordinate System

The coordinate system is shown in Figure 4.1.1-1, but it is difficult to understand.

`IfcCurveSegment.Placement.RefDirection` is tangent to the "distance along" the base curve in the horizontal plane, (e.g. `IfcSegmentedReferenceCurve.BaseCurve` => `IfcGradientCurve.BaseCurve` => `IfcCompositeCurve`).

The railway cross section is in the plane defined by `Y` and `Z`. Looking in the positive sense down the alignment, `Y` is towards the left and `Z` is upwards. The cross slope angle is measured clockwise from `Y`.

4.1.3 Transition Functions

The transition functions used to shape cant variation have the same functional form as functions used for horizontal transition curves: line, clothoid (linear cant), Helmert, Bloss, cosine, sine, and Viennese bend. In practice, the cant transition type always matches the horizontal transition type and segment length, though IFC does not enforce this constraint.

As an example, the horizontal tangent direction, $\theta(t)$, and the radius of curvature, $\kappa(t)$, for a Helmert curve is a second order polynomial of the form

$$\theta(t) = \frac{t^3}{3A_2^3} + \frac{A_1}{2|A_1^3|}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{\theta(t)}{dt} = \frac{1}{A_2^3}t^2 + \frac{A_1}{|A_1^3|}t + \frac{1}{A_0}$$

The cant deviating elevation for a Helmert transition is of the same form as the curvature

$$\frac{D(s)}{L^2} = \frac{1}{A_2^3}s^2 + \frac{A_1}{|A_1^3|}s + \frac{1}{A_0}$$

The deviating elevation has the same functional form as the radius of curvature of the horizontal alignment segment.

In IFC, the semantic cant profile is encoded in `IfcAlignmentCant` and its child `IfcAlignmentCantSegment` entities. The geometric representation is an `IfcSegmentedReferenceCurve`, which evaluates the cant at every point along the alignment and, in conjunction with the horizontal alignment `IfcCompositeCurve` and the vertical alignment `IfcGradientCurve`, produces the full 3D track centerline geometry.

Each `IfcCurveSegment` in an `IfcSegmentedReferenceCurve` is evaluated in a two-dimensional coordinate system whose axes are *distance along the horizontal alignment* ℓ and *deviating elevation* D. The deviating elevation is the vertical offset applied to the track centerline, away from the gradient curve, to produce the correct cross-slope angle at every point along the segment.

The `StartCantLeft` D_{sl}, `EndCantLeft` D_{el}, `StartCantRight` D_{sr}, and `EndCantRight` D_{er} attributes of `IfcAlignmentCantSegment` determine D_s and D_e as the averages of their respective left and right values:

$$D_s = \frac{D_{sl}+D_{sr}}{2}, \quad D_e = \frac{D_{el}+D_{er}}{2} \quad \Delta D = D_e - D_s$$

The cross-slope angle ϕ at a given point is not obtained directly from the parent curve equation; instead it is interpolated between the start and end cross-slope angles encoded in the `Axis` vector of the `IfcCurveSegment.Placement`, using the deviating elevation as the interpolation parameter:

$$\phi(\ell) = \phi_s + \left(\frac{\phi_e - \phi_s}{\Delta D}\right)\left(D(\ell) - D_s\right)$$

$$\phi_s = \cos^{-1}\left(\frac{D_{sr}-D_{sl}}{D_{rh}}\right)$$

where ϕ_s and ϕ_e are the cross-slope angles at the start and end of the segment and $D(\ell)$ is the deviating elevation at ℓ. The cross-slope angle ϕ is the angle from the transverse y axis to the projection of the `Axis` vector onto the Y-Z plane. In sections without cant, $\phi = \frac{\pi}{2}$.

The cross-slope angle can be determined from the `Axis` vector as $\phi = tan^{-1}\left(\frac{Axis.dz}{Axis.dy}\right)$

Together, the distance along, the deviating elevation $D(\ell)$, and the cross-slope angle $\phi(\ell)$ fully specify a 3D placement frame for the track centerline at each ℓ. Section 4.2 describes an algorithm for constructing that frame and composing it with the horizontal and vertical matrices to produce a 3D position.

All examples use a railhead distance $D_{rh} = 1.5\ m$.

4.2 Curve Segment Evaluation Algorithm

Cant segments are evaluated in a two-dimensional "distance along, deviating elevation" $(\ell, D(\ell))$ coordinate system in which ℓ is the distance measured along the horizontal `IfcCompositeCurve` and $D(\ell)$ is the deviating elevation: the vertical offset applied to the track centerline to accommodate the cross slope. Unlike horizontal and vertical segments, each cant point also carries a cross slope angle $\phi(s)$, making the local frame inherently three-dimensional.

Let $s_0 =$ `IfcAlignmentCantSegment.StartDistAlong`.

Step 1 — Form the curve segment placement matrix M_{CSP}

M_{CSP} is constructed from `IfcCurveSegment.Placement`: (s_p, D_p) is the `Location` (distance along, deviating elevation).

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p$

$\mathbf{X}_p = \mathbf{Y}_p \times \mathbf{Axis}_p$

$\mathbf{Z}_p = \mathbf{Axis}_p$

The placement in matrix form is

$$M_{CSP} = \begin{matrix} X_p.x & Y_p.x & Z_p.x & s_p \\ X_p.y & Y_p.y & Z_p.y & D_p \\ X_p.z & Y_p.z & Z_p.z & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start - M_{PCS}

Compute the deviating elevation $D_s = D(s_0)$ and the slope angle $\theta_s = \tan^{-1}(D'(s_0))$.

Compute the cross slope

$$\phi(s_0) = \phi_s + \left(\frac{\phi_c - \phi_s}{\Delta D}\right)(D(s_0) - D_s)$$

Because $D(s_0) = D_s,\ \phi(s_0) = \phi_s$.

Construct the orthoginal vectors

$$\mathbf{X}_{pcs} = (\cos\theta_s,\ \sin\theta_s,\ 0)$$

$$\mathbf{Z}_{pcs} = (0,\ \cos\phi_s,\ \sin\phi_s)$$

$$\mathbf{Y}_{pcs} = \mathbf{Z}_{pcs} \times \mathbf{X}_{pcs}$$

$$\mathbf{Axis}_{pcs} = \mathbf{X}_{pcs} \times \mathbf{Y}_{pcs}$$

$$\mathbf{RefDir}_{pcs} = \mathbf{Y}_{pcs} \times \mathbf{Axis}_{pcs}$$

In matrix form

$$M_{PCS} = \begin{matrix} RefDir_{pcs}.x & Y_{pcs}.x & Axis_{pcs}.x & s_0 \\ RefDir_{pcs}.y & Y_{pcs}.y & Axis_{pcs}.y & D_s \\ RefDir_{pcs}.z & Y_{pcs}.z & Axis_{pcs}.z & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate the parent curve at the point under consideration, ℓ - $M_{PC\ell}$

This step follows the same calculations as Step 2, except D and D' are evaluated at ℓ.

The resulting matrix is

$$M_{PC\ell} = \begin{matrix} RefDir_{\ell}.x & Y_{\ell}.x & Axis_{\ell}.x & \ell \\ RefDir_{\ell}.y & Y_{\ell}.y & Axis_{\ell}.y & D_{\ell} \\ RefDir_{\ell}.z & Y_{\ell}.z & Axis_{\ell}.z & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Compute the cant placement matrix M_c

The cant frame at ℓ is obtained by applying the incremental change in the parent curve — from the trim start to ℓ — to the curve segment placement. Because the 4×4 matrices encode both orientation and parameter-space coordinates (distance along, deviating elevation) in column 4, the rotation and translation must be computed separately to prevent the rotation from acting on the position coordinates.

Rotation

Let R_{CSP}, R_{PCS}, and $R_{PC\ell}$ denote the upper-left 3×3 rotation submatrix of M_{CSP}, M_{PCS}, and $M_{PC\ell}$ respectively. The incremental rotation from trim start to ℓ is:

$$\Delta R = R_{PC\ell} \cdot R_{PCS}^{T}$$

Apply the increment to the curve segment placement:

$$R_c = \Delta R \cdot R_{CSP} = R_{PC\ell} \cdot R_{PCS}^{T} \cdot R_{CSP}$$

Translation

Let $\mathbf{T}_{CSP}$, $\mathbf{T}_{PCS}$, and $\mathbf{T}_{PC\ell}$ denote column 4, rows 1–3, of M_{CSP}, M_{PCS}, and $M_{PC\ell}$ respectively. The incremental translation from trim start to ℓ is:

$$\Delta\mathbf{T} = \mathbf{T}_{PC\ell} - \mathbf{T}_{PCS}$$

Apply the increment to the curve segment placement:

$$\mathbf{T}_c = \mathbf{T}_{CSP} + \Delta\mathbf{T} = \mathbf{T}_{CSP} + \mathbf{T}_{PC\ell} - \mathbf{T}_{PCS}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \mathbf{T}_c \\ \mathbf{0}^T & 1 \end{bmatrix}$$

Step 5 is performed immediately for this point before moving to the next ℓ.

Step 5 — Combine with horizontal and vertical to produce the 3D placement matrix

The cant result must be combined with the horizontal matrix M_h (evaluated at distance ℓ along the `IfcCompositeCurve`) and the vertical matrix M_v (evaluated at the same ℓ). All three coordinate systems share the distance-along axis, so the positional components of M_v and M_c must be extracted before multiplication and added back afterward to avoid incorrectly rotating the position offsets.

Construct M'_v as described in Section 3.2.

Construct M'_c by zeroing the distance-along ℓ component in row 1, column 4 and moving the deviating elevation D_ℓ from row 2 to row 3 in column 4 of M_c:

$$M'_c = \begin{matrix} X.x & Y.x & Z.x & 0 \\ X.y & Y.y & Z.y & 0 \\ X.z & Y.z & Z.z & D_\ell \\ 0 & 0 & 0 & 1 \end{matrix}$$

Extract position vectors from each modified matrix, $M'_v,\ M'_c$, (setting row 4 to zero), then zero column 4 before multiplying:

$$P_v = M'_v \text{ column 4, row 4 set to } 0 = \begin{matrix} 0 \\ 0 \\ z \\ 0 \end{matrix}, \qquad P_c = M'_c \text{ column 4, row 4 set to } 0 = \begin{matrix} 0 \\ 0 \\ D \\ 0 \end{matrix}$$

$$M''_v = M'_v \text{ with column 4 set to } (0,0,0,1)^T, \qquad M''_c = M'_c \text{ with column 4 set to } (0,0,0,1)^T$$

Multiply the three orientation matrices, then add back both position offsets:

$$M' = M_h \cdot M''_v \cdot M''_c$$

$$M_{3Dcant} = M' + \begin{matrix} 0 & 0 & 0 & P_{v.x} \\ 0 & 0 & 0 & P_{v.y} \\ 0 & 0 & 0 & P_{v.z} \\ 0 & 0 & 0 & 0 \end{matrix} + \begin{matrix} 0 & 0 & 0 & P_{c.x} \\ 0 & 0 & 0 & P_{c.y} \\ 0 & 0 & 0 & P_{c.z} \\ 0 & 0 & 0 & 0 \end{matrix}$$

Column 4 of the final matrix M_{3Dcant} is the full 3D position of the track centerline. Column 3 is the cross-slope direction, encoding the railhead superelevation at that point.

4.3 Constant Cant

Cant has a constant deviating elevation and cross-slope angle in rail segments between curves or in the circular portion of curves where the tracks are banked at a constant rate.

The parent curve is `IfcLine`. The slope-intercept form of a line is $y = mx + b$, where m is the slope and b is the y-intercept. The derivative is $y' = m$, and the second derivative is $y'' = 0$.

In the context of cant, $D(s) = y'(x)$, so the deviating elevation is a constant value, m, and the rate of change of the deviating elevation $D'(s) = y''(x) = 0$.

4.3.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$D(s) = D_s = D_e$$

$$\frac{d}{ds}D(s) = D'(s) = 0.0$$

4.3.2 Semantic Definition to Geometry Mapping

Consider 100 m long segment of a railway that is turning towards the left. The right rail is raised 0.16 m through the circular portion of the curve. The semantic definition is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 0.16, 0.16, .CONSTANTCANT.);
```

$D_s = \frac{0.0\ m + 0.16\ m}{2} = 0.08\ m$

$D_e = \frac{0.0\ m + 0.16\ m}{2} = 0.08\ m$

$\Delta D = D_e - D_s = 0.08\ m - 0.08\ m = 0.0\ m$

The parent curve has a constant deviating elevation at $0.08\ m$ and a slope of 0.0.

Define the parent curve as an `IfcLine` passing through (0,0) in the direction (1,0).

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCVECTOR(#97,1.);
#99=IFCLINE(#96,#98);
```

The `IfcCurveSegment` trims a segment from the parent curve. The curve segment placement must capture the cross-slope rotation between the rail heads. For this reason the placement is an `IfcAxis2Placement3D` (instead of `IfcAxis2Placement2D` used for horizontal and vertical representations).

The cross-slope at the start of the segment is

$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.463926346) = 0.106666667$

$dz_z = sin(\phi_s) = sin(1.463926346) = 0.994294837$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10666666666666667,0.99429483666678176));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.3.3 Compute Point on Curve

Compute the placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1,\ 0,\ 0)$

$\mathbf{Axis}_p = (0,\ 0.106667,\ 0.994295)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0,\ 0.994295,\ -0.106667)$

$$M_{CSP} = \begin{matrix} 1 & 0 & 0 & 0 \\ 0 & 0.994295 & 0.106667 & 0.08 \\ 0 & -0.106667 & 0.994295 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$s_0 = 0,\ D(0) = 0\ m,\ D'(0) = 0,\ \theta_s = 0$

For constant cant $\Delta D = 0$, so $\phi(s_0) = \phi_s = \phi_e = 1.463926346$.

$\mathbf{X}_s = (1,\ 0,\ 0)$

$\mathbf{Z}_s = (0,\ 0.106667,\ 0.994295)$

$\mathbf{Y}_s = \mathbf{Z}_s \times \mathbf{X}_s = (0,\ 0.994295,\ \ -0.106667)$

$\mathbf{Axis}_s = \mathbf{X}_s \times \mathbf{Y}_s = (0,\ 0.106667,\ 0.994295)$

$\mathbf{RefDir}_s = \mathbf{Y}_s \times \mathbf{Axis}_s = (1,\ 0,\ 0)$

$$M_{PCS} = \begin{matrix} 1 & 0 & 0 & 0 \\ 0 & 0.994295 & 0.106667 & 0 \\ 0 & -0.106667 & 0.994295 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$D(50\ m) = 0\ m,\ D'(50\ m) = 0,\ \theta_\ell = 0$

Since $\Delta D = 0$, $\phi(50\ m) = \phi_s = 1.463926346$.

$\mathbf{X}_\ell = (1,\ 0,\ 0)$

$\mathbf{Z}_\ell = (0,\ 0.106667,\ 0.994295)$

$\mathbf{Y}_\ell = \mathbf{Z}_\ell \times \mathbf{X}_\ell = (0,\ 0.994295,\ \ -0.106667)$

$$M_{PC\ell} = \begin{matrix} 1 & 0 & 0 & 50 \\ 0 & 0.994295 & 0.106667 & 0 \\ 0 & -0.106667 & 0.994295 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_{CSP} = \begin{matrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \\ 0 & 0 & 0 \end{matrix}$$

$$R_{PCS} = \begin{matrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \\ 0 & 0 & 0 \end{matrix}$$

$$R_{PC\ell} = \begin{matrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \\ 0 & 0 & 0 \end{matrix}$$

$$\backslash \mathrm{R}_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP} = \begin{matrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 & 0 & 0.994295 & -0.106667 & 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 & 0 & 0.106667 & 0.994295 & 0 & -0.106667 & 0.994295 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{matrix}$$

$$R_c = \begin{matrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \\ 0 & 0 & 0 \end{matrix}$$

Translation

$$\mathbf{T_c} = \mathbf{T_{CSP}} + \mathbf{T_{PC\ell}} - \mathbf{T_{PCS}}$$

$$\mathbf{T_{CSP}} = (0,\ 0.08,\ 0)^T$$

$$\mathbf{T_{PC\ell}} = (50,\ 0,\ 0)^T$$

$$\mathbf{T_{PCS}} = (0,\ 0,\ 0)^T$$

$$\mathbf{T_c} = (0,\ 0.08,\ 0)^T + (50,\ 0,\ 0)^T - (0,\ 0,\ 0)^T$$

$$\mathbf{T_c} = (50,\ 0.08,\ 0\ ,\ 0)^T$$

Assemble

$$M_c = \begin{bmatrix} R_c & \mathbf{T}_c \\ \mathbf{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{matrix} 1 & 0 & 0 & 50 \\ 0 & 0.994295 & 0.106667 & 0.08 \\ 0 & -0.106667 & 0.994295 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

4.4 Linear Transition

A linear transition in cant is represented with an `IfcClothoid`.

4.4.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{A_1}{|A_1^3|}s$$

$$\frac{d}{ds}D(s) = L^2\frac{A_1}{|A_1^3|}$$

$$a_0 = D_s,\ A_0 = \frac{L^{2/1}}{\sqrt[1]{a_0}}\frac{a_0}{|a_0|},\ A_0 = \frac{L^2}{a_0}$$

$$a_1 = \Delta D,\ A_1 = \frac{L^{\frac{3}{2}}}{\sqrt{|a_1|}}\frac{a_1}{|a_1|}$$

A_1 is the clothoid constant.

4.4.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a clothoid transition curve towards the left. The track banks at a constantly increasing rate resulting in a linear deviation in the right rail elevation relative to the left rail. The right rail raises 160 mm over the 100 m segment length.

The semantic representation of the cant alignment is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 0.16, 0., .LINEARTRANSITION.);
```

Compute the parent curve parameters.

$D_{sl} = 0\ m \quad D_{el} = 0\ m$

$D_{sr} = 0.16\ m \quad D_{er} = 0\ m$

$D_s = \frac{(0.16+0)}{2} = 0.08\ m$

$D_e = \frac{(0+0)}{2} = 0\ m$

$\Delta D = D_e - D_s = 0.0 - 0.08 = -0.08\ m$

$a_0 = D_s = 0.08\ m,\ A_0 = \frac{(100\ m)^2}{0.08\ m} = 125000\ m$

$a_1 = \Delta D = -0.08\ m \quad A_1 = \frac{(100\ m)^{\frac{3}{2}}}{\sqrt{|-0.08\ m|}} \frac{-0.08\ m}{|-0.08\ m|} = -3535.533906\ m$

The clothoid parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCCLOTHOID(#98,-3535.533905932738);
```

The cant segment begins with a deviating elevation of

$$D(0\ m) = (100\ m)^2 \left(\frac{1}{125000} + \frac{-3535.533906}{|-3535.533906|^3} (0\ m) \right) = 0.08\ m$$

The slope at the start of the segment is

$$D'(0\ m) = (100\ m)^2 \frac{-3535.533906}{|-3535.533906^3|} = -0.0008$$

$$\theta_0 = tan^{-1}(-0.0008) = -0.00079999$$

$dx_x = cos(\theta_0) = 0.999999680$

$dy_x = sin(\theta_0) = -0.000799999744$

The cross-slope at the start of the segment is

$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16-0.0}{1.5}\right) = 1.463926346$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.463926346) = 0.106666667$

$dz_z = sin(\phi_s) = sin(1.463926346) = 0.994294837$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10666666666666667,0.99429483666678176));
#102=IFCDIRECTION((0.999999680,-0.000799999744,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.4.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (0.9999996800,\ -0.000799999744,\ 0)$

$\mathbf{Axis}_p = (0,\ 0.106064981,\ 0.994359201)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0.0007954871,\ 0.9943588828,\ -0.1060649471)$

$$M_{CSP} = \begin{matrix} 0.999999683641039 & 0.00079543561769016 & 0 & 0 \\ -0.000790897527570178 & 0.9942945221127 & 0.106666666666667 & 0.08 \\ 8.48464658869504e-05 & -0.106666632921711 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$s_0 = 0,\ D(0) = 0\ m,\ D'(0) = -0.0008,\ \theta_s = -0.00079999$

$dx_x = \cos(\theta_s) = \cos(-0.00079999) = 0.999999680$

$dx_y = \sin(\theta_s) = \sin(-0.00079999) = -0.000799999744$

$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.463926346) = 0.106666667$

$dz_z = sin(\phi_s) = sin(1.463926346) = 0.994294837$

$\mathbf{X}_s = (0.999999680,\ -0.000799999744,\ 0)$

$\mathbf{Z}_s = (0,\ 0.106666667,\ 0.994294837)$

$\mathbf{Y}_s = \mathbf{Z}_s \times \mathbf{X}_s = (0.0007954871,\ 0.9943588828,\ -0.1060649471)$

$\mathbf{Axis}_s = \mathbf{X}_s \times \mathbf{Y}_s = (0.0000849,\ 0.106666667,\ 0.994294837)$

$\mathbf{RefDir}_s = \mathbf{Y}_s \times \mathbf{Axis}_s = (0.999999680,\ -0.000799999744,\ 0)$

$$M_{PCS} = \begin{matrix} 0.999999999968 & 7.95435869307971e-06 & 8.5333333327872e-07 & 0 \\ -7.999999999744e-06 & 0.994294836634964 & 0.10666666665984 & -0 \\ 0 & -0.106666666663253 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ **—** $M_{PC\ell}$

$D(50\ m) = 0.04\ m,\ D'(50\ m) = -0.0008,\ \theta_\ell = -0.00079999$

$\phi_e = \cos^{-1}(\frac{0.0}{1.5}) = \frac{\pi}{2}$ (both rails level at segment end)

$\phi(50\ m) = 1.464531464 + \left(\frac{\frac{\pi}{2} - 1.464531464}{-0.08}\right)(0.04 - 0.08) = 1.517663895$

$\mathbf{X}_\ell = (0.999999680,\ -0.000799999744,\ 0)$

$\mathbf{Z}_\ell = (0,\ 0.053107436,\ 0.998588804)$

$\mathbf{Y}_\ell = \mathbf{Z}_\ell \times \mathbf{X}_\ell = (0.000798871,\ 0.998588485,\ -0.053107419)$

$$M_{PC\ell} = \begin{matrix} 0.999999999968 & 7.98858152422898e-06 & 4.27276522453101e-07 & 50 \\ -7.999999999744e-06 & 0.998572690528623 & 0.0534095653066376 & -0.04 \\ 0 & -0.0534095653083467 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\mathbf{T}_c = \mathbf{T}_{CSP} + \mathbf{T}_{PC\ell} - \mathbf{T}_{PCS}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \mathbf{T}_c \\ \mathbf{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{matrix} 0.999999683613726 & 0.000795469840510406 & -4.2605681082593e-07 & 50 \\ -0.000794311703395977 & 0.99857237464344 & 0.0534095653134254 & 0.04 \\ 4.29111469629201e-05 & -0.0534095480769501 & 0.99857269055985 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

4.5 Helmert Curve

Like the Helmert transition spiral, the Helmert cant semantic definition maps to two `IfcCurveSegment` instances for the geometric representation. The parent curve is `IfcSecondOrderPolynomialSpiral`.

4.5.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_2^3}s^2 + \frac{A_1}{|A_1^3|}s + \frac{1}{A_0}$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{2s}{A_2^3} + \frac{A_1}{|A_1^3|}\right)$$

The polynomial coefficients carry a second subscript to indicate first half 1, and second half 2. For example, A_{21} is coefficient A_2 for the first half and A_{02} is coefficient A_0 for the second half.

In the first half of the cant transition

Constant Term:

$a_{01} = 4D_s$, $A_{01} = \frac{L^2}{|a_{o1}|}\frac{a_{01}}{|a_{o1}|}$

Linear Term:

$a_{11} = 0$, $A_{11} = \frac{L^{\frac{3}{2}}}{\sqrt{|a_{11}|}}\frac{a_{11}}{|a_{11}|} = 0$

Quadratic Term:

$a_{21} = 8\Delta D$, $A_{21} = \frac{L^{4/3}}{\sqrt[3]{|a_{21}|}}\frac{a_{21}}{|a_{21}|}$

In the second half

Constant Term:

$a_{02} = -4\Delta D + 4D_s$, $A_{02} = \frac{L^2}{|a_{02}|}\frac{a_{01}}{|a_{02}|}$

Linear Term:

$a_{12} = 16\Delta D$, $A_{12} = \frac{L^{\frac{3}{2}}}{\sqrt{|a_{12}|}}\frac{a_{12}}{|a_{12}|}$

Quadratic Term:

$a_{22} = -8\Delta D,\ A_{22} = \frac{L^{\frac{4}{3}}}{\sqrt[3]{|a_{22}|}} \frac{a_{22}}{|a_{22}|}$

4.5.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Helmert transition curve towards the left. The start cant is 160 mm and transitions to zero over 100 m.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.HELMERTCURVE.);
```

Compute the parent curve parameters

$$L = 100\ m$$

$$D_{sl} = 0\ m,\ D_{el} = 0\ m$$

$$D_{sr} = 0.16\ m,\ D_{er} = 0\ m$$

$$D_s = \frac{0+0.16\ m}{2} = 0.08\ m$$

$$D_e = \frac{0+0}{2} = 0.\ m$$

$$\Delta D = 0.0 - 0.08 = -0.08\ m$$

First half:

$$a_{01} = 4D_s = 4(0.08\ m) = 0.32\ m$$

$$A_{01} = \frac{L^2}{|a_{o1}|} \frac{a_{01}}{|a_{o1}|} = \frac{(100\ m)^2}{|0.32\ m|} \frac{0.32\ m}{|0.32\ m|} = 31250\ m$$

$$a_{11} = 0,\ A_{11} = 0$$

$$a_{21} = 8\Delta D = 8(-0.08\ m) = -0.64\ m$$

$$A_{21} = \frac{L^{4/3}}{\sqrt[3]{|a_{21}|}} \frac{a_{21}}{|a_{21}|} = \frac{(100\ m)^{\frac{4}{3}}}{\sqrt[3]{|-0.64\ m|}} \frac{-0.64\ m}{|-0.64\ m|} = -538.6086725\ m$$

The first half parent curve `IfcSecondOrderPolynomialSpiral` is

```
#104=IFCCARTESIANPOINT((0.,0.));
#105=IFCDIRECTION((1.,0.));
#106=IFCAXIS2PLACEMENT2D(#104,#105);
#107=IFCSECONDORDERPOLYNOMIALSPIRAL(#106,-538.60867250797071,$,31250.);
```

Determine the first half placement and trimming.

Note that the length of the first half curve is $L_1 = 50\ m$, half the length of the full Helmert transition.

[todo - Need to do a better job explaining why L/2 when A is computed on full L. There is a plot of the two parent curves planned, perhaps this will help explain the situation.]

The deviation elevation at the start of the first half transition is

$$D(0\ m) = (50\ m)^2 \left(\frac{1}{31250\ m} + \frac{1}{(-538.6086725\ m)^3} (0\ m)^2 \right) = 0.08\ m$$

$$D'(0\ m) = (50\ m)^2 \left(\frac{2(0\ m)}{(-538.6086725\ m)^3} \right) = 0$$

$\theta = 0,\ dx_x = 1, dy_x = 0$

The cross-slope at the start of the segment is

$\phi_s = \cos^{-1}\left(\frac{D_{sr}-D_{si}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16-0.0}{1.5}\right) = 1.463926346$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.463926346) = 0.106666667$

$dz_z = sin(\phi_s) = sin(1.463926346) = 0.994294837$

```
#108=IFCCARTESIANPOINT((0.,0.08,0.));
#109=IFCDIRECTION((0.,0.10666666666666667,0.99429483666678176));
#110=IFCDIRECTION((1.,0.,0.));
#111=IFCAXIS2PLACEMENT3D(#108,#109,#110);
#112=IFCCURVESEGMENT(.CONTINUOUS.,#111,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(50.),#107);
```

Second half:

$$a_{02} = -4\Delta D + 4D_s = -4(-0.08\ m) + 4(0.08)\ m = 0.64\ m$$

$$A_{02} = \frac{L^2}{|a_{02}|}\frac{a_{01}}{|a_{02}|} = \frac{(100\ m)^2}{|0.64\ m|}\frac{0.64\ m}{|0.64\ m|} = 15625\ m$$

$$a_{12} = 16\Delta D = 16(-0.08\ m) = -1.28\ m$$

$$A_{12} = \frac{L^{\frac{3}{2}}}{\sqrt{|a_{12}|}}\frac{a_{12}}{|a_{12}|} = \frac{(100\ m)^{\frac{3}{2}}}{\sqrt{|-1.28\ m|}}\frac{-1.28\ m}{|-1.28\ m|} = -883.8834765\ m$$

$$a_{22} = -8\Delta D = -8(-0.08\ m) = 0.64\ m$$

$$A_{22} = \frac{L^{\frac{4}{3}}}{\sqrt[3]{|a_{22}|}}\frac{a_{22}}{|a_{22}|} = \frac{(100\ m)^{\frac{4}{3}}}{\sqrt[3]{|0.64\ m|}}\frac{0.64\ m}{|0.64\ m|} = 538.6086725\ m$$

Figure 4.5.2-1 shows the first and second half parent curves.

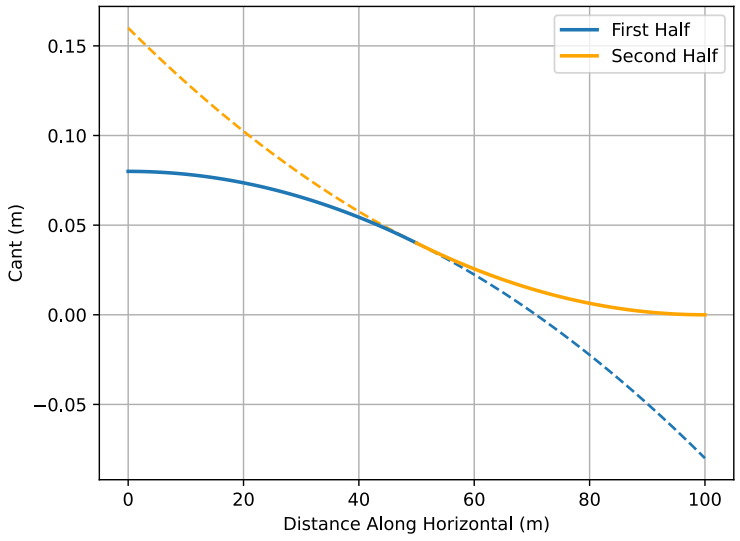


Figure 4.5.2-1 - `IfcPolygonalSpiralCurve` parent curves. The dashed lines represent the projection of the parent curve over the full length of the semantic segment.

The second half parent curve `IfcSecondOrderPolynomialSpiral` is

```
#113=IFCCARTESIANPOINT((0.,0.));
#114=IFCDIRECTION((1.,0.));
#115=IFCAXIS2PLACEMENT2D(#113,#114);
#116=IFCSECONDORDERPOLYNOMIALSPIRAL(#115,538.60867250797071,-883.8834764831845,15625.);
```

Determine the second half placement and trimming.

The starting elevation of the second half is the end elevation of the first half. Use the first half parent curve to determine the start of the second half curve.

$$D(50\ m) = (50\ m)^2\left(\frac{1}{31250\ m} + \frac{1}{(-538.6086725\ m)^3}(50\ m)^2\right) = 0.04\ m$$

$$D'(50\ m) = (50\ m)^2\left(\frac{2{\cdot}50\ m}{(-538.6086725\ m)^3}\right) = -0.0016$$

$dx_x = \frac{1}{\sqrt{(-0.0016)^2+1}} = 0.999998724,\ dy_x = \frac{-0.0016}{\sqrt{(-0.0016)^2+1}} = -0.00159999795$

The cant at the end of the first half is one-half of the total cant change over the length of the transition segment.

$\frac{(D_{sr}-D_{sl})+(D_{er}-D_{el})}{2} = \frac{(0.16-0.0)+(0.0+0.0)}{2} = 0.08$

The cross slope at the end of the first half is

$\phi(50\ m) = \cos^{-1}\left(\frac{0.08}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.08}{1.5}\right) = 1.517437677$

$dy_z = cos(\phi_s) = cos(1.517437677) = 0.053333333$

$dz_z = sin(\phi_s) = sin(1.517437677) = 0.998576765$

The trimming begins at 50 m and progresses for a length of 50 m for the second half segment.

```
#117=IFCCARTESIANPOINT((50.,0.04,0.));
#118=IFCDIRECTION((0.,0.053333333333333337,0.99857676497881498));
#119=IFCDIRECTION((0.9999987200024576,-0.0015999979520039342,0.));
#120=IFCAXIS2PLACEMENT3D(#117,#118,#119);
#121=IFCCURVESEGMENT(.CONTINUOUS.,#120,IFCLENGTHMEASURE(50.),IFCLENGTHMEASURE(50.),#116);
```

4.5.3 Compute Point on Curve

Compute the cant placement matrix for a point 75 m from the start of the curve segment using the algorithm in Section 4.2. 75 m falls in the second half of the Helmert transition, $\ell = 75\ m - 50\ m = 25\ m$.

Step 1 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (0.9999987200024576,\ -0.0015999979520039342,\ 0.)$

$\mathbf{Axis}_p = (0.,\ 0.053333333333333337,\ 0.99857676497881498)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0.00159772,\ 0.998575,\ -0.0533333)$

$$M_{CSP} = \begin{matrix} 0.999998723643333 & 0.00159772078470193 & 0 & 50 \\ -0.00159544685252706 & 0.998575490438703 & 0.0533333333333333 & 0.04 \\ 8.52117751841028e-05 & -0.0533332652609777 & 0.998576764978815 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$$D_s = D(50\ m) = (50\ m)^2\left(\frac{1}{(538.6086725\ m)^3}(50\ m)^2 + \frac{-883.8834765\ m}{|(-883.8834765\ m)^3|}(50\ m) + \frac{1}{15625\ m}\right) = 0.16\ m$$

$$D'(s) = D'(50\ m) = (50\ m)^2\left(\frac{(2)(50\ m)}{(538.6086725\ m)^3} + \frac{(-883.8834765\ m)}{|(-883.8834765\ m)^3|}\right) = -0.0016$$

$dx_x = \frac{1}{\sqrt{(-0.0016)^2+1}} = 0.99999872$

$dy_x = \frac{-0.0064}{\sqrt{(-0.0016)^2+1}} = -0.001599998$

$\phi_s = \tan^{-1}\left(\frac{dz_z}{dz_y}\right) = \tan^{-1}\left(\frac{0.99858080467782662}{0.053257642916150753}\right) = 1.517513475$

$\mathbf{X}_s = (0.99999872,\ -0.001599998,\ 0)$

$\mathbf{Z}_s = (0,\ 0.053257642916150753,\ 0.99858080467782662)$

$\mathbf{Y}_s = \mathbf{Z}_s \times \mathbf{X}_s = (0.0015977272423949591,\ 0.99857952649685078,\ -0.053257574746498774)$

$\mathbf{Axis}_s = \mathbf{X}_s \times \mathbf{Y}_s = (8.5212010523094261e-05,\ 0.053257506576933983,\ 0.99858080467782662)$

$\mathbf{RefDir}_s = \mathbf{Y}_s \times \mathbf{Axis}_s = (0.99999872,\ -0.001599998,\ 0)$

$$M_{PCS} = \begin{matrix} 0.999998720002458 & 0.00159772077888481 & 8.5333114880559e-05 & 0 \\ -0.00159999795200393 & 0.99857548680301 & 0.0533331968003495 & 0.0400000000000001 \\ 0 & -0.0533332650667977 & 0.998576764978815 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 3 — Evaluate the parent curve at $\ell = 25\ m$ — $M_{PC\ell}$

Add curve start $25 + 50 = 75\ m$.

$$D_\ell = D(75\ m) = (50\ m)^2\left(\frac{1}{(538.6086725\ m)^3}(75\ m)^2 + \frac{-883.8834765\ m}{|(-883.8834765\ m)^3|}(75\ m) + \frac{1}{15625\ m}\right) = 0.01\ m$$

$$D'_\ell = D'(75\ m) = (50\ m)^2\left(\frac{(2)(75\ m)}{(538.6086725\ m)^3} + \frac{(-883.8834765\ m)}{|(-883.8834765\ m)^3|}\right) = -0.0008$$

$dx_x = cos(\theta) = 0.99999968000015360$

$dx_y = sin(\theta) = -0.00079999974400011946$

$\phi(\ell) = 1.517437677 + \left(\frac{\frac{\pi}{2} - 1.517437677}{-0.04}\right)(0.01 - 0.08) = 1.5574756139049735$

$dz_y = \cos(1.5574756139049735) = 0.013320318952445506$

$dz_z = \sin(1.5574756139049735) = 0.99991128061593804$

$\mathbf{X}_\ell = (0.99999968000015360,\ -0.00079999974400011946,\ 0)$

$\mathbf{Z}_\ell = (0,\ 0.013320318952445506,\ 0.99991128061593804)$

$\mathbf{Y}_\ell = \mathbf{Z}_\ell \times \mathbf{X}_\ell = (0.00079992876851558200,\ 0.99991096064448182,\ -0.013320314689945486)$

$\mathbf{Axis}_\ell = \mathbf{X}_\ell \times \mathbf{Y}_\ell = (1.0656248341957419e-05,\ 0.013320310427446832,\ 0.99991128061593804)$

$\mathbf{RefDir}_\ell = \mathbf{Y}_\ell \times \mathbf{Axis}_\ell = (0.99999968000015360,\ -0.00079999974400011946,\ 0)$

$$M_{PC\ell} = \begin{matrix} 0.999999680000154 & 0.000799928566440938 & 1.06714066139122e-05 & 25 \\ -0.000799999744000119 & 0.999910708051177 & 0.0133392582673903 & 0.0100000000000002 \\ 0 & -0.0133392625359523 & 0.999911028022552 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\mathbf{T}_c = \mathbf{T}_{CSP} + \mathbf{T}_{PC\ell} - \mathbf{T}_{PCS}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \mathbf{T}_c \\ \mathbf{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{matrix} 0.999999677269901 & 0.000799928563528498 & -7.4661790264052e-05 & 75 \\ -0.000798861459176414 & 0.999910704410622 & 0.0133393264368146 & 0.0100000000000001 \\ 8.53256315305252e-05 & -0.0133392624873857 & 0.999911020741441 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

4.6 Bloss Curve

A Bloss transition in cant is represented with an `IfcThirdOrderPolynomialSpiral`.

4.6.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{2|A_1^3|}s + \frac{1}{A_0}$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{3A_3}{|A_3^5|}s^2 + \frac{2}{A_2^3}s + \frac{A_1}{2|A_1^3|}\right)$$

Constant Term

$a_0 = D_s,\ A_0 = \frac{L^{\frac{2}{1}}}{\sqrt[0]{|a_0|}}\frac{a_0}{|a_0|} = \frac{L^2}{|a_0|}\frac{a_0}{|a_0|}$

Linear Term

$a_1 = 0,\ A_1$

Quadratic Term

$a_2 = 3\Delta D,\ A_2 = \frac{L^{\frac{4}{3}}}{\sqrt[3]{|a_2|}}\frac{a_2}{|a_2|}$

Cubic Term

$a_3 = -2\Delta D,\ A_3 = \frac{L^{\frac{5}{4}}}{\sqrt[4]{|a_3|}}\frac{a_3}{|a_3|}$

4.6.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Bloss transition curve towards the left. The start cant is 160 mm and transitions to zero over 100 m.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.BLOSSCURVE.);
```

Compute the parent curve parameters

$D_s = \frac{0+0.16}{2} = 0.08\ m,\ D_e = \frac{0+0\ m}{2} = 0.\ m,\ \Delta D = 0.0 - 0.16 = -0.08\ m$

Constant Term

$a_0 = -0.08\ m,\ A_0 = \frac{(100\ m)^2}{|-0.08\ m|}\frac{-0.08\ m}{|-0.08\ m|} = 125000\ m$

Linear Term

$A_1 = 0\ m$

Quadratic Term

$a_2 = 3(-0.08\ m) = -0.24\ m,\ A_2 = \frac{(100\ m)^{\frac{4}{3}}}{\sqrt[3]{|-0.24\ m|}}\left(\frac{-0.24\ m}{|-0.24\ m|}\right) = -746.9007911\ m$

Cubic Term

$a_3 = -2\Delta D = -2(-0.08\ m) = 0.16\ m,\ A_3 = \frac{(100\ m)^{\frac{5}{4}}}{\sqrt[4]{|0.16\ m|}}\left(\frac{0.16\ m}{|0.16\ m|}\right) = 500\ m$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCTHIRDORDERPOLYNOMIALSPIRAL(#98,500.00000000000006,-746.90079109286057,$,125000.);
```

$$D_0 = D(0\ m) = (100\ m)^2\left(\frac{500\ m}{|(500\ m)^5|}(0\ m)^3 + \frac{1}{(-746.9007911\ m)^3}(0\ m)^2 + \frac{1}{125000\ m}\right) = 0.08\ m$$

$$D'(0) = 0,\ \theta_0 = 0$$

$dx_x = cos(\theta_0) = 1$

$dy_x = sin(\theta_0) = 0$

The cross-slope at the start of the segment is

$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.463926346) = 0.106666667$

$dz_z = sin(\phi_s) = sin(1.463926346) = 0.994294837$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10666666666666667,0.99429483666678176));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.6.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1,\ 0,\ 0)$

$\mathbf{Axis}_p = (0,\ 0.106667,\ 0.994295)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0,\ 0.994295,\ -0.1066667)$

$$M_{CSP} = \begin{matrix} 1 & 0 & 0 & 0 \\ -0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$s_0 = 0,\ D(0) = 0.08\ m,\ D'(0) = 0,\ \theta_s = 0$

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0}{1.5}\right) = 1.464531464$$

$\mathbf{X}_s = (1,\ 0,\ 0)$

$\mathbf{Z}_s = (0,\ 0.106667,\ 0.994295)$

$\mathbf{Y}_s = \mathbf{Z}_s \times \mathbf{X}_s = (0,\ 0.994295,\ -0.106667)$

$\mathbf{Axis}_s = \mathbf{X}_s \times \mathbf{Y}_s = (0,\ 0.106667,\ 0.994295)$

$\mathbf{RefDir}_s = \mathbf{Y}_s \times \mathbf{Axis}_s = (1,\ 0,\ 0)$

$$M_{PCS} = \begin{matrix} 1 & 0 & -0 & 0 \\ 0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$M_{PCS} = M_{CSP}$, because $\theta_s = 0$ and the placement location and orientation match the parent curve at the trim start exactly.

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$D(50\ m) = 0.04\ m$

$D'(50\ m) = (100\ m)^2\left(3\frac{500\ m}{|(500\ m)^5|}(50\ m)^2 + \frac{2}{(-746.9007911\ m)^3}(50\ m)\right) = -0.0012$

$\theta_\ell = \tan^{-1}(-0.0012) = -0.0011999994240005001$

$\phi(\ell) = 1.464531464 + \left(\frac{\frac{\pi}{2} - 1.464531464}{-0.08}\right)(0.04 - 0.08) = 1.517663895$

$\mathbf{X}_\ell = (0.9999980,\ -0.0019997,\ 0)$

$\mathbf{Z}_\ell = (0,\ 0.053107436,\ 0.998588804)$

$\mathbf{Y}_\ell = \mathbf{Z}_\ell \times \mathbf{X}_\ell = (0.001997,\ 0.998587,\ -0.053107)$

$\mathbf{Axis}_\ell = \mathbf{X}_\ell \times \mathbf{Y}_\ell = (0.000106,\ 0.053107,\ 0.998589)$

$\mathbf{RefDir}_\ell = \mathbf{Y}_\ell \times \mathbf{Axis}_\ell = (0.9999980,\ -0.0019997,\ 0)$

$$M_{PC\ell} = \begin{matrix} 0.999999280000778 & 0.00119828636590682 & 6.40913860804715e-05 & 50 \\ -0.00119999913600094 & 0.998571971589017 & 0.0534094884003928 & 0.0399999999999999 \\ 0 & -0.0534095268552106 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\mathbf{T}_c = \mathbf{T}_{CSP} + \mathbf{T}_{PC\ell} - \mathbf{T}_{PCS}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \mathbf{T}_c \\ \mathbf{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{matrix} 0.999999280000778 & 0.00119828636590682 & 6.40913860804715e-05 & 50 \\ -0.00119999913600094 & 0.998571971589017 & 0.0534094884003928 & 0.0399999999999999 \\ 0 & -0.0534095268552106 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

4.7 Cosine Curve

A Cosine transition in cant is represented with an `IfcCosineSpiral`.

4.7.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{1}{A_1}\cos\left(\pi\frac{s}{L}\right)$

$\frac{d}{ds}D(s) = L^2\left(-\frac{\pi}{A_1 L}\sin\left(\pi\frac{s}{L}\right)\right)$

Constant term,

$a_0 = D_s + \frac{\Delta D}{2}$

$A_0 = L^2\frac{1}{a_0}\frac{a_0}{|a_0|}$

Cosine term,

$a_1 = -\frac{1}{2}\Delta D$

$A_1 = L^2\frac{1}{a_1}\frac{a_1}{|a_1|}$

4.7.2 Semantic Definition to Geometry Mapping

Consider and alignment segment that has a Cosine transition curve towards the left. The start cant is 160 mm and transitions to zero over 100 m.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.COSINECURVE.);
```

Compute the parent curve parameters

$D_s = \frac{0+0.16}{2} = 0.08\ m,\ D_e = \frac{0+0\ m}{2} = 0.\ m,\ \Delta D = 0.0\ m - 0.08\ m = -0.08\ m$

$a_0 = 0.08\ m + \frac{-0.08\ m}{2} = 0.04$

$A_0 = (100\ m)^2\frac{1}{0.04\ m}\frac{0.04\ m}{|0.04\ m|} = 250000\ m$

$a_1 = -\frac{1}{2}(-0.08\ m) = -0.04\ m$

$A_1 = (100\ m)^2\frac{1}{-0.04\ m}\frac{-0.04\ m}{|-0.04\ m|} = 250000\ m$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCCOSINESPIRAL(#98,250000.,250000.);
```

$$D(0\ m) = (100\ m)^2\left(\frac{1}{250000\ m} + \frac{1}{250000\ m}\cos\left(\pi\frac{0\ m}{100\ m}\right)\right) = 0.08\ m$$

$$D'(0\ m) = (100\ m)^2\left(-\frac{\pi}{(250000\ m)(100\ m)}\sin\left(\pi\frac{0\ m}{100\ m}\right)\right) = 0$$

$\theta_0 = 0,\ dx_x = 1,\ dy_x = 0$

The cross-slope at the start of the segment is

$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.463926346) = 0.106666667$

$dz_z = sin(\phi_s) = sin(1.463926346) = 0.994294837$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10666666666666667,0.99429483666678176));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.7.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1,\ 0,\ 0)$

$\mathbf{Axis}_p = (0,\ 0.106064981,\ 0.994359201)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0,\ 0.9943592,\ -0.10606498)$

$$M_{CSP} = \begin{matrix} 1 & 0 & 0 & 0 \\ -0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$s_0 = 0,\ D(0) = 0.08\ m,\ D'(0) = 0,\ \theta_s = 0$

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0}{1.5}\right) = 1.464531464$$

$\mathbf{X}_s = (1,\ 0,\ 0)$

$\mathbf{Z}_s = (0,\ 0.106064981,\ 0.994359201)$

$\mathbf{Y}_s = \mathbf{Z}_s \times \mathbf{X}_s = (0,\ 0.9943592,\ -0.10606498)$

$\mathbf{Axis}_s = \mathbf{X}_s \times \mathbf{Y}_s = (0,\ 0.106064981,\ 0.994359201)$

$\mathbf{RefDir}_s = \mathbf{Y}_s \times \mathbf{Axis}_s = (1,\ 0,\ 0)$

$$M_{PCS} = \begin{matrix} 1 & 0 & 0 & 0 \\ -0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$M_{PCS} = M_{CSP}$, because $\theta_s = 0$ and the placement location and orientation match the parent curve at the trim start exactly.

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$D(50\ m) = 0.04\ m,\ D'(50\ m) = -0.00125664,\ \theta_\ell = -0.001256636$

$\phi(\ell) = 1.464531464 + \left(\frac{\frac{\pi}{2} - 1.464531464}{-0.08}\right)(0.04 - 0.08) = 1.517663895$

$\mathbf{X}_\ell = (0.99999921,\ -0.001256636,\ 0)$

$\mathbf{Z}_\ell = (0,\ 0.053107436,\ 0.998588804)$

$\mathbf{Y}_\ell = \mathbf{Z}_\ell \times \mathbf{X}_\ell = (0.001255,\ 0.998588,\ -0.053107)$

$\mathbf{Axis}_\ell = \mathbf{X}_\ell \times \mathbf{Y}_\ell = (0.0000667,\ 0.053107,\ 0.998589)$

$\mathbf{RefDir}_\ell = \mathbf{Y}_\ell \times \mathbf{Axis}_\ell = (0.99999921,\ -0.001256636,\ 0)$

$$M_{PC\ell} = \begin{matrix} 0.999999999921043 & 1.25484345139712e-05 & 6.71164391931997e-07 & 50 \\ -1.2566370613367e-05 & 0.998572690481733 & 0.0534095653016217 & 0.04 \\ 0 & -0.0534095653058388 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\mathbf{T}_c = \mathbf{T}_{CSP} + \mathbf{T}_{PC\ell} - \mathbf{T}_{PCS}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \mathbf{T}_c \\ \mathbf{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{matrix} 0.999999999921043 & 1.25484345139712e-05 & 6.71164391931997e-07 & 50 \\ -1.2566370613367e-05 & 0.998572690481733 & 0.0534095653016218 & 0.04 \\ 0 & -0.0534095653058388 & 0.998572690560577 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

4.8 Sine Curve

A Sine transition in cant is represented with an `IfcSineSpiral`

4.8.1 Parent Curve Parametric Equations

The deviation elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 s + \frac{1}{A_2}\sin\left(2\pi\frac{s}{L}\right)$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 + \frac{2\pi}{LA_2}\cos\left(2\pi\frac{s}{L}\right)\right)$$

Constant term:

$$a_0 = D_s$$

$$A_0 = L^2 \frac{1}{a_0}\frac{a_0}{|a_0|}$$

Linear term:

$$a_1 = \Delta D$$

$$A_1 = L^{\frac{3}{2}} \frac{1}{\sqrt{|a_1|}}\frac{a_1}{|a_1|}$$

Sine term:

$$a_2 = -\frac{1}{2\pi}\Delta D$$

$$A_2 = L^2 \frac{1}{a_2}\frac{a_2}{|a_2|}$$

4.8.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Sine transition curve towards the left. The start cant is 160 mm and transitions to zero over 100 m.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.SINECURVE.);
```

Compute the paraent curve parameters

$D_s = \frac{0+0.16}{2} = 0.08\ m,\ D_e = \frac{0+0\ m}{2} = 0.\ m,\ \Delta D = 0.0\ m - 0.08\ m = -0.08\ m$

Constant term: $a_0 = 0.08\ m$

$A_0 = (100\ m)^2 \frac{1}{0.08\ m}\frac{0.08\ m}{|0.08\ m|} = 125000\ m$

Linear term: $a_1 = -0.08\ m$

$A_1 = (100\ m)^{\frac{3}{2}} \frac{1}{\sqrt{|-0.08\ m|}}\frac{-0.08\ m}{|-0.08\ m|} = -3535.533906\ m$

Sine term: $a_2 = -\frac{1}{2\pi}(0.08\ m) = -0.0127324\ m$

$A_2 = (100\ m)^2 \frac{1}{-0.0127324\ m}\frac{-0.0127324\ m}{|-0.0127324\ m|} = 785398.1634\ m$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCSINESPIRAL(#98,785398.16339744814,-3535.533905932738,125000.);
```

$$D(0\ m) = (100\ m)^2 \left(\frac{1}{125000\ m} + \left(\frac{-3535.533906\ m}{|-3535.533906\ m|}\right)\left(\frac{1}{-3535.533906\ m}\right)^2 (0\ m) + \frac{1}{785398.1634\ m}\sin\left(2\pi\frac{0\ m}{100\ m}\right)\right) = 0.08\ m$$

$$D'(0\ m) = (100\ m)^2 \left(\frac{-3535.533906}{|-3535.533906|}\left(\frac{1}{-3535.533906}\right)^2 + \frac{2\pi}{(100\ m)(785398.1634\ m)}\cos\left(2\pi\frac{0\ m}{100\ m}\right)\right) = 0.$$

$\theta_0 = 0$

$dx_x = cos(\theta_0) = 1$

$dx_y = sin(\theta_0) = 0$

The cross-slope at the start of the segment is

$\phi_s = \cos^{-1}\left(\frac{D_{sr}-D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16-0.0}{1.5}\right) = 1.463926346$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.463926346) = 0.106666667$

$dz_z = sin(\phi_s) = sin(1.463926346) = 0.994294837$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10666666666666667,0.99429483666678176));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.8.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1,\ 0,\ 0)$

$\mathbf{Axis}_p = (0,\ 0.106064981,\ 0.994359201)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0,\ 0.9943592,\ -0.10606498)$

$$M_{CSP} = \begin{matrix} 1 & 0 & 0 & 0 \\ -0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$s_0 = 0,\ D(0) = 0.08\ m,\ D'(0) = 0,\ \theta_s = 0$

todo - check this value, it is different than above for the same inputs

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0}{1.5}\right) = 1.464531464$$

$\mathbf{X}_s = (1,\ 0,\ 0)$

$\mathbf{Z}_s = (0,\ 0.106064981,\ 0.994359201)$

$\mathbf{Y}_s = \mathbf{Z}_s \times \mathbf{X}_s = (0,\ 0.9943592,\ -0.10606498)$

$\mathbf{Axis}_s = \mathbf{X}_s \times \mathbf{Y}_s = (0,\ 0.106064981,\ 0.994359201)$

$\mathbf{RefDir}_s = \mathbf{Y}_s \times \mathbf{Axis}_s = (1,\ 0,\ 0)$

$$M_{PCS} = \begin{matrix} 1 & -3.36880194376639e-21 & -3.61400724161835e-22 & 0 \\ 3.3881317890172e-21 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$M_{PCS} = M_{CSP}$, because $\theta_s = 0$ and the placement location and orientation match the parent curve at the trim start exactly.

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$D(50\ m) = 0.04\ m,\ D'(50\ m) = -0.0016,\ \theta_\ell = -0.0016$

$\phi(\ell) = 1.464531464 + \left(\frac{\frac{\pi}{2} - 1.464531464}{-0.08}\right)(0.04 - 0.08) = 1.517663895$

$\mathbf{X}_\ell = (0.99999872,\ -0.0016,\ 0)$

$\mathbf{Z}_\ell = (0,\ 0.053107436,\ 0.998588804)$

$\mathbf{Y}_\ell = \mathbf{Z}_\ell \times \mathbf{X}_\ell = (0.001598,\ 0.998588,\ -0.053107)$

$\mathbf{Axis}_\ell = \mathbf{X}_\ell \times \mathbf{Y}_\ell = (0.0000850,\ 0.053107,\ 0.998589)$

$\mathbf{RefDir}_\ell = \mathbf{Y}_\ell \times \mathbf{Axis}_\ell = (0.99999872,\ -0.0016,\ 0)$

$$M_{PC\ell} = \begin{matrix} 0.999999999872 & 1.59771630469242e-05 & 8.54553044742128e-07 & 50 \\ -1.5999999997952e-05 & 0.99857269043276 & 0.053409565296383 & 0.04 \\ 0 & -0.0534095653032194 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\mathbf{T}_c = \mathbf{T}_{CSP} + \mathbf{T}_{PC\ell} - \mathbf{T}_{PCS}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \mathbf{T}_c \\ \mathbf{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{matrix} 0.999999999872 & 1.59771630469242e-05 & 8.54553044742129e-07 & 50 \\ -1.5999999997952e-05 & 0.99857269043276 & 0.053409565296383 & 0.04 \\ -1.80958646087621e-22 & -0.0534095653032194 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$\mathbf{Axis} = (0.0000850,\ 0.053107,\ 0.998589), \quad \phi(50\ m) = 1.517663895$

4.9 Viennese Bend

A Viennese Bend transition in cant is represented with an `IfcSeventhOrderPolynomialSpiral`.

4.9.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{A_7}{|A_7^9|}s^7 + \frac{1}{A_6^7}s^6 + \frac{A_5}{|A_5^7|}s^5 + \frac{1}{A_4^5}s^4 + \frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{2|A_1^3|}s + \frac{1}{A_0}$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{7A_7}{|A_7^9|}s^6 + \frac{6}{A_6^7}s^5 + \frac{5A_5}{|A_5^7|}s^4 + \frac{4}{A_4^5}s^3 + \frac{3A_3}{|A_3^5|}s^2 + \frac{2}{A_2^3}s + \frac{A_1}{2|A_1^3|}\right)$$

$$D_s = \frac{D_{sl}+D_{sr}}{2},\ D_e = \frac{D_{el}+D_{er}}{2},\ \Delta D = D_e - D_s$$

Constant Term

$$a_0 = D_1, A_0 = \frac{L^2}{|a_0|}\frac{a_0}{|a_0|}$$

Linear Term

$$a_1 = 0, A_1 = \frac{L^{\frac{3}{2}}}{\sqrt{|a_1|}}\frac{a_1}{|a_1|}$$

Quadratic Term

$$a_2 = 0, A_2 = \frac{L^{\frac{4}{3}}}{\sqrt[3]{|a_2|}}\frac{a_2}{|a_2|}$$

Cubic Term

$$a_3 = 0, A_3 = \frac{L^{\frac{5}{4}}}{\sqrt[4]{|a_3|}} \frac{a_3}{|a_3|}$$

Quartic Term

$$a_4 = 35\Delta D, A_4 = \frac{L^{\frac{6}{5}}}{\sqrt[5]{|a_4|}} \frac{a_4}{|a_4|}$$

Quintic Term

$$a_5 = -84\Delta D, A_5 = \frac{L^{\frac{7}{6}}}{\sqrt[6]{|a_5|}} \frac{a_5}{|a_5|}$$

Sextic Term

$$a_6 = 70\Delta D, A_6 = \frac{L^{\frac{8}{7}}}{\sqrt[7]{|a_6|}} \frac{a_6}{|a_6|}$$

Septic Term

$$a_7 = -20\Delta D, A_7 = \frac{L^{\frac{9}{8}}}{\sqrt[8]{|a_7|}} \frac{a_7}{|a_7|}$$

4.9.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Viennese Bend transition curve **???**. [todo - finish the description]

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.10,0.03,.VIENNESEBEND.);
```

Compute the parent curve parameters.

$$D_{sl} = 0.0\ m$$

$$D_{el} = 0.0\ m$$

$$D_{sr} = 0.1\ m$$

$$D_{er} = 0.03\ m$$

$$D_s = \frac{0\ m + 0.1\ m}{2} = 0.05\ m, D_e = \frac{0\ m + 0.03\ m}{2} = 0.015\ m,\ \Delta D = 0.015 - 0.05 = -0.035\ m$$

Constant Term

$$a_0 = 0.05\ m, A_0 = \frac{(100\ m)^2}{|0.05\ m|} \frac{0.05\ m}{|0.05\ m|} = 200000\ m$$

Linear Term

$$a_1 = 0, A_1 = 0\ m$$

Quadratic Term

$$a_2 = 0, A_2 = 0\ m$$

Cubic Term

$$a_3 = 0, A_3 = 0\ m$$

Quartic Term

$$a_4 = 35(-0.035\ m) = -1.225\ m, A_4 = \frac{(100\ m)^{\frac{6}{5}}}{\sqrt[5]{|-1.225\ m|}} \frac{-1.225\ m}{|-1.225\ m|} = -241.1974890085123\ m$$

Quintic Term

$$a_5 = -84(-0.035\ m) = 2.94\ m, A_5 = \frac{(100\ m)^{\frac{7}{6}}}{\sqrt[6]{|2.94\ m|}}\ \frac{2.94\ m}{|2.94\ m|} = 180.0012184608678\ m$$

Sextic Term

$$a_6 = 70(-0.035\ m) = -2.45\ m, A_6 = \frac{(100\ m)^{\frac{8}{7}}}{\sqrt[7]{|-2.45\ m|}}\ \frac{-2.45\ m}{|-2.45\ m|} = -169.87095595653895\ m$$

Septic Term

$$a_7 = -20(-0.035\ m) = 0.7\ m, A_7 = \frac{(100\ m)^{\frac{9}{8}}}{\sqrt[8]{|0.7\ m|}}\ \frac{0.7\ m}{|0.7\ m|} = 185.93568367635672\ m$$

The parent curve is

```
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCSEVENTHORDERPOLYNOMIALSPIRAL(#98,185.93568367635649,-169.87095595653892,180.00121846086768,-241.19748900851218,$,$,$,200000.);
```

$$D(0\ m) = (100\ m)^2 \left(\frac{185.93568367635672\ m}{|(185.93568367635672\ m)^9|}(0\ m)^7 + \frac{1}{(-169.87095595653895\ m)^7}(0\ m)^6 + \frac{180.0012184608678\ m}{|(180.0012184608678\ m)^7|}(0\ m)^5 + \frac{1}{(-241.1974890085123\ m)^5} \right.$$

$D'(0) = 0,\ \theta_0 = 0$

$dx_x = cos(\theta) = 1$

$dx_y = sin(\theta) = 0$

todo - fix this, mixing cos and tan

The cross-slope at the start of the segment is

$\phi_s = \cos^{-1}\left(\frac{D_{sr} - Dsl}{D_{rh}}\right) = tan^{-1}\left(\frac{0.1 - 0.0}{1.5}\right) = 1.504080178$

The cross slope orientation is

$dy_z = cos(\phi_s) = 0.066666667$

$dz_z = sin(\phi_s) = 0.997775303$

```
#100=IFCCARTESIANPOINT((0.,0.05,0.));
#101=IFCDIRECTION((0.,0.066666666666666666,0.9977753031397174));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.9.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1,\ 0,\ 0)$

$\mathbf{Axis}_p = (0,\ 0.066519011,\ 0.99778516)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0,\ 0.99778516,\ -0.066519011)$

$$M_{CSP} = \begin{matrix} 1 & 0 & 0 & 0 \\ -0 & 0.997775303139718 & 0.0666666666666667 & 0.05 \\ 0 & -0.0666666666666667 & 0.997775303139718 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$s_0 = 0,\ D(0) = 0.05\ m,\ D'(0) = 0,\ \theta_s = 0$

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.10 - 0.0}{1.5}\right) = 1.504228163$$

$\mathbf{X}_s = (1,\ 0,\ 0)$

$\mathbf{Z}_s = (0,\ 0.066519011,\ 0.99778516)$

$\mathbf{Y}_s = \mathbf{Z}_s \times \mathbf{X}_s = (0,\ 0.99778516,\ -0.066519011)$

$\mathbf{Axis}_s = \mathbf{X}_s \times \mathbf{Y}_s = (0,\ 0.066519011,\ 0.99778516)$

$\mathbf{RefDir}_s = \mathbf{Y}_s \times \mathbf{Axis}_s = (1,\ 0,\ 0)$

$$M_{PCS} = \begin{matrix} 1 & 0 & -0 & 0 \\ 0 & 0.997775303139718 & 0.0666666666666667 & 0.05 \\ 0 & -0.0666666666666667 & 0.997775303139718 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$M_{PCS} = M_{CSP}$, because $\theta_s = 0$ and the placement location and orientation match the parent curve at the trim start exactly.

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$D(50\ m) = 0.0325\ m,\ D'(50\ m) = -0.00076525,\ \theta_\ell = -0.00076562$

$\phi_e = cos^{-1}\left(\frac{D_{er} - D_{el}}{D_{rh}}\right) = cos^{-1}\left(\frac{0.03 - 0.0}{1.5}\right) = 1.550794993$

$\phi(\ell) = 1.504228163 + \left(\frac{1.550799 - 1.504228163}{-0.035}\right)(0.0325 - 0.05) = 1.527511578$

$\mathbf{X}_\ell = (0.999999707,\ -0.00076562,\ 0)$

$\mathbf{Z}_\ell = (0,\ 0.04327,\ 0.999063)$

$\mathbf{Y}_\ell = \mathbf{Z}_\ell \times \mathbf{X}_\ell = (0.000765,\ 0.999063,\ -0.04327)$

$\mathbf{Axis}_\ell = \mathbf{X}_\ell \times \mathbf{Y}_\ell = (3.31e-05,\ 0.04327,\ 0.999063)$

$\mathbf{RefDir}_\ell = \mathbf{Y}_\ell \times \mathbf{Axis}_\ell = (0.999999707,\ -0.00076562,\ 0)$

$$M_{PC\ell} = \begin{matrix} 0.999999706909309 & 0.000764905208550319 & 3.318611612348e-05 & 50 \\ -0.000765624775602475 & 0.999059864228941 & 0.0433451312633188 & 0.0324999999999996 \\ 0 & -0.043345143967377 & 0.999060157044173 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\mathbf{T}_c = \mathbf{T}_{CSP} + \mathbf{T}_{PC\ell} - \mathbf{T}_{PCS}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \mathbf{T}_c \\ \mathbf{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{matrix} 0.999999706909309 & 0.000764905208550319 & 3.318611612348e-05 & 50 \\ -0.000765624775602475 & 0.999059864228941 & 0.0433451312633188 & 0.0324999999999996 \\ 0 & -0.043345143967377 & 0.999060157044173 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

4.10 Combined 3D

The cant segment result is computed with the 2D vertical and 2D horizontal to complete the evaluation of a point on the alignment. The result is a 3D point and reference frame.

Using the example horizontal Bloss curve from Section 2.8 M_h at $s = 50\ m$ is

$$M_h = \begin{matrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Assume a vertical alignment that is flat. At $s = 50\ m$

$$M_v = \begin{matrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

From the Bloss cant example in Section 4.6

$$M_c = \begin{matrix} 0.999999280 & 0.00119830570 & 0.0 & 50.0 \\ -0.00119999914 & 0.998588085 & 0.0531073592 & 0.04 \\ 0 & 0 & 0.9985888041 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Step 5 — Combine with horizontal and vertical to produce the 3D placement matrix

Construct M_v' as described in Section 3.2

$$M_v' = \begin{matrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Construct M_c' by zeroing the distance-along ℓ component in row 1, column 4 and moving the deviating elevation D from row 2 to row 3 in column 4 of M_c:

$$M_c' = \begin{matrix} 0.999999280 & 0.00119830570 & 0.0 & 0.0 \\ -0.00119999914 & 0.998588085 & 0.0531073592 & 0 \\ 0 & 0 & 0.9985888041 & 0.04 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Extract position vectors from each modified matrix, $M_v',\ M_c'$, (setting row 4 to zero), then zero column 4:

$$P_v = M_v' \text{ column 4, row 4 set to 0} = \begin{matrix} 0 \\ 0 \\ 0 \\ 0 \end{matrix}, \qquad P_c = M_c' \text{ column 4, row 4 set to 0} = \begin{matrix} 0 \\ 0 \\ 0.04 \\ 0 \end{matrix}$$

$$M_v'' = M_v' \text{ with column 4 set to } (0,0,0,1)^T, \qquad M_c'' = M_c' \text{ with column 4 set to } (0,0,0,1)^T$$

$$M_v'' = \begin{matrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_c'' = \begin{matrix} 0.999999280 & 0.00119830570 & 0.0 & 0.0 \\ -0.00119999914 & 0.998588085 & 0.0531073592 & 0.0 \\ 0 & 0 & 0.9985888041 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Multiply the three orientation matrices, then add back both position offsets:

$$M' = M_h \cdot M_v'' \cdot M_c''$$

$$M' = \begin{matrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} \begin{matrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} \begin{matrix} 0.999999280 & 0.00119830570 & 0.0 & 0.0 \\ -0.00119999914 & 0.998588085 & 0.0531073592 & 0.0 \\ 0 & 0 & 0.9985888041 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M' = \begin{matrix} 0.999473545 & -0.0324443047 & 0 & 49.9962109 \\ 0.0324443047 & 0.999473545 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

$$M_{3Dcant} = \begin{matrix} 0.999473545 & -0.0324443047 & 0 & 49.9962109 \\ 0.0324443047 & 0.999473545 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix} + \begin{matrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix} + \begin{matrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.04 \\ 0 & 0 & 0 & 0 \end{matrix}$$

$$M_{3Dcant} = \begin{matrix} 0.999473545 & -0.0324443047 & 0 & 49.9962109 \\ 0.0324443047 & 0.999473545 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0.04 \\ 0 & 0 & 0 & 1 \end{matrix}$$

4.11 Deviation from EnrichIfc4x3 Reference Implementation

The cant calculations in this section deviate from the ones performed by the EnrichIfc4x3 reference implementation. The calculations in the reference implementation, published at IFC-Rail-Unit-Test-Reference-Code/EnrichIFC4x3/EnrichIFC4x3/business2geometry at master · bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code (github.com), yield coefficients for sine, cosine, clothoid, and polynomial spirals that are not in units of length. The IFC specification is clear that the coefficients have units of length and are represented by `IfcLengthMeasure`.

The basic form of the coefficient of the i^{th} term is

$$A_i = \frac{L^{\frac{i+2}{i+1}}}{\sqrt[(i+1)]{|a_i|}} \frac{a_i}{|a_i|}$$

Performing a dimensional analysis with l representing term with length units, we see that the resulting coefficient A_i has units of length.

$$\frac{l^{\frac{i+2}{i+1}}}{\sqrt[(i+1)]{|l|}} \frac{l}{|l|} = \frac{l^{\frac{i+2}{i+1}}}{l^{\frac{1}{i+1}}} = l^{\frac{i+2}{i+1}} l^{\frac{-1}{(i+1)}} = l^{\frac{i+1}{i+1}} = l$$

The error in the EnrichIfc4x3 reference implementation is illustrated by way of an example. Consider the Bloss Curve example GENERATED__CantAlignment_BlossCurve_100.0_1000_300_1_Meter.ifc.

The semantic definition of the cant transition segment is

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.,0.16,.BLOSSCURVE.);
```

The EnrichIfc4x3 reference implementation maps the semantic definition to the geometric representation as follows:

$D_s = \frac{0.0+0.0}{2} = 0\ m,\ D_e = \frac{0.0+0.16}{2} = 0.08\ m,\ \Delta D = 0.08 - 0.0 = 0.08\ m$

Quadratic Term:

$a_2 = 3\Delta D = 3(0.08\ m) = 0.24\ m$

$A_2 = \frac{(100\ m)}{\sqrt[3]{|0.24\ m|}}\left(\frac{0.24\ m}{|0.24\ m|}\right) = 160.917897\ m^{2/3}$

Cubic Term:

$a_3 = -2\Delta D = -0.016\ m$

$A_3 = \frac{100\ m}{\sqrt[4]{|-0.16\ m|}}\frac{-0.16\ m}{|-0.16\ m|} = -158.113883\ m^{3/4}$

Notice that the polynomial coefficents do not have whole length units (e.g. the units aren't m).

The mapped geometric representation is

```
#127 = IFCTHIRDORDERPOLYNOMIALSPIRAL(#128, -158.113883008419, 160.914897434272, $, $);
```

Recalling from Bloss Curve example above,

Quadratic Term

$A_2 = \frac{(100m)^{\frac{4}{3}}}{\sqrt[3]{|0.24m|}}\left(\frac{0.24m}{|0.24m|}\right) = 746.9007911\ m$

Cubic Term

$A_3 = \frac{(100\ m)^{\frac{5}{4}}}{\sqrt[4]{|-0.16\ m|}}\left(\frac{-0.16\ m}{|-0.16\ m|}\right) = -500\ m$

The polynomial coefficients have units of length as required by IFC.

The resulting geometric representation is

```
#127=IFCTHIRDORDERPOLYNOMIALSPIRAL(#98,-500.,746.900791092861,$,$);
```

Table 4.10-1 compares the third order polynomal terms.

Polynomial Term	**EnrichIfc4x3**	**This Guide**
Constant	0.0 (unitless)	$0\ m$
Linear	$0\ m^{1/2}$	$0\ m$
Quadratic	$160.91789\ m^{2/3}$	$746.90078\ m$
Cubic	$-158.113883\ m^{3/4}$	$-500\ m$

Table 4.10-1 — Comparison of polynomial coefficients

The EnrichIfc4x3 reference implementation computes the correct value for the deviating elevation. This is because there is a compensating error in the implementation. The deviating elevation is computed as

$D(s) = L\left(\frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2\right)$

Evaluated at $s = 50\ m$

$$D(50\ m) = (100\ m)\left(\frac{-158.113883\ m^{3/4}}{|(-158.113883\ m^{3/4})^5|}(100\ m)^3 + \frac{1}{(160.914897434272\ m^{2/3})^3}(100\ m)^2\right) = (100\ m)(-0.0002 + 0.0006) = 0.04\ m$$

The deviating elevation as computed in this guide is

$$D(s) = L^2\left(\frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2\right)$$

Notice that the first term is L^2, not L as in the reference implementation.

$$D(50\ m) = (100\ m)^2\left(\frac{-500\ m}{|(-500\ m)^5|}(100\ m)^3 + \frac{1}{(746.90079\ m)^3}(100\ m)^2\right) = (100\ m)^2(-0.000002\ m^{-1} + 0.000006\ m^{-1}) = 0.04\ m$$

Both approaches give the same result, however, a serious problem emerges if the EnrichIfc4x3 semantic to geometry mapping is mixed with the deviating elevation equation from this guide and visa-versa. Table 4.10-2 compares the results for all the combinations of the sematic to geometry mapping and deviation elevation computations for the Bloss curve evaluated at $s = 50\ m$. The deviating elevation can be incorrect proportional (or inversely proportional) to the length of the segment (100 or 1/100 in this example).

Mapping	$D(s)$ **Equation**	$D(50\ m)$
EnrichIfc4x3	EnrichIfc4x3	$0.04\ m$
This Guide	This Guide	$0.04\ m$
EnrichIfc4x3	This Guide	$4.0\ m$
This Guide	EnrichIfc4x3	$0.0004\ m$

Table 4.10-2 — Comparison of deviation elevation results

Section 5 - Offset Curves 5.0 Introduction

Offset curves are very common in infrastructure geometry. Some examples are:

- Edge of pavement
- Bridge girder centerlines
- Lane striping
- Utility centerline

`IfcOffsetCurveByDistances` defines the geometry of offset curves. It has three attributes:

- `BasisCurve` : The curve from which offsets are measured
- `OffsetValues` : Defines the offsets along the curve
- `Tag` (optional): An identifier for the curve, used to correlate offset curve points with positions in a variable cross-section (see Section 10)

A single offset value indicates a constant offset over the entire length of the curve.

If the offsets do not span the full length of the curve, the first and last offset are implicitly continued to the head and tail of the basis curve, respectively.

The `IfcOffsetCurveByDistances.OffsetValues` are of type `IfcPointByDistanceExpression`. `IfcOffsetCurveByDistances.BasisCurve` and `IfcPointByDistanceExpression.BasisCurve` must reference the same curve.

The `IfcOffsetCurveByDistances` geometry is derived by interpolating the `OffsetValues`.

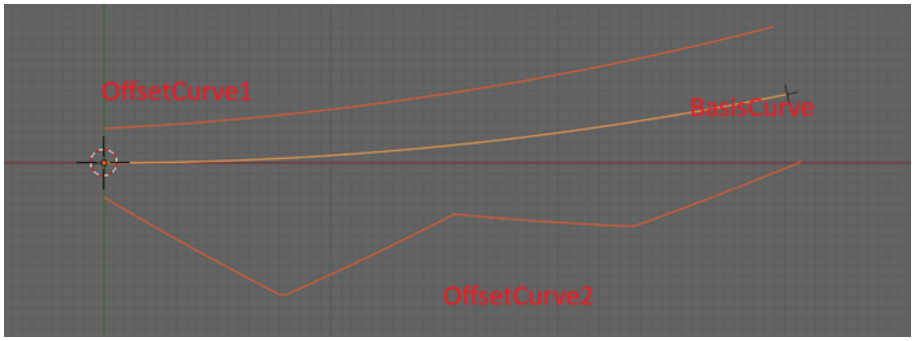


Figure 7.0-1 Offset curves

Figure 7.0-1 shows two offset curves that share a single basis curve. Offset curve 1 has different offset values defined at the start and the end of the basis curve. The start offset is 10 ft and the end offset is 20 ft. The basis curve is 100 ft long. The point on Offset curve 1 that corresponds to the mid-point of the basis curve is determined by evaluating the basis curve at 50 ft, then applying the linearly interpolated offset of 15 ft.

Offset curve 2 has several offsets. Linear interpolation is used to determine the offset from the basis curve to the offset curve between each pair of consecutive offset points.

Example model: IfcOffsetCurveByDistances.ifc

5.1 Offset Sign Convention

Positive offset values place the offset curve to the left of the basis curve, measured perpendicular to the curve's direction of travel at each chainage position. Negative values place it to the right. This matches the curvature sign convention used throughout this guide — positive means left, negative means right.

Offset values can change sign along the curve, meaning the offset curve can cross from one side of the basis curve to the other. Practical infrastructure uses for sign-crossing offsets are rare, but the representation allows it.

5.2 Offset Curves and IfcAlignment

In IFC4x3, `IfcOffsetCurveByDistances` is a resource-layer geometric entity and cannot exist independently in a model. Validation rule IFC105 requires that every resource entity be reachable from a rooted entity. In practice this means the offset curve must be assigned as the shape representation of an `IfcAlignment` (or another product), and that alignment must be aggregated into the project. Without this, the IFC validation service will report an IFC105 violation.

The consequence is that offset curves in IFC4x3 models are represented as alignments. This is natural: an offset alignment has its own object identity, can carry properties, can support linear placement of objects, and participates in the project's spatial breakdown just like the basis alignment.

The example file `IfcOffsetCurveByDistances.ifc` demonstrates this structure. The basis alignment "E-Line" and its two offsets, "Offset1" and "Offset2," are all `IfcAlignment` instances aggregated under the project. Each offset alignment's `Representation` references an `IfcShapeRepresentation` whose item is the corresponding `IfcOffsetCurveByDistances`.

5.3 Stationing on Offset Alignments

The IFC validation service warns when an `IfcAlignment` has no stationing referent — a `Pset_Stationing` property set attached via an `IfcReferent`. For many offset alignments this warning is valid and should be resolved: a ramp, a lane edge used for striping construction, or any offset alignment along which objects are linearly placed benefits from defined stationing.

For offset alignments that are purely geometric — a drainage channel modeled for clash detection, a clearance envelope, a construction limit line — independent stationing may not be meaningful. In those cases the warning is acceptable to leave. An alternative is to assign a stationing referent that mirrors the basis alignment's stationing, so that chainage values on the offset alignment correspond directly to chainage on the basis. This is often the least confusing option when the two alignments are used together in quantity takeoff or reporting.

5.4 Accuracy of Offset Curves

`IfcOffsetCurveByDistances` is an exact representation for two curve types and an approximation for everything else.

Lines and circular arcs. The geometric parallel of a line is another line; the geometric parallel of a circular arc of radius R at offset d is another circular arc of radius $R + d$ (left offset) or $R - d$ (right offset). For alignments composed entirely of tangents and circular arcs, a constant `IfcOffsetCurveByDistances` with a single `IfcPointByDistanceExpression` reproduces the true parallel exactly.

Transition curves. The six transition curve types — Clothoid, Bloss, Cosine, Sine, Helmert, and VienneseBend — do not have closed-form geometric parallels. The true parallel of a Clothoid spiral, for example, is not a Clothoid. `IfcOffsetCurveByDistances` handles this by defining the offset through linearly interpolated lateral distances rather than a parallel curve formula. The resulting geometry is a piecewise-linear approximation of the true offset. Accuracy improves by adding more `IfcPointByDistanceExpression` values at shorter chainage intervals, densifying the approximation.

For most infrastructure applications the linear interpolation error is small relative to construction tolerances. For applications requiring tighter accuracy — such as rail superelevation ramps or precision survey control — the approximation should be evaluated against the required tolerance and the offset point spacing adjusted accordingly.

5.4.1 Approximate Distance Along and Its Effect on IfcLinearPlacement

A related but distinct accuracy issue arises when `IfcOffsetCurveByDistances` is used as the `BasisCurve` in `IfcPointByDistanceExpression` for `IfcLinearPlacement`. The problem is not just that the shape of the offset curve is approximate — it is that **distance along** the offset curve is also approximate, and two independent implementations may compute different arc lengths for the same curve definition.

The arc length of `IfcOffsetCurveByDistances` at any given chainage is obtained by numerically integrating the offset curve geometry between the sample points. Because the IFC specification does not prescribe the interpolation method (see §6.7), different software may produce different intermediate curve shapes from identical `OffsetValues`, and therefore different arc lengths. Even if the interpolation

method is agreed upon, the arc length integral has no closed form for a general offset from a spiral, so implementations must use numerical quadrature with finite precision.

The consequence is that a `DistanceAlong` value specified against an `IfcOffsetCurveByDistances` does not map to a unique, reproducible point across implementations. This is in direct contrast to `DistanceAlong` measured along an `IfcCompositeCurve` or `IfcGradientCurve`, where arc length is determined exactly by the curve equations.

For precise linear placement, the recommended practice (discussed fully in §6.5) is to reference the **parent alignment** as the `BasisCurve` in `IfcPointByDistanceExpression` and use `OffsetLateral` to account for the transverse distance — rather than placing objects directly along the offset curve. This avoids the arc-length approximation entirely and keeps placement reproducible across software implementations.

5.5 Arc Length of an Offset Curve

The arc length of an offset curve differs from the basis curve length. For a constant offset d from a circular arc of radius R and arc length L:

$$L_{\text{offset}} = L \cdot \frac{R + d}{R}$$

A positive (left) offset increases arc length; a negative (right) offset inside the curve decreases it. For a left offset of 10 ft from a 1000-ft radius arc of length 200 ft:

$$L_{\text{offset}} = 200 \cdot \frac{1010}{1000} = 202 \text{ ft}$$

For varying offsets or non-circular basis curves there is no simple closed-form expression. `IfcOffsetCurveByDistances` does not expose an arc length attribute. When linear placement of objects relative to an offset alignment is needed, the recommended practice (see §6.5) is to use the parent alignment as the `BasisCurve` in `IfcPointByDistanceExpression` rather than the offset curve itself, so the arc length of the offset curve is rarely needed in practice.

5.6 Example Model Walkthrough

The file `IfcOffsetCurveByDistances.ifc` contains a basis alignment and two offset alignments using feet as the length unit. The relevant IFC excerpts are shown below.

Basis curve. The composite curve `#20` consists of a 200-foot circular arc (radius 1000 ft) starting at the origin heading east, followed by a zero-length line tangent. The arc subtends an angle of $200/1000 = 0.2$ radians, ending at approximately $(198.7, 19.9)$ ft. The basis alignment "E-Line" has stationing from 100+00 to 102+00.

Offset1 — left side, linearly varying:

```
#88=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(0.),10.,$,$,#20);
#89=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(200.),20.,$,$,#20);
#94=IFCOFFSETCURVEBYDISTANCES(#20,(#88,#89),$); /* Tag omitted */
```

The offset begins at +10 ft at chainage 0 and increases linearly to +20 ft at chainage 200. The midpoint of the basis curve (chainage 100) has an interpolated offset of +15 ft. This produces a left-side offset curve that diverges from the basis alignment.

Offset2 — right side, multi-point:

```
#102=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(0.),-10.,$,$,#20);
#103=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(50.),-40.,$,$,#20);
#104=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(100.),-20.,$,$,#20);
#105=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(150.),-30.,$,$,#20);
#106=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(200.),-20.,$,$,#20);
#111=IFCOFFSETCURVEBYDISTANCES(#20,(#102,#103,#104,#105,#106),$); /* Tag omitted */
```

The right-side offset widens quickly from −10 ft to −40 ft over the first 50 feet, then narrows to −20 ft at mid-curve, widens again to −30 ft, and finishes at −20 ft. The piecewise-linear profile of the offset distance is visible in the geometry of the resulting curve.

Both offset curves share the same basis curve `#20`. Each `IfcPointByDistanceExpression` also references `#20` directly, satisfying the requirement that the `BasisCurve` in `IfcOffsetCurveByDistances` and in its `OffsetValues` be the same curve object.

5.7 Interpolation of Offset Values

The IFC specification does not prescribe how offset values are to be interpolated between `IfcPointByDistanceExpression` entries. Several methods are possible:

- **Linear interpolation of offset values** — the lateral offset distance is interpolated linearly between consecutive offset points, and the interpolated distance is applied perpendicular to the basis curve at each chainage. This keeps the offset curve anchored to the basis curve geometry.
- **Linear interpolation between 3D points** — the 3D positions of consecutive offset points are computed and connected by straight line segments. This produces a true piecewise-linear curve in space, but one that drifts away from the basis curve between offset points.
- **Spline interpolation** — cubic spline, B-spline, Bézier, or NURBS methods applied to either the offset values or the 3D positions, producing smoother curves but with greater implementation complexity.

Different interpolation methods produce materially different geometry from the same `IfcOffsetCurveByDistances` data. Without a formal implementation agreement, two compliant applications may render the same model differently.

The recommended best practice is **linear interpolation of offset values**: interpolate the lateral distance at each chainage position, then apply that distance perpendicular to the basis curve. This method is the most natural given the structure of `IfcOffsetCurveByDistances`, produces predictable geometry, and is the least ambiguous for interchange.

5.8 The Tag Attribute

The `Tag` attribute of `IfcOffsetCurveByDistances` is an optional label that assigns an identifier to the offset curve. Its primary purpose is to correlate an offset curve with a named point in a variable cross-section definition.

`IfcSectionedSurface` and `IfcSectionedSolidHorizontal` define geometry by sweeping a series of cross-sections along a basis curve. Each cross-section can contain named control points — edge-of-pavement, top-of-rail, daylight line, and so on. The `Tag` on an `IfcOffsetCurveByDistances` matches one of those named points, allowing the surface or solid geometry to be driven by the offset curve's lateral positions rather than by fixed cross-section ordinates. This is the mechanism by which variable-width surfaces and solids are defined in IFC4x3.

The full treatment of `IfcSectionedSurface` and `IfcSectionedSolidHorizontal`, including how `Tag` values are resolved against cross-section geometry, is covered in Section 10.

Section 6 — Approximate Alignment Geometry 6.0 Introduction

The parametric `IfcCurveSegment`-based representations described in Sections 2 through 4 provide closed-form equations for position, tangent, and curvature at any arc-length along the alignment. That precision is essential when geometry drives design, construction staking, linear placement, or quantity takeoff.

Not all alignment data arrives in this form. Field surveys of existing infrastructure may produce a series of measured points rather than fitted curve parameters. Early-stage planning alignments may exist only as a sketched centerline digitized from a map. GIS systems commonly store linear features as coordinate strings. IFC accommodates these cases through two representation types: `IfcPolyline` and `IfcIndexedPolyCurve`.

Both are **discrete** rather than parametric: geometry is defined by a sequence of control points with linear interpolation between them. The result is a piecewise-linear approximation that encodes position only.

The IFC specification lists both types as valid `IfcAlignment` geometry for 2D horizontal alignments (planning and mapping contexts) and 3D alignments (survey-derived). See Section 1.5 for the complete list of valid representation types.

6.1 Geometric Forms

Both `IfcPolyline` and `IfcIndexedPolyCurve` provide simplified, approximate representations of alignment geometry. Rather than encoding the mathematical intent of the design — curve type, radius, spiral parameter — they encode only sampled positions. The two types differ in storage layout and in how much approximation they can absorb.

`IfcPolyline` holds geometry as an ordered list of `IfcCartesianPoint` instances — each vertex is a discrete entity in the model. Consecutive vertices are connected by straight line segments, so every curve in the original alignment is approximated by a series of chords. The curve accepts 2D or 3D coordinates, making it suitable for both plan-view sketches and full survey point strings.

`IfcIndexedPolyCurve` stores coordinates compactly in an `IfcCartesianPointList2D` or `IfcCartesianPointList3D` and references individual vertices by integer index. When the optional `Segments` attribute is omitted, the result is functionally identical to `IfcPolyline`. When `Segments` is provided, each entry is either an `IfcLineIndex` (two indices, straight segment) or an `IfcArcIndex` (three indices: start, a through-point on the arc, and end). Arc segments allow circular curves to be represented exactly rather than approximated by chords, reducing the number of vertices needed for a given accuracy. This makes `IfcIndexedPolyCurve` better suited to map-digitized or GIS-derived data where circular arcs are common but transition spirals are absent.

6.2 Alignments Without Semantic Layout

In contrast to Concept Templates 4.1.7.1.1.1 through 4.1.7.1.1.4, the IFC specification does not provide concept templates that show how `IfcPolyline` or `IfcIndexedPolyCurve` provide a geometric representation of an `IfcAlignment`. (The same gap applies to `IfcOffsetCurveByDistances`, covered in Section 5.)

Section 1.4 establishes that every `IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant` must terminate with a zero-length `IfcAlignmentSegment`. When a geometric representation accompanies the semantic definition, the corresponding last `IfcCurveSegment` must also be zero length.

Neither type is composed of `IfcCurveSegment` entities and therefore neither can satisfy the requirements for the geometric counterpart to a full alignment layout.

While `IfcPolyline` and `IfcIndexedPolyCurve` could each serve as an `IfcCurveSegment.ParentCurve`, they cannot produce the required zero-length closing segment. A zero-length polyline segment requires two consecutive coincident vertices; `IfcPolyline` WHERE rule WR1 explicitly prohibits this. `IfcIndexedPolyCurve` has no equivalent schema-level prohibition, but a coincident point pair produces a degenerate segment with an undefined tangent direction.

For this reason, `IfcPolyline` and `IfcIndexedPolyCurve` (as well as `IfcOffsetCurveByDistances`) can only represent an `IfcAlignment` without an accompanying semantic layout definition.

Section 7 — Alignments Reusing Horizontal Layout 7.0 Introduction

In infrastructure projects it is common for several alignments to share the same horizontal path. Three situations arise frequently:

Design alternatives. A corridor study fixes the horizontal alignment and evaluates multiple vertical profiles. Each profile becomes its own alignment sharing the common horizontal baseline. Comparison software can overlay the profiles without any coordinate transformation.

Existing versus proposed. When an existing road is to be reconstructed, the existing centerline may be surveyed and represented as one alignment; the proposed centerline shares the same horizontal but carries a new vertical profile.

Parallel infrastructure elements. Features that follow the same horizontal path at different elevations — top of sub-grade, finished grade, top of rail — can each be modelled as a separate alignment sharing a common horizontal. This is more precise than using `IfcOffsetCurveByDistances` for vertical offset, because each element carries its own parametric vertical definition rather than a sampled elevation difference.

IFC4x3 accommodates all three cases directly. Because horizontal layout is represented by discrete entities in the model, those entities can be referenced by more than one alignment. The horizontal curve is defined once and shared; each alignment then adds its own vertical and, if applicable, cant definition on top of that shared base.

The shared-horizontal pattern relies on a **parent** `IfcAlignment` that owns the common horizontal layout, and **child** `IfcAlignment` instances — each with its own vertical profile — aggregated under that parent. This parent/child structure is prescribed by Concept Template 4.1.4.4.1.2 *Alignment Layout — Reusing Horizontal Layout*.

This section describes the alignment layout structure for the reusing-horizontal-layout pattern, the geometric representations of the child alignments, and the open implementation questions that remain to be resolved in the specification.

7.1 Alignment Layout and Geometric Representation 7.1.1 Alignment Layout — Parent/Child Structure

The alignment layout uses a three-tier hierarchy:

1. A **parent** `IfcAlignment` nests a single shared `IfcAlignmentHorizontal` via `IfcRelNests`.
2. One or more **child** `IfcAlignment` instances are aggregated under the parent via `IfcRelAggregates`.
3. Each child `IfcAlignment` nests its own `IfcAlignmentVertical` (and, for rail alignments, its own `IfcAlignmentCant`) via `IfcRelNests`.

The shared `IfcAlignmentHorizontal` exists exactly once in the model, owned by the parent via `IfcRelNests`. No child alignment nests an `IfcAlignmentHorizontal` of its own; they reuse the horizontal definition from the parent. The parent itself carries no vertical layout; its sole role is to hold the shared horizontal and aggregate the children. Children that require cant also nest an `IfcAlignmentCant` via `IfcRelNests`. Figure 7.1.1-1 shows the complete parent/child layout structure.

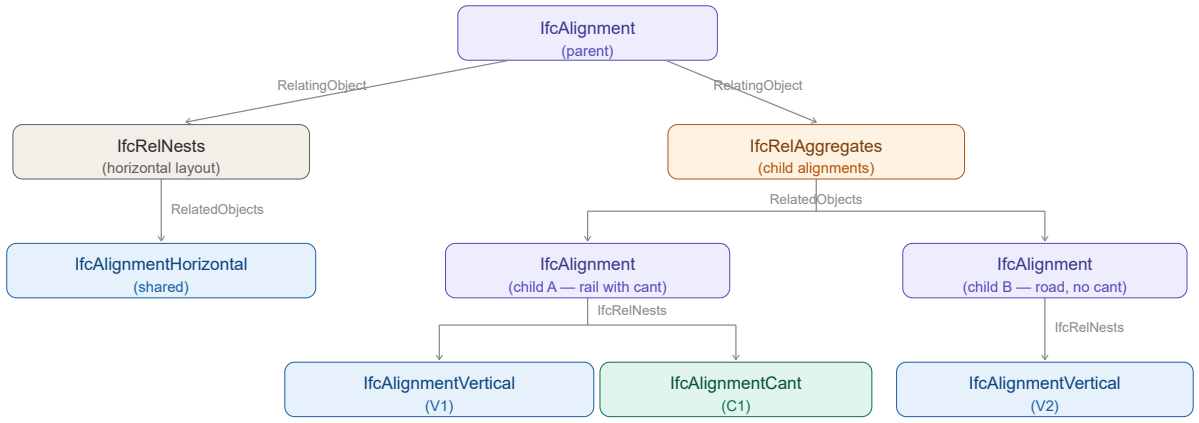


Figure 7.1.1-1 — Alignment layout for the reusing-horizontal-layout pattern.

Only the `IfcAlignmentHorizontal` is shared. Each child alignment owns its `IfcAlignmentVertical` and `IfcAlignmentCant` exclusively — those layout entities are never shared between children.

At the geometric level, `IfcSegmentedReferenceCurve` adds cant to a gradient curve by taking an `IfcGradientCurve` as its own `BaseCurve`. Each child alignment that needs a cant definition gets its own `IfcSegmentedReferenceCurve` and its own `IfcGradientCurve`; only the `IfcCompositeCurve` at the base of the chain is the shared instance, as shown in Figure 7.1.1-2:

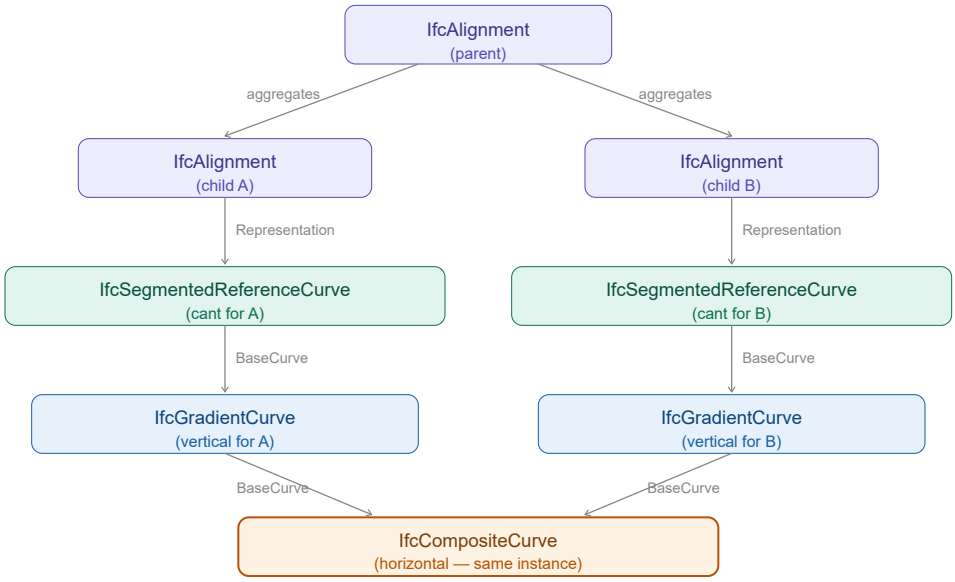


Figure 7.1.1-2 — Geometric chain for child alignments with cant. Each child alignment has its own `IfcSegmentedReferenceCurve` and `IfcGradientCurve`; the `IfcCompositeCurve` is the shared instance.

7.1.2 Geometric Representation — Shared BaseCurve

Each child alignment's 3D curve references the horizontal `IfcCompositeCurve` through its `BaseCurve` attribute. `IfcGradientCurve` carries `BaseCurve: IfcBoundedCurve` as the 2D horizontal curve, and two `IfcGradientCurve` instances can reference the same `BaseCurve` entity. The horizontal geometry — typically an `IfcCompositeCurve` composed of `IfcCurveSegment` instances — exists in the model exactly once and is referenced by both gradient curves. Figure 7.1.1-2 illustrates this shared reference.

Shape Representation Placement

Pending resolution of the open questions in §7.2, the following placement convention is recommended based on implementation experience:

- The `IfcAlignment` entity carries the `IfcCompositeCurve` as its own `IfcShapeRepresentation` with `RepresentationIdentifier` = `"FootPrint"` and `RepresentationType` = `"Curve2D"`.
- Each child `IfcAlignment` carries its full 3D curve as its own `IfcShapeRepresentation` with `RepresentationIdentifier` = `"Axis"` and `RepresentationType` = `"Curve3D"`.

  This recommendation consistently relates the geometric representation to an instance of `IfcAlignment`.

7.2 Open Implementation Questions

The IFC4x3 specification does not yet provide a concept template for the geometric representation when reusing horizontal layout. The following questions are not resolved and remain active topics for a future normative concept template:

- **Representation ownership.** Should all representations be contained within the parent `IfcAlignment.Representation.Representations`, within each child alignment's `Representation.Representations`, or split between them?
- **Horizontal curve ownership.** Should the `IfcCompositeCurve` representation of the horizontal layout be a representation on the parent `IfcAlignment` or on the `IfcAlignmentHorizontal`?
- **All-or-nothing geometry.** Is the presence of a geometric representation for one alignment a requirement for all alignments in the group to have geometric representations?
- **Mixed representation types.** Are mixed representation types valid within the same group — for example, one child alignment represented as an `IfcGradientCurve` and another as an `IfcOffsetCurveByDistances`?

Until these questions are resolved normatively, implementations should follow the convention described in §7.1.2 and document which representation placement strategy has been adopted.

7.3 Concept Template Transitions in Live Model Management

An application that maintains an IFC model incrementally — adding and removing vertical layouts as the designer works — must handle transitions between two different concept templates:

Vertical count	Required concept template
1	CT 4.1.4.4.1.1 — Alignment Layout – Horizontal, Vertical and Cant
2 or more	CT 4.1.4.4.1.2 — Alignment Layout – Reusing Horizontal Layout

These two templates have structurally different models. CT 4.1.4.4.1.1 nests both `IfcAlignmentHorizontal` and `IfcAlignmentVertical` directly under a single `IfcAlignment` via `IfcRelNests`. CT 4.1.4.4.1.2 uses the parent/child structure described in §7.1.1: the parent nests only the `IfcAlignmentHorizontal`, and each vertical lives in its own child `IfcAlignment` aggregated under the parent. Switching between them is not additive — it requires restructuring the existing model.

7.3.1 Transitioning to Reusing Horizontal Layout

Figure 7.3.1-1 shows the CT 4.1.4.4.1.1 structure used when a single vertical is present:

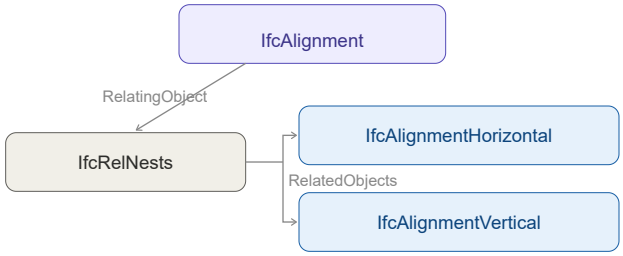


Figure 7.3.1-1 — CT 4.1.4.4.1.1: single `IfcAlignmentVertical` nested directly under `IfcAlignment`.

When a second vertical is added, the model must be restructured to CT 4.1.4.4.1.2. The `IfcAlignmentVertical (V1)` that was directly nested under the original `IfcAlignment` must be moved into a new child alignment. A second child alignment is created for V2, yielding the structure shown in Figure 7.3.1-2:

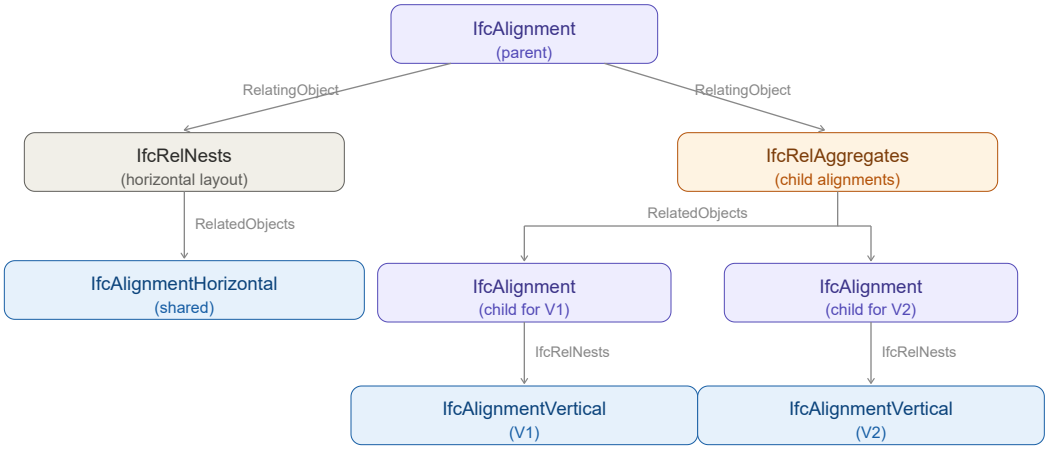


Figure 7.3.1-2 — CT 4.1.4.4.1.2: vertical layouts moved into child alignments aggregated under the parent.

The inverse transition is equally important but easier to overlook. When the designer removes a vertical layout and only one remains, the model must revert back to CT 4.1.4.4.1.1. The sole remaining `IfcAlignmentVertical` is re-nested directly under the alignment, the child `IfcAlignment` instances are dissolved, and the `IfcRelAggregates` relationship is removed — restoring the structure shown in Figure 7.3.1-1.

7.4 Relationship to Offset Curves

Section 5 describes `IfcOffsetCurveByDistances`, which defines a new curve laterally displaced from a basis alignment. Offset curves address a different need: they shift the curve horizontally (in plan) rather than reusing the same horizontal path at a different elevation. A left edge-of-shoulder offset and a right edge-of-shoulder offset use `IfcOffsetCurveByDistances` because their horizontal positions differ from the centerline. Features that share the exact same horizontal trace but differ only in elevation use the shared-horizontal-layout pattern described in this section.

The two patterns can coexist. For example, a road model might define:

- The centerline as a parent `IfcAlignment` with a full horizontal definition, plus a child carrying the design vertical profile
- The left and right edges of pavement as offset curves (lateral displacement, Section 5)
- The sub-grade profile as another child `IfcAlignment` under the same parent, sharing the centerline's horizontal but carrying an independent vertical profile (this section)

7.5 Use Cases Not Supported by Alignment - Reusing Horizontal Layout Concept Template

The following configurations arise in practice but cannot be fully represented using Concept Template 4.1.4.4.1.2 as currently defined. The IFC specification does not provide other concept templates that support these use cases.

Double-track railway. In this configuration, sometimes called the **7-line geometry** in railway practice, a single central horizontal reference defines both tracks. Each track carries its own vertical profile and cant, while stationing is carried along the common central alignment. The reusing-horizontal-layout template accommodates independent verticals per child alignment, but it provides no mechanism to associate independent horizontal offsets (one track to each side of the reference) with the shared horizontal. The two tracks would require separate horizontal definitions even where they are geometrically parallel, which defeats the purpose of horizontal sharing.

Dual carriageway. A highway with a central reference alignment defining two independent carriageways faces the same problem: each carriageway has a distinct horizontal position offset from the reference, so the shared-horizontal-layout pattern does not apply. Modelling each carriageway as a child of a common parent would require the child alignments to carry their own `IfcAlignmentHorizontal`, which is not permitted under the concept template. IFC4x3 has no explicit concept template for dual-carriageway or multi-track configurations.

Section 8 - Linear Placement 8.0 Introduction

Most elements in the built environment for infrastructure works are located **relative to an alignment** rather than by absolute coordinates. A bridge pier, a drainage inlet, a light pole, a traffic sign — all are described in traditional engineering plans by where they fall along a road or railway centerline, how far they sit to the left or right of that centerline, and how high above (or below) a reference elevation they stand. This intuitive "station, offset, elevation" system has been the language of civil engineers for over a century.

`IfcLinearPlacement` is the IFC mechanism that formalizes this concept. It places an object relative to a *directrix* — typically an alignment curve — using a distance along the curve combined with optional lateral and vertical offsets.

This section covers:

- The concept of linear placement and how it maps from traditional engineering practice into IFC.
- The IFC class hierarchy: `IfcLinearPlacement`, `IfcAxis2PlacementLinear`, and `IfcPointByDistanceExpression`.
- How the local coordinate system is constructed at a placement point.
- Special cases: longitudinal offset for unreachable points, and placement along `IfcOffsetCurveByDistances`.
- ISO 19148 Linear Referencing and how its concepts relate to IFC linear placement.

8.1 Linear Placement in Practice 8.1.1 The Traditional Approach

Before examining IFC, it helps to recall how placement is expressed in conventional civil engineering plans. Two common cases illustrate the idea:

Case 1 — Bridge Pier (2D plan view). A bridge pier is located in plan by its station along the roadway alignment and its lateral offset from the centerline. A plan note might read "Pier 2 CL: Sta. 142+35.75, 12.50 ft Lt." The elevation of the pier top is read separately from a profile view. No absolute coordinates are needed; the alignment provides the reference frame.

Case 2 — Drainage Inlet (3D). A storm drain inlet is placed at Sta. 98+12.4, offset 18.0 ft right of centerline, with its top grate set at elevation 312.75 ft (NAVD88). Here all three spatial dimensions are given: station gives position along the alignment, lateral offset gives the transverse position, and the elevation fixes the vertical position independently of the alignment profile.

Both cases share the same structure: **a distance along a reference curve, plus offsets from it**. `IfcLinearPlacement` captures exactly this structure.

8.1.2 The IFC Object Graph

The IFC classes that implement linear placement form a short chain:

```
IfcProduct
  └─ ObjectPlacement: IfcLinearPlacement
       ├─ PlacementRelTo: (omitted or reference to alignment context)
       └─ RelativePlacement: IfcAxis2PlacementLinear
            ├─ Location: IfcPointByDistanceExpression
            │    ├─ DistanceAlong: IfcLengthMeasure  (or IfcExpressionBasedValue)
            │    └─ BasisCurve: IfcCurve  (typically the alignment or its composite curve)
            ├─ Axis: IfcDirection  (optional – "up" direction)
            └─ RefDirection: IfcDirection  (optional – "forward" direction)
```

The `PlacementRelTo` attribute of `IfcLinearPlacement` establishes the reference context. When it is **omitted**, the placement is measured from the start of the basis curve defined in `IfcPointByDistanceExpression`. This is the standard case when the basis curve is an alignment defined in the project coordinate system.

Figure 6.1.2-1 schematically represents the linear placement of a bridge pier and a drain inlet.

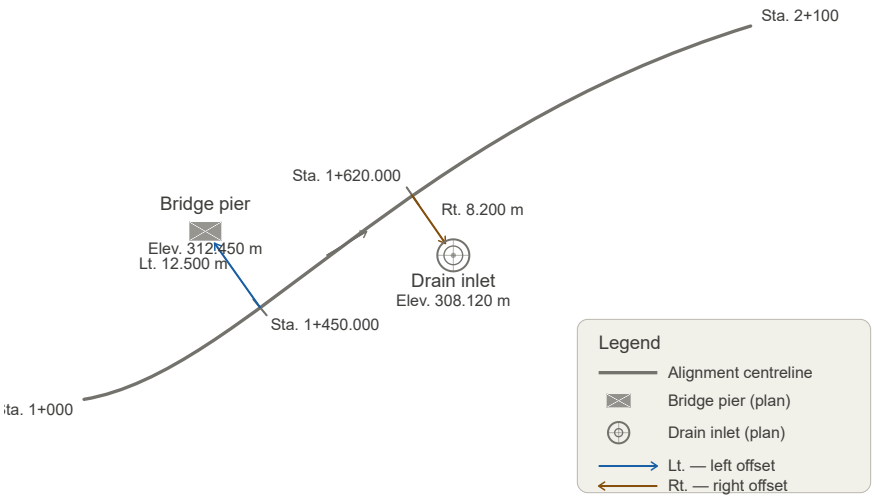


Figure 6.1.2-1 — Conceptual diagram showing a plan view of an alignment with a bridge pier placed at a station/offset and a drain inlet placed at station/offset/elevation.

8.2 IfcLinearPlacement and IfcAxis2PlacementLinear 8.2.1 Distance Along the Directrix

`IfcAxis2PlacementLinear.Location` is typed as `IfcPoint` but is constrained by a WHERE rule to be `IfcPointByDistanceExpression`. This class has two key attributes:

Attribute	Type	Description
`DistanceAlong`	`IfcCurveMeasureSelect`	The parametric distance measured along `BasisCurve`. Typically a plain `IfcLengthMeasure`.
`BasisCurve`	`IfcCurve`	The curve along which the distance is measured.
`OffsetLateral`	`OPTIONAL IfcLengthMeasure`	Signed lateral offset. Positive to the left of the curve's forward tangent; negative to the right (consistent with ISO 19148).
`OffsetVertical`	`OPTIONAL IfcLengthMeasure`	Signed vertical offset. Positive upward.
`OffsetLongitudinal`	`OPTIONAL IfcLengthMeasure`	Signed offset along the tangent direction. See §6.4 for uses.

The `BasisCurve` attribute of `IfcPointByDistanceExpression` is the key to understanding which curve the distance is measured along. For a full 3D alignment (`IfcGradientCurve` or `IfcSegmentedReferenceCurve`), the `DistanceAlong` is measured along the **horizontal projection** of the 3D curve — that is, along the underlying `IfcCompositeCurve` that represents the plan layout. This is consistent with how stationing is defined in transportation engineering: stationing is a horizontal measure.

8.2.2 Stationing and DistanceAlong

Stationing is addressed comprehensively in Section 9. For the purposes of linear placement, the key point is that `DistanceAlong` is a **geometric distance from the start of the basis curve**, not a station label. These two quantities are related but not identical:

- **Geometric distance** begins at zero and increases continuously to the total length of the curve.
- **Station value** may begin at an arbitrary value (e.g., 10+00.00 = 1000 ft from some project reference), may include equation gaps or overlaps where stationing is reset, and may use different units (feet vs. metres).

When using `IfcPointByDistanceExpression`, supply the **geometric distance**, not the raw station label. If the alignment has an `IfcReferent` that defines the starting station, the geometric distance equals the station value minus the starting station value (adjusted for any station equations encountered along the way).

8.3 The Placement Coordinate System 8.3.1 Role of Axis and RefDirection

`IfcAxis2PlacementLinear` defines a local right-handed coordinate system at the placement point. Two optional attributes control its orientation:

Attribute	Role in local CS	Default
`RefDirection`	X-axis (forward direction)	Tangent to the 3D curve at `Location`
`Axis`	Z-axis (up direction)	See §6.3.2

The Y-axis is derived as the cross product of `Axis` × `RefDirection` (after normalisation).

When `Axis` and `RefDirection` are both omitted, the implementation must supply defaults. This is the most common scenario and is described below.

8.3.2 Default Axis — An Open Issue

The default value of `Axis` is not unambiguously defined in the IFC schema documentation. A known open issue in the buildingSMART community tracks this ambiguity (see IFC4.x-IF Issue #125).

Two interpretations exist in practice:

1. **Axis = global Z = (0, 0, 1).** This is the most common implementation. It produces a coordinate system whose Z-axis is always vertical, regardless of the slope of the alignment. The X-axis (RefDirection) follows the horizontal tangent direction even when the curve has a vertical grade. This is natural for plan-oriented placement and matches traditional station-offset-elevation thinking.
2. **Axis = perpendicular to RefDirection in the plane containing RefDirection and global Z.** This produces a coordinate system whose Z-axis tilts with the grade of the alignment. The local X-axis is truly tangent to the 3D curve. This is more mathematically rigorous but less intuitive for typical civil engineering use.

Recommendation: Use interpretation 1 (Axis = global Z) unless the application specifically requires the tilted interpretation. When writing files, explicitly supply `Axis = (0, 0, 1)` to remove ambiguity.

8.3.3 Step-by-Step Construction of the Default Local CS

When `Axis` and `RefDirection` are not provided, the local coordinate system is constructed as follows. Let **T** be the unit tangent vector to the 3D alignment curve at the placement distance, and let **Z_g** = (0, 0, 1) be the global vertical.

1. **Set Z** = (0, 0, 1) *(the Axis direction)*
2. **Compute RefDirection** = unit tangent **T** to the 3D curve at `DistanceAlong`.
   For a horizontal curve (grade = 0), **T** lies in the XY plane.
   For a graded curve, **T** has a non-zero Z component.
3. **Y** = normalise(**Z** × **T**) *(points to the left of travel)*
4. **X** = normalise(**Y** × **Z**) *(points forward, projected onto the horizontal plane)*

The result is a coordinate system whose:

- X-axis points forward along the alignment (horizontal projection of the tangent),
- Y-axis points to the left (lateral direction in the horizontal plane),
- Z-axis points upward (global vertical).

This is the classic highway “station-offset” reference frame and is illusrated in Figure 6.3.3-1.

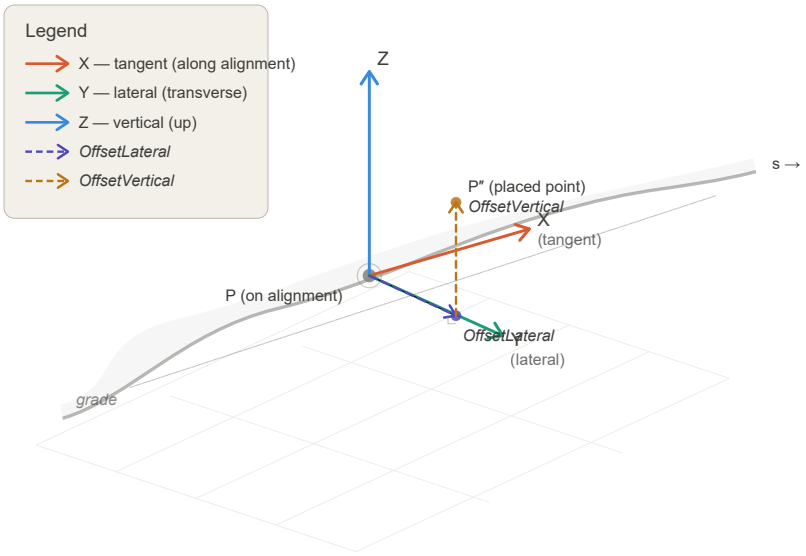


Figure 6.3.3-1 — Diagram showing the local coordinate system axes (X forward, Y left, Z up) at a point on a curved, graded alignment, with annotations for DistanceAlong, OffsetLateral, and OffsetVertical.

8.4 Longitudinal Offset and Unreachable Points 8.4.1 What Is an Unreachable Point?

In plane geometry, every point off a smooth curve can be reached by some combination of distance along the curve plus a perpendicular offset. However, certain geometric configurations in horizontal alignment create locations that **cannot be expressed** as a station plus a purely transverse offset.

The classic example is an **angle point** — the intersection of two tangents in a horizontal alignment where no curve has been inserted. At an angle point, the curve has a sharp corner. A point located “outside” the angle — beyond the apex — lies in a zone where the perpendicular from the curve never reaches. This is depicted in Figure 6.4.1-1.

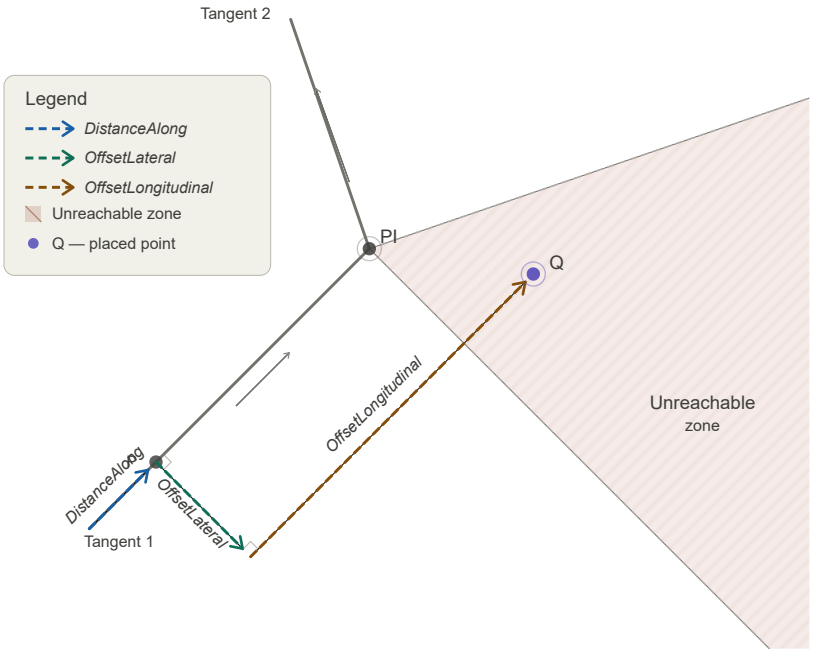


Figure 6.4.1-1 — Plan view showing two tangent lines meeting at an angle point (PI). The shaded region outside the angle cannot be reached by a station + lateral offset alone. An object in this region requires a longitudinal offset.

8.4.2 Using OffsetLongitudinal

`IfcPointByDistanceExpression.OffsetLongitudinal` provides the solution. A non-zero `OffsetLongitudinal` moves the placement point along the local X-axis (the forward tangent direction) after the perpendicular offset is applied. The procedure is:

1. Locate the point on the basis curve at `DistanceAlong`.
2. Apply `OffsetLateral` perpendicular to the curve tangent in the local XY plane.
3. Apply `OffsetVertical` in the local Z direction.
4. Apply `OffsetLongitudinal` along the local X direction (the tangent at step 1).

The resulting point is no longer "on" a perpendicular to the curve at `DistanceAlong`, but it is precisely located in 3D space.

Practical note: `OffsetLongitudinal` should be used only when necessary. For all ordinary station-offset placements, it should be omitted.

8.5 Linear Placement along IfcOffsetCurveByDistances

`IfcOffsetCurveByDistances` is an interpolated curve defined by a series of offset values measured from a basis curve. The offset values at intermediate positions are linearly interpolated between the defined sample points, forming a piecewise-linear offset profile. This is used, for example, to define a road edge line whose lateral distance from the centreline varies gradually. Offset curves are comprehensively discussed in Section 5.0.

8.5.1 The Approximate Length Problem

Because `IfcOffsetCurveByDistances` is a sampled, interpolated curve rather than an analytically defined curve, its **arc length is only approximate**. The arc length depends on the density of the sample points along the curve: more sample points produce a more accurate length estimate, but the length is never exact for a truly curved basis.

This approximation has a critical consequence for linear placement: when `IfcPointByDistanceExpression.BasisCurve` is an `IfcOffsetCurveByDistances`, the `DistanceAlong` value cannot be mapped to a unique, precisely determined point on the curve. Two implementations with different sampling densities may compute slightly different positions for the same `DistanceAlong` value.

8.5.2 Recommendations

For applications requiring precise linear placement:

1. **Prefer using the parent alignment** (e.g., the `IfcCompositeCurve` representing the horizontal alignment) as the `BasisCurve`, and use `OffsetLateral` to account for any transverse offset from the centreline. This avoids the approximation problem entirely.
2. **If placement along an offset curve is unavoidable**, document the sampling density of the `IfcOffsetCurveByDistances` so that receivers can evaluate the precision of derived positions.
3. **Do not rely on** `DistanceAlong` values along an `IfcOffsetCurveByDistances` being reproducible across different software implementations.

8.6 ISO 19148 Linear Referencing 8.6.1 Background

ISO 19148 *Geographic information — Linear referencing* is the international standard that formalises the concept of locating features along a linear element. IFC4x3's infrastructure extensions draw on ISO 19148 concepts, and `Pset_LinearReferencingMethod` (applicable to `IfcAlignment` and `IfcReferent`) is defined in terms of ISO 19148.

Understanding the ISO 19148 model helps implementers correctly interpret `DistanceAlong` values, especially when data is exchanged between systems that use different linear referencing conventions.

8.6.2 Key ISO 19148 Concepts

Linear Referencing Method (LRM). An LRM defines the rules for measuring distance along a linear element. The most common types are:

LRM Type	Description	Highway Example
Absolute	Distance measured continuously from a fixed start point.	Route mileage from a state border.
Relative	Distance measured from a nominated referent (not the start).	"0.4 km past interchange 12."
Interpolated Position	Location expressed as a fraction between two known referents.	Between mileposts 43 and 44.

`Pset_LinearReferencingMethod` records the LRM type (`LRMType`), its name (`LRMName`), and the units of measure (`LRMUnit`) for an alignment or referent element.

Referents and Milestones vs. Stationing. In European road practice, distance along a route is often expressed using *kilometre posts* (KP) or *reference posts* — physical markers at known locations. A KP system is a Relative or Reference Post LRM: distance is measured to the nearest upstream post, plus an offset. In North American highway practice, *stationing* is an Absolute LRM: every point on the alignment is assigned a cumulative distance from the project start, expressed as `ccc+dd.dd` (hundreds of feet) or in metres.

The key difference:

- **Stationing (Absolute LRM):** `DistanceAlong` in IFC closely matches the station value (after accounting for any starting station offset defined by an `IfcReferent`).
- **KP / Reference Post (Relative LRM):** `DistanceAlong` in IFC is always the absolute geometric distance from the curve start. A KP value must be converted to a geometric distance before use in `IfcPointByDistanceExpression`.

8.6.3 LRM Name Examples from ISO 19148 Annex C

ISO 19148 Annex C lists recognised LRM name aliases. Common examples include:

Name	Common Alias	Typical Region
milepoint	milepost, MP	USA, Canada
kilometre point	KP, PK	Europe, Latin America
chainage	ch	UK, Australia, India
reference post	RP	Rail, some roads

`Pset_LinearReferencingMethod.LRMName` should use one of these recognised names where possible for maximum interoperability.

8.6.4 Impact on DistanceAlong

Regardless of the LRM in use for labelling purposes, `IfcPointByDistanceExpression.DistanceAlong` is always the **geometric distance from the start of** `BasisCurve`. LRM labels (station values, KP values, etc.) are a display convention managed through `IfcReferent` and `Pset_LinearReferencingMethod`, not through `DistanceAlong` directly.

See Section 9 (Referents and Stationing) for a detailed treatment of how station labels are stored and how to convert between station labels and geometric distances.

8.7 Complete Example

The following example illustrates a point located at distance 1435.75 m along an alignment, offset 5.25 m to the right of centreline.

```
#100 = IFCALIGNMENT(...);
#101 = IFCALIGNMENTHORIZONTAL(...);
#102 = IFCCOMPOSITECURVE(...);     /* horizontal geometry */

/* Point on alignment at distance 1435.75 m, 5.25 m right (negative lateral offset) */
#200 = IFCPOINTBYDISTANCEEXPRESSION(1435.75, $, -5.25, $, #102);

/* Axis2Placement using default Axis and RefDirection */
#201 = IFCAXIS2PLACEMENTLINEAR(#200, $, $);

/* LinearPlacement — PlacementRelTo omitted → relative to curve start */
#202 = IFCLINEARPLACEMENT($, #201);
```

In this example:

- `DistanceAlong = 1435.75` is the geometric distance from the start of `#102`.
- `OffsetLateral = -5.25` places the point 5.25 m to the right of the horizontal alignment (negative = right of travel).
- `OffsetVertical` is omitted; the is placed in the same plane as the horizontal alignment.
- `Axis` and `RefDirection` are omitted; the default CS is constructed as described in §6.3.3.

8.8 Summary and Implementation Checklist

#	Item	Notes
1	Use `IfcLinearPlacement` for all infrastructure elements located relative to an alignment.	Prefer this over `IfcLocalPlacement` with absolute coordinates for alignment-relative objects.
2	Supply `DistanceAlong` as the geometric arc length from the start of `BasisCurve`.	Convert station labels to geometric distances first; use `IfcReferent` to record station label metadata.
3	Use signed `OffsetLateral`: positive = left, negative = right.	Consistent with ISO 19148 convention.
4	Set `Axis = (0,0,1)` explicitly to remove ambiguity.	Do not rely on the default; the default is not unambiguously defined in the IFC schema.
5	Use `OffsetLongitudinal` only for geometrically unreachable points (e.g., outside an angle point).	For all ordinary placements, omit or set to zero.
6	Avoid using `IfcOffsetCurveByDistances` as `BasisCurve` for precise placement.	Use the parent alignment with `OffsetLateral` instead.
7	Record the LRM type and units in `Pset_LinearReferencingMethod` on the `IfcAlignment`.	Required for correct interpretation of `IfcReferent` stationing labels.

Section 9 - IfcReferent and Stationing 9.0 Introduction

Section 6 established that `IfcPointByDistanceExpression.DistanceAlong` is a **geometric distance** — a raw arc-length measure from the start of a curve. But civil engineering practice does not communicate location this way. Engineers say "Pier 3 is at Station 14+235.75," not "Pier 3 is 14,235.75 metres from the start of the survey baseline." The station label is a human-readable, project-specific identifier that may begin at an arbitrary value, may use different units than the project, and may not even progress continuously along the alignment.

`IfcReferent` is the IFC mechanism that bridges these two worlds. It is a positioned object that attaches semantic location information — most importantly, stationing — to a specific geometric point on an alignment. It does not change the geometry; it annotates it.

This section covers:

- What `IfcReferent` is and the range of uses it serves beyond stationing.
- How stationing is expressed using `Pset_Stationing`, and how station values relate to geometric distances.
- Station equations: gap and overlap breaks, their physical causes, and how they affect `DistanceAlong` conversions.
- How referents are nested to alignments using `IfcRelNests`, including the recommended approach for mixed nesting with alignment layouts.
- How `IfcReferent` informs the semantic position of other objects via `IfcRelPositions`, without affecting their geometry.

9.1 What Is a Referent?

`IfcReferent` is a subtype of `IfcPositioningElement`, which is itself a subtype of `IfcProduct`. Like any product, it has an `ObjectPlacement` (typically `IfcLinearPlacement`) that locates it geometrically on an alignment, and it can carry property sets that provide additional semantic information at that location.

The single attribute `IfcReferent` adds beyond its supertypes is `PredefinedType` (`IfcReferentTypeEnum`), which guides receivers on how to interpret the referent and which property sets are relevant:

- **Stationing / chainage** (`STATION`). The most common use. Marks a position with a human-readable station label, defines the alignment's starting station, records a station equation, or identifies a key geometry point such as PC, PT, or PVI.
- **Linear referencing events** (`SUPERELEVATIONEVENT`, `WIDTHEVENT`). Locations where a design parameter changes — superelevation transitions, width changes, and similar cross-section events.
- **Reference markers and mileposts** (`REFERENCEMARKER`, `KILOPOINT`, `MILEPOINT`). Physical objects in the right-of-way that are part of a real-world linear referencing system.
- **Administrative boundaries** (`BOUNDARY`). Locations where a jurisdiction, maintenance zone, or asset ownership boundary crosses the alignment.
- **Intersection locations** (`INTERSECTION`). Points where another road or route crosses the alignment.
- **General positioned location** (`POSITION`). A fully described linearly referenced location combining alignment, LRM, and measure value.

9.2 Stationing: Concept and Practice 9.2.1 What Is a Station?

Stationing (also called *chainage* in British practice) is a distance-based coordinate system used to identify locations along a linear route. In North American highway practice, stations are expressed in the form `ccc+dd.dd`, where the number before the `+` is the count of hundreds of feet (or hundreds of metres in metric projects) and the digits after are the remainder. For example, Sta. 142+35.75 means 14,235.75 feet (or metres) from the project datum.

The project datum — the zero point of stationing — is defined by the first `IfcReferent` nested in the alignment (see §8.4). Stationing typically begins at a round number such as 10+00 or 100+00 rather than 0+00, to avoid negative stations at points that may be

surveyed upstream of the formal project start.

Stationing is a **display convention**, not a geometric quantity. The geometric position of a point is fully determined by `DistanceAlong` in `IfcPointByDistanceExpression`. The station label is metadata that identifies that point to engineers and field crews in human-readable terms.

9.2.2 Pset_Stationing

Stationing metadata is stored in `Pset_Stationing`, attached to an `IfcReferent` via `IfcRelDefinesByProperties`. The property set has three properties:

Property	Type	Description
`Station`	`IfcLengthMeasure`	The station value at this referent's location. For the first referent, this defines the starting station of the alignment.
`IncomingStation`	`IfcLengthMeasure`	Present only at a station equation. The station value on the *incoming* (upstream) side of the break. The `Station` property holds the *outgoing* value immediately after the break.
`HasIncreasingStation`	`IfcBoolean`	Controls the direction of stationing. If `TRUE` (or absent), subsequently nested referents have increasing station values. If `FALSE`, they decrease. Covers reverse-stationing schemes.

The `Station` value is expressed in the same units as the project length unit (typically metres or feet), **not** in the `ccc+dd.dd` string form. A station of 142+35.75 ft is stored as `14235.75` with feet as the project unit.

9.3 Station Equations: Gaps and Overlaps 9.3.1 Why Station Equations Exist

A station equation (also called a *chainage break* or *stationing equation*) is a discontinuity in the stationing system at which the station value jumps. Equations arise in several common situations:

Realignment. When a road is realigned — a curve is straightened or a new bypass is built — the new alignment geometry is a different length than the old. Rather than re-station the entire project downstream of the change (which would invalidate all existing plan sheets, permits, and as-built records), engineers introduce a station equation at the point where the old and new alignments reconnect. The geometry changes; the stationing on the downstream portion is preserved.

Survey accumulation. Long routes are often surveyed in segments. Small differences in measurement between survey crews produce a gap or overlap at the junction point. Rather than re-survey everything, an equation is introduced.

Existing route adoption. When a new project ties into an existing road with its own stationing, an equation reconciles the two stationing systems at the junction.

9.3.2 Gap Equations

A **gap equation** (also called a *station ahead* or *forward equation*) occurs when stationing jumps forward. The alignment geometry is continuous, but the station number increases by more than the physical length at the equation point.

Example: The alignment arrives at a point with incoming station 142+35.75. Due to a realignment that shortened the route, the outgoing station is 145+00.00. There is a gap of 264.25 ft. A distance of 14,235.75 ft of geometry corresponds to a label of 145+00, not 142+35.75.

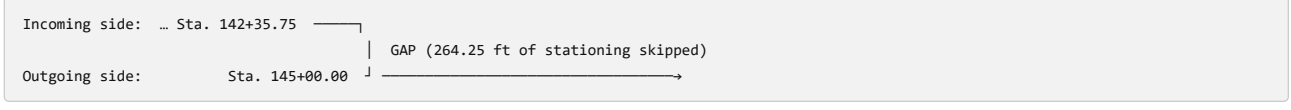


Figure 8.3.2-1 — Station gap equation. Geometry is continuous; station numbers jump forward.

9.3.3 Overlap Equations

An **overlap equation** (also called a *station back* or *backward equation*) occurs when stationing jumps backward. The alignment geometry is continuous, but the station number decreases at the equation point.

Example: Incoming station 78+42.10, outgoing station 76+00.00. There is an overlap of 242.10 ft. Two different geometric points — one upstream and one downstream of the equation — carry station labels between 76+00 and 78+42.

```
Incoming side:  Sta. 78+42.10  ──────┐
                                     │  OVERLAP (242.10 ft of station range used twice)
Outgoing side:  Sta. 76+00.00  ──────┘──────────────────────────────────────→
```

Figure 8.3.3-1 — Station overlap equation. Geometry is continuous; station numbers jump backward.

9.3.4 Effect on DistanceAlong Conversion

The presence of station equations means that **you cannot convert a station label to `DistanceAlong` by simple subtraction of the starting station**. You must account for every equation encountered between the alignment start and the point of interest.

The conversion algorithm is:

1. Begin at the alignment start. Set `accumulated_distance = 0`, `current_station = starting_station`.
2. Process each `IfcReferent` in nesting order. At each referent with a `Pset_Stationing`:

- Compute the geometric distance from the previous referent to this one (from their `DistanceAlong` values).
- Add that geometric distance to `accumulated_distance`.
- If the referent has only `Station` (no `IncomingStation`): update `current_station = Station`. No equation.
- If the referent has both `IncomingStation` and `Station`: this is an equation. Verify that `current_station + elapsed_distance ≈ IncomingStation`. Then reset `current_station = Station` (the outgoing value).

1. For a target station label `S`, find the segment where `S` falls between consecutive referents (using the appropriate incoming/outgoing station values at equations), then compute:

$$\text{DistanceAlong} = \text{referent\_distance} + \frac{S - \text{segment\_start\_station}}{\text{segment\_station\_span}} \times \text{segment\_geometry\_length}$$

For simple alignments with no equations, this reduces to the familiar $\text{DistanceAlong} = S - \text{starting\_station}$.

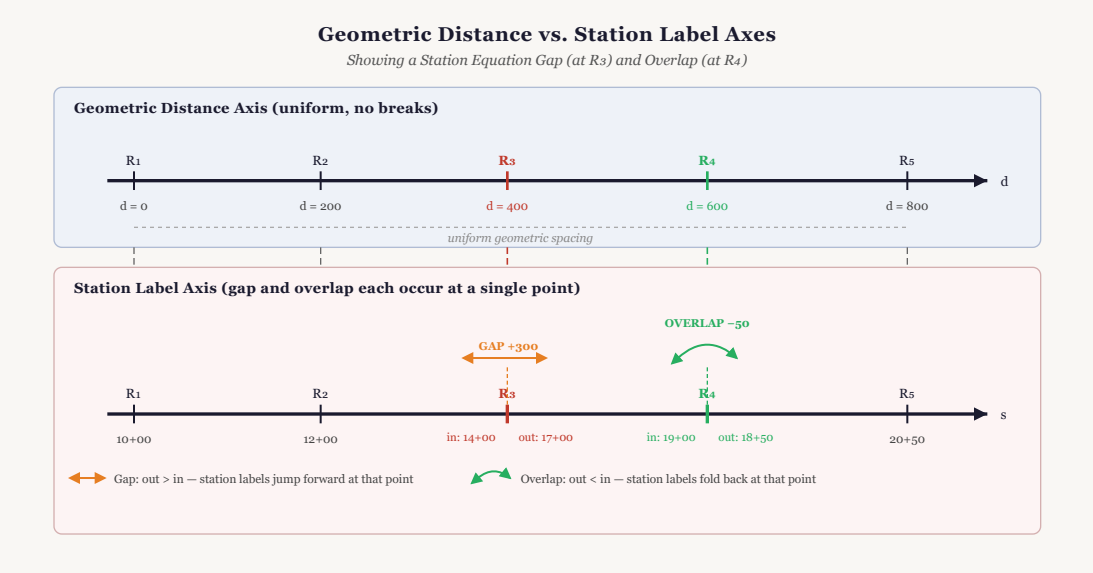


Figure 8.3.4-1 — Diagram showing a timeline of DistanceAlong (geometric) vs. station label for an alignment with one gap equation and one overlap equation. Annotate the start station, equation points, and the non-linear relationship between station and distance.

9.3.5 Example: Starting Station and One Equation

The following pseudo-STEP example shows an alignment with:

- Starting station = 10+00.00 (1000.00 ft)
- A gap equation at geometric distance 5432.10: incoming Sta. 154+32.10, outgoing Sta. 160+00.00 (gap = 567.90 ft)

```
/* Alignment */
#10 = IFCALIGNMENT(guid, $, 'Route 15 Mainline', ...);

/* === LAYOUT NEST === */
#11 = IFCALIGNMENTHORIZONTAL(guid, ...);
#12 = IFCRELNESTS(guid, $, $, $, #10, (#11));

/* Horizontal geometry composite curve */
#20 = IFCCOMPOSITECURVE(...);

/* === REFERENT NEST === */

/* Ref 1: Starting station = Sta. 10+00.00 = 1000.00 ft from project datum */
#30 = IFCPOINTBYDISTANCEEXPRESSION(0.0, $, $, $, #20);
#31 = IFCAXIS2PLACEMENTLINEAR(#30, $, $);
#32 = IFCLINEARPLACEMENT($, #31);
#33 = IFCREFERENT(guid, $, 'Start Station', $, $, #32, $, .STATION.);
#34 = IFCPROPERTYSINGLEVALUE('Station', $, IFCLENGTHMEASURE(1000.00), $);
#35 = IFCPROPERTYSET(guid, $, 'Pset_Stationing', $, (#34));
#36 = IFCRELDEFINESBYPROPERTIES(guid, $, $, $, (#33), #35);

/* Ref 2: Gap equation at geometric distance 5432.10 */
/*        IncomingStation = Sta. 154+32.10 = 15432.10 ft */
/*        Station (outgoing) = Sta. 160+00.00 = 16000.00 ft */
#50 = IFCPOINTBYDISTANCEEXPRESSION(5432.10, $, $, $, #20);
#51 = IFCAXIS2PLACEMENTLINEAR(#50, $, $);
#52 = IFCLINEARPLACEMENT($, #51);
#53 = IFCREFERENT(guid, $, 'Sta. Eq.', $, $, #52, $, .STATION.);
#54 = IFCPROPERTYSINGLEVALUE('IncomingStation', $, IFCLENGTHMEASURE(15432.10), $);
#55 = IFCPROPERTYSINGLEVALUE('Station', $, IFCLENGTHMEASURE(16000.00), $);
#56 = IFCPROPERTYSET(guid, $, 'Pset_Stationing', $, (#54, #55));
#57 = IFCRELDEFINESBYPROPERTIES(guid, $, $, $, (#53), #56);

/* IfcRelNests for referents */
#60 = IFCRELNESTS(guid, $, $, $, #10, (#33, #53));
```

Conversion check: To find the `DistanceAlong` for a feature at Sta. 162+45.00 (16245.00 ft):

1. Start: geometric distance 0 = Sta. 1000.00.
2. Equation referent at geometric distance 5432.10: incoming 15432.10, outgoing 16000.00 (gap of 567.90).
3. Target Sta. 16245.00 > 16000.00 (outgoing), so it falls after the equation.
4. `DistanceAlong = 5432.10 + (16245.00 - 16000.00) = 5432.10 + 245.00 = 5677.10`

Without accounting for the gap, a naive subtraction would give `16245.00 - 1000.00 = 15245.00` — an error of 432.10 ft.

9.4 Nesting Referents to Alignments 9.4.1 IfcRelNests

`IfcReferent` instances are connected to their parent `IfcAlignment` through `IfcRelNests`. This relationship has two important properties:

1. **`RelatingObject`** — the `IfcAlignment` (the parent).
2. **`RelatedObjects`** — an **ordered list** of objects nested within. The order matters: referents must appear in the list in order of increasing `DistanceAlong` along the alignment.

The first `IfcReferent` in `RelatedObjects` defines the **starting station** of the alignment. Its `Pset_Stationing.Station` value is the station label at `DistanceAlong = 0.0` of the alignment curve.

9.4.2 The Mixed-Nesting Problem

`IfcRelNests` is also used to attach alignment layout components (`IfcAlignmentHorizontal` , `IfcAlignmentVertical` , `IfcAlignmentCant`) to `IfcAlignment` . Both layout sub-objects and referents decompose the same parent through the same relationship type. This creates ambiguity: should layout sub-objects and referents share a single `IfcRelNests` instance, or should they use separate ones?

The IFC specification does not resolve this clearly. Two patterns are seen in practice:

Pattern A — Shared nest. A single `IfcRelNests` instance contains both layout objects and referents interleaved or grouped. This works but can make parsing complex, because consumers must distinguish layout objects from referents by type.

Pattern B — Separate nests (recommended). Two distinct `IfcRelNests` instances are used: one whose `RelatedObjects` contains only the alignment layout sub-objects, and one whose `RelatedObjects` contains only `IfcReferent` instances.

Recommendation: Use Pattern B — separate nests. This is the pattern illustrated in the existing Figure 8.4.2-1 and is more robust for software implementations. The layout nest and the referent nest are independent, and each can be processed without concern for the content of the other.

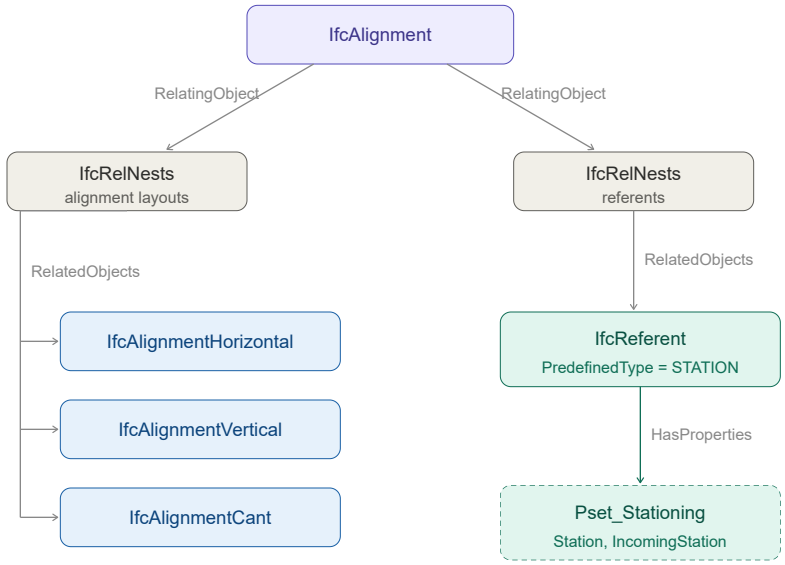


Figure 8.4.2-1 — Two `IfcRelNests` relationships: one for alignment layout sub-objects (horizontal, vertical, cant) and one for `IfcReferent` instances.

9.4.3 Referent Ordering Requirement

Referents in `IfcRelNests.RelatedObjects` must be ordered by their geometric position (increasing `DistanceAlong`). This ordering is not enforced by a WHERE rule in the schema but is stated as a requirement in the IFC concept template for 4.1.4.4 and 4.1.4.4.3 Object Nesting as applied to referents. Implementations that read referent data should not assume the list is sorted and should sort by `DistanceAlong` before processing; implementations that write referent data must produce a sorted list.

9.5 Semantic vs. Geometric Positioning 9.5.1 The Distinction

It is important to understand that `IfcReferent` serves two distinct roles that must not be conflated:

Geometric role. When an `IfcReferent` carries an `IfcLinearPlacement`, it has a precise geometric location on the alignment. This location participates in coordinate computations.

Semantic role. An `IfcReferent` can *annotate* the position of another object — not by moving it, but by providing a human-readable station label that names its location. This is purely informational metadata.

The geometric placement of an infrastructure element (a bridge pier, a sign, a drain inlet) is defined entirely by its own `IfcLinearPlacement` with an `IfcPointByDistanceExpression`. An associated referent gives that location a name. Removing or changing the referent does not move the object.

9.5.2 IfcRelPositions

The relationship between an `IfcReferent` and the object whose position it annotates is `IfcRelPositions`:

- `RelatingPositioningElement` — the `IfcReferent` (the name-giver).
- `RelatedProducts` — the set of `IfcProduct` instances (the positioned objects) whose station label is defined by this referent.

Example: A bridge pier (`IfcBridgePart.PIER`) is geometrically placed at `DistanceAlong = 14235.75`. An `IfcReferent` with `PredefinedType = STATION` and `Pset_Stationing.Station = 14235.75` (relative to the starting station) is connected to the pier via `IfcRelPositions`. The pier's station label is now Sta. 142+35.75. The pier's geometry is unchanged. This is shown in Figure 8.5.2-1.

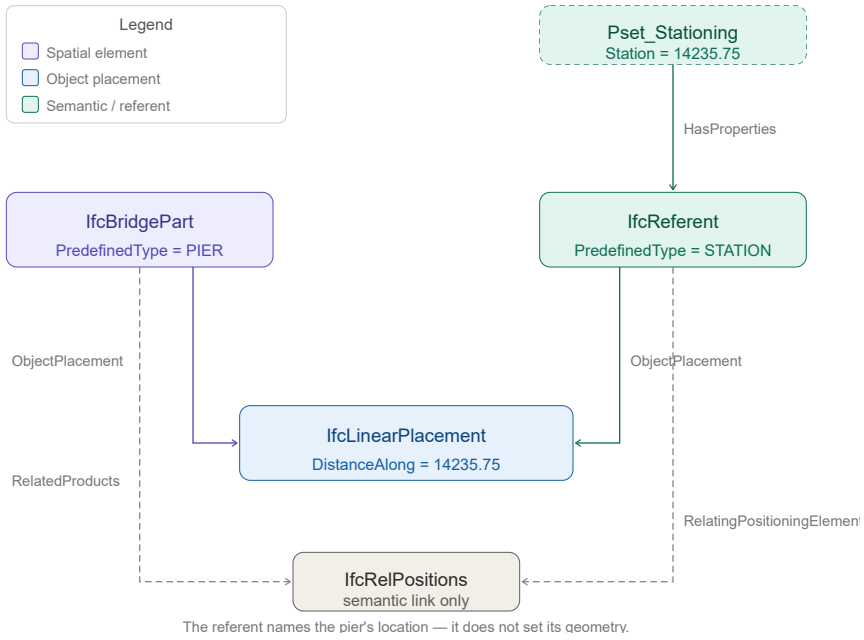


Figure 8.5.2-1 — Object graph showing an `IfcBridgePart.PIER` with its own `IfcLinearPlacement`, an `IfcReferent` (STATION type) with `Pset_Stationing`, and the `IfcRelPositions` link between them. Annotate that the referent provides the station label; it does not set the geometry.

9.6 Summary and Implementation Checklist

#	Item	Notes
1	Use `IfcReferent` to attach station labels and other semantic location information to alignment positions.	Referents are informational overlays on geometry, not geometry themselves.
2	Store the station value in `Pset_Stationing.Station` as a plain `IfcLengthMeasure` in project length units.	Do not store the `ccc+dd.dd` string — compute it from the numeric value when displaying.
3	For the first referent (starting station), set `Pset_Stationing.Station` to the station label at `DistanceAlong = 0`.	This is commonly a round number like 1000.00 (Sta. 10+00).
4	At a station equation, provide both `IncomingStation` and `Station` on the same referent.	`IncomingStation` = upstream label; `Station` = downstream label.
5	Do not convert station labels to `DistanceAlong` by simple subtraction. Iterate through all equation referents between the alignment start and the target.	See §8.3.4 for the full algorithm.
6	Use separate `IfcRelNests` instances for alignment layout sub-objects and referents.	Mixing them in a single nest is allowed but makes parsing harder.
7	Order referents in `IfcRelNests.RelatedObjects` by increasing `DistanceAlong`.	Required by the nesting concept template; sort defensively when reading.
8	Use `IfcRelPositions` to link a referent to the object(s) whose station it names.	This is semantic annotation only; it does not affect geometry.
9	Use named referents (PC, PT, PVC, PVI, etc.) for alignment key points, linked via `IfcRelPositions` to the corresponding `IfcAlignmentSegment`.	Enables station-based queries without re-deriving geometry.
10	Set `Pset_Stationing.HasIncreasingStation = FALSE` only for reverse-stationing schemes.	When absent or `TRUE`, stationing increases in the direction of the alignment.

todo

- Figure 10.3.1-1 either eliminate the second surface or explain it.
- add IFC example files for Figure 10.3-1 and 10.3.1-1
- create new example files to illustrate string lines
- create example that shows a section on a horizontal curve. With templates, they are conneced with straight lines. With stringlines, the edges of the surface follow the curve.

Section 10 - Sectioned Surfaces and Solids 10.0 Introduction

A common approach in roadway design software is template-based modeling: define a cross-section template, apply it at regular intervals along the alignment, and interpolate the geometry between positions. For simple, uniform roadways — constant width, constant cross-slope — this works well. But most roadways are not uniform. They taper in and out; include medians that appear and disappear; require superelevation transitions; and feature traffic islands, widenings, and complex intersection geometry. For these cases, interpolating between evenly-spaced template stamps produces piecewise-linear geometry that can only approximate the intended design.

Road design software has long addressed this limitation through the concept of *strings* — continuous three-dimensional threads tracing significant features along the route: the edge of travel path, the shoulder break, the top of curb, the back of curb. Rather than deriving edges from template interpolation, strings define the edges explicitly. The cross-sections through the design are understood as sections through the string model, not the primary definition.

IFC4x3 introduces two geometry entities specifically for infrastructure: `IfcSectionedSurface` and `IfcSectionedSolidHorizontal`. Both define geometry by sweeping cross-sections along a `Directrix` — typically an alignment curve. `IfcSectionedSurface` uses open cross-sections to produce a surface; `IfcSectionedSolidHorizontal` uses closed cross-sections to produce a volumetric solid. Both support two complementary approaches to defining the geometry: template-based (cross-sections at defined distances along the directrix, linearly interpolated between) and stringline-based (guide curves that control where tagged cross-section points travel between sections).

10.1 Template-Based Approach

In the template-based approach, cross-sections are defined at discrete distances along the `Directrix`. The geometry engine interpolates linearly between consecutive cross-section positions to generate the surface or solid. The cross-sections need not be identical — width, slope, and shape can all vary from one distance to the next — but the transition between any two consecutive sections is linear.

For simple geometry this is entirely adequate. A road with a constant typical section can be fully described by two cross-sections with linear interpolation between them. A superelevation transition with a uniform rotation rate can be captured with sections at each end of the transition. The template approach becomes strained when the geometry between defined sections is not linear — a curved edge taper through a widening, for example — requiring progressively denser section spacing to reduce the approximation error.

Profile Types

The cross-section profile type differs between the two entities.

`IfcSectionedSurface` uses `IfcOpenCrossProfileDef`, an infrastructure-specific profile type introduced in IFC4x3. It defines an open profile as a series of segments radiating from a starting point, each characterized by a width and a slope.

Attribute	Type	Description
`HorizontalWidths`	`IfcBoolean`	If true, `Widths` are measured horizontally; if false, along the slope
`Widths`	`LIST [1:?] OF IfcNonNegativeLengthMeasure`	Width of each segment
`Slopes`	`LIST [1:?] OF IfcPlaneAngleMeasure`	Slope of each segment
`Tags`	`OPTIONAL LIST [2:?] OF IfcLabel`	Label for each vertex; one more entry than the number of segments
`OffsetPoint`	`OPTIONAL IfcCartesianPoint`	Starting point; if absent, the profile originates at the alignment

Table 10.1-1 — IfcOpenCrossProfileDef attributes

The profile starts at `OffsetPoint` (or at the alignment if `OffsetPoint` is absent) and extends segment by segment. With N segments there are $N + 1$ vertices — the starting point plus the far endpoint of each segment. This is enforced by the formal proposition `CorrespondingTags`, which requires the `Tags` list to contain exactly one more entry than the `Slopes` list. A typical half-section for a two-lane road might have three segments: crown to edge of travel path, edge of travel path to shoulder break, and shoulder break to edge of roadway.

`IfcSectionedSolidHorizontal` uses closed profile types — typically `IfcArbitraryClosedProfileDef` with an `IfcIndexedPolyCurve` referencing an `IfcCartesianPointList2D`. The profile polygon is defined in two dimensions with points listed in counter-clockwise order when viewed from the direction the profile normal points. The `IfcCartesianPointList2D` entity includes a `TagList` attribute, introduced in IFC4x3, that assigns a label to each coordinate in the list and enables the stringline mechanism described in Section 10.2.

10.2 Stringlines

A stringline is a continuous three-dimensional curve tracing the path of a specific feature — edge of pavement, shoulder break, top of curb — along the full extent of the design. Where the template approach interpolates linearly between section endpoints, a stringline defines the exact trajectory of a tagged cross-section point, independent of section density.

In IFC, stringlines are represented as `IfcOffsetCurveByDistances` instances. The `Tag` attribute on `IfcOffsetCurveByDistances` — an optional `IfcLabel` — connects a guide curve to the cross-section points that carry the matching tag value. A tagged point in a cross-section profile is constrained to follow the guide curve carrying the same tag.

The tag mechanism differs between the two entities:

- For `IfcSectionedSurface`, tags are carried by `IfcOpenCrossProfileDef.Tags` — a list of $N + 1$ labels, one per vertex of the open profile. Each vertex can be independently associated with a guide curve.
- For `IfcSectionedSolidHorizontal`, tags are carried by `IfcCartesianPointList2D.TagList` — one label per coordinate in the point list. Any point in the closed profile can be tagged.

In both cases, the geometry of the surface or solid at a tagged point follows the corresponding guide curve rather than relying on linear interpolation between the authored section positions. A widening with a curved edge taper can be represented exactly with sections only at the start and end of the transition — the guide curve carries the taper geometry between them.

The stringline approach is not an alternative to the template approach but an augmentation of it. Cross-sections are always required; guide curves control interpolation between them. In the limiting case with infinitely dense sections, both approaches produce the same geometry. In practice, stringlines allow compact models with few sections and geometrically exact results where section-only interpolation would require dense section spacing to achieve acceptable approximation.

10.3 IfcSectionedSurface

`IfcSectionedSurface` defines a surface by sweeping open cross-sections along a directrix curve.

Attribute	Type	Description
`Directrix`	`IfcCurve`	The curve along which sections are swept
`CrossSectionPositions`	`LIST [2:?] OF IfcAxis2PlacementLinear`	Positions along the directrix at which sections are placed
`CrossSections`	`LIST [2:?] OF IfcProfileDef`	Open cross-section profiles, one per position

Table 10.3-1 — IfcSectionedSurface attributes

The surface is generated by sweeping the cross-sections between the defined `CrossSectionPositions`. It does not extend to the head or tail of the `Directrix` — the surface is bounded by its first and last cross-section positions. The `CrossSectionPositions` and `CrossSections` lists must be equal in length.

The profile orientation for `IfcSectionedSurface` is correctly described in the specification: the profile normal is derived from the associated `IfcAxis2PlacementLinear`. The corresponding attribute in `IfcSectionedSolidHorizontal` contains a documentation error discussed in Section 10.5. Figure 10.3-1 illustrates a sectioned surface through a superelevation transition, showing how the open cross-section profile varies along the directrix.

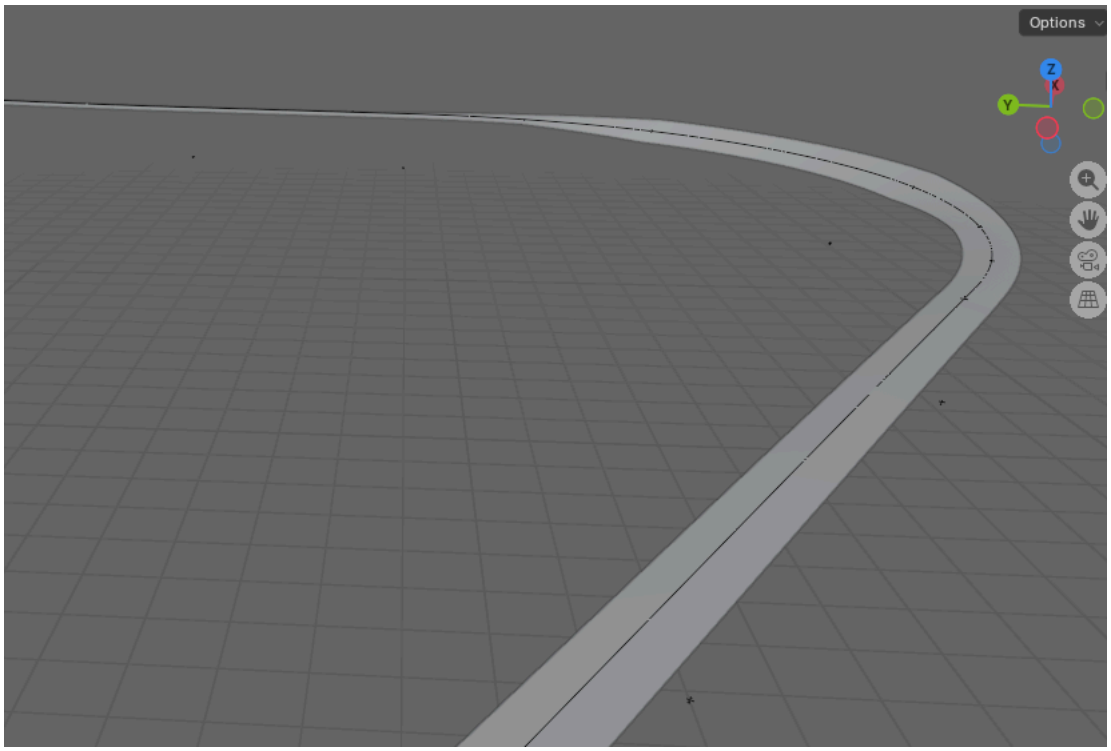


Figure 10.3-1 — Sectioned surface illustrating a superelevation transition

10.3.1 Breaklines

In the context of `IfcSectionedSurface`, a breakline is a line along the surface where the cross-slope changes abruptly — an edge of travel path, a shoulder break, the face of a curb. This usage is distinct from DTM breaklines, which force TIN triangulation to honor a feature edge. Here the term describes a topological feature of the swept surface itself: a ridge or valley line where adjacent surface facets meet at a non-tangent angle.

Breaklines arise from the tag mechanism when consecutive cross-sections have different numbers of segments. A widening lane introduces a new vertex at the distance along the directrix where the widening begins. Tags identify which vertices in one section correspond to vertices in the next, even when the two sections have different segment counts. A vertex present in one section but absent in the adjacent section causes the surface to initiate or terminate at that longitudinal position, which is the geometric definition of a breakline in this context. Figure 10.3.1-1 illustrates a sectioned surface with breaklines resulting from cross-section topology changes along the route.

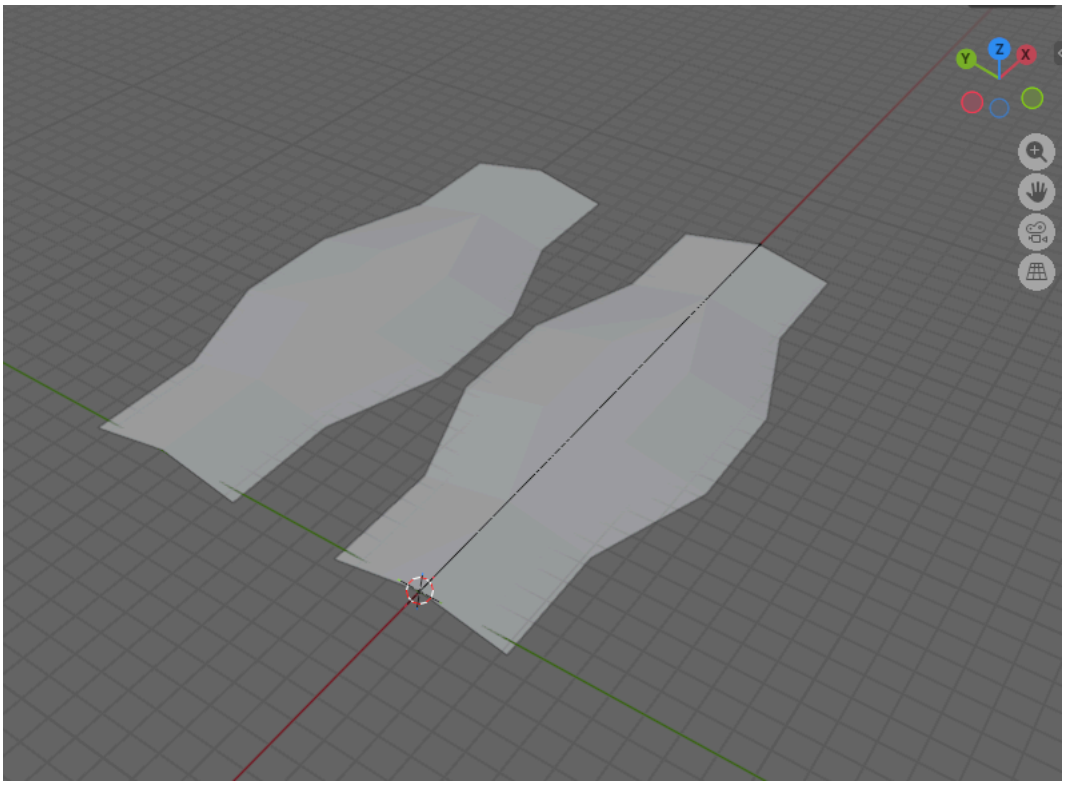


Figure 10.3.1-1 — Sectioned surface with breaklines at cross-section topology changes. Conceptually based on IFC Figure 8.8.3.37.B; unlike that figure, which shows only a widening, this figure shows the surface widening and returning to its original width.

Although the IFC specification does not state this explicitly, the `Width` of an `IfcOpenCrossProfileDef` segment must be zero at the distance along the directrix where a breakline initiates or terminates. When a new feature vertex first appears — the start of a widening lane, for example — the cross-section at that distance must include the new tagged vertex with a zero-width segment. This zero-width entry establishes the vertex at a defined position without introducing any lateral extent, allowing the geometry to grow from that point as subsequent sections carry non-zero widths. Without a zero-width segment at the breakpoint position, the surface has no defined starting position for the new feature and the geometry is ambiguous. IFC Figure 8.8.3.37.B — *Sectioned surface with branching longitudinal breaklines* — illustrates this pattern for a surface where breaklines branch from the main cross-section.

The tag mechanism serves both breaklines and stringlines simultaneously. A tag on a profile vertex identifies that vertex's correspondence across sections with different topology and, when a matching `IfcOffsetCurveByDistances` guide curve exists, controls the vertex's trajectory between sections. The two functions — topological correspondence and geometric guidance — share the same tag infrastructure.

10.4 IfcSectionedSolidHorizontal

`IfcSectionedSolidHorizontal` defines a solid by sweeping closed cross-sections along a directrix curve. It follows the same fundamental structure as `IfcSectionedSurface` but produces a volumetric solid bounded by the swept faces and the cross-section planes at each end.

Attribute	Type	Description
`Directrix`	`IfcCurve`	The curve along which sections are swept
`CrossSections`	`LIST [2:?] OF IfcProfileDef`	Closed cross-section profiles, one per position
`CrossSectionPositions`	`LIST [2:?] OF IfcAxis2PlacementLinear`	Positions along the directrix at which sections are placed

Table 10.4-1 — IfcSectionedSolidHorizontal attributes

As with `IfcSectionedSurface`, the solid is generated by sweeping only between the defined `CrossSectionPositions`. It does not extend to the head or tail of the `Directrix`. The `CrossSections` and `CrossSectionPositions` lists must be equal in length, and the position expressions must not use longitudinal offsets.

Cross-sections are typically `IfcArbitraryClosedProfileDef` profiles. The profile outline is defined as an `IfcIndexedPolyCurve` referencing an `IfcCartesianPointList2D`. Profile points are listed in counter-clockwise order when viewed from the direction the profile normal points. The `IfcCartesianPointList2D.TagList` attribute assigns a label to each point in the coordinate list, enabling the stringline mechanism of Section 10.2.

For a note on a documentation error in the profile orientation specification for this entity, see Section 10.5.

10.4.1 Rotations

Cross-section rotation accommodates superelevation — the banking of a road or rail cross-section along a curve. `IfcSectionedSolidHorizontal` supports two approaches.

Single superelevation applies a uniform rotation to the entire cross-section at each defined position along the directrix. `IfcDerivedProfileDef` expresses this rotation, with its `ParentProfile` referencing the base (unrotated) profile. Each entry in `CrossSections` can reference its own `IfcDerivedProfileDef` instance with a different rotation angle, allowing the rotation to vary along the directrix while the underlying profile shape remains constant.

Multiple independent superelevations apply when different parts of the cross-section rotate independently — for example, a divided highway where left and right carriageways have different cross-slopes, or a cross-section with a varying median shape. In this case each `CrossSection` is a distinct profile instance. The tagged points in `IfcCartesianPointList2D.TagList` and their corresponding `IfcOffsetCurveByDistances` guide curves control where each point travels independently along the route. The guide curves effectively encode the superelevation variation for each tagged feature point without requiring explicit rotation parameters.

10.5 Specification Gaps and Implementation Notes BasisCurve of Guide Curves Must Equal the Directrix

The specification does not require the `BasisCurve` of an `IfcOffsetCurveByDistances` guide curve to be the same curve as the `Directrix` of the surface or solid it serves. This constraint is nevertheless geometrically necessary. `CrossSectionPositions` are parameterized along the `Directrix` — each position is a distance along that curve. A guide curve must share the same distance parameterization for tag-matching interpolation to be well-defined. A guide curve relative to a different basis curve carries a different parameterization, making the correspondence between section distances and guide curve positions undefined.

The intended interpretation is that guide curves are always `IfcOffsetCurveByDistances` instances whose `BasisCurve` is the same curve as the surface or solid `Directrix`. This interpretation also provides a natural scoping mechanism for tag matching: in a model with multiple surfaces each having their own directrix, only guide curves sharing the same `BasisCurve` as a given surface's `Directrix` are candidates for that surface's tag matching. Global tag uniqueness across the entire model is therefore not required — only uniqueness within the set of guide curves sharing a common basis curve with the surface or solid.

Tag Uniqueness Within Scope

The IFC specification does not state that tags must be unique within the scope of a given surface or solid. Uniqueness is nevertheless a necessary precondition for the tag-matching mechanism to function. If three `IfcOffsetCurveByDistances` guide curves all carry `Tag = "A"` and three cross-section vertices all carry `Tag = "A"`, there is no defined rule for which guide curve governs which vertex. Implementers authoring models with guide curves should treat tag uniqueness as a required constraint within each surface or solid. Validators currently have no machine-verifiable rule to enforce this, which is a gap in the formal rules.

Extent Along the Directrix

`IfcSectionedSurface` and `IfcSectionedSolidHorizontal` are bounded by their `CrossSectionPositions` — geometry is generated by sweeping only between the first and last defined position and does not extend to the head or tail of the `Directrix`. Figure 8.8.3.35.C in the IFC specification incorrectly depicts the solid extending to the head and tail of the `Directrix` when `CrossSectionPositions` are at interior distances, contradicting the specification text. The practical consequence of such extension would be severe: multiple

`IfcSectionedSolidHorizontal` instances modeling sequential prisms along the same alignment — pavement layers, cut sections, fill sections — would each extend to the full alignment endpoints and overlap one another. Implementers should follow the specification text, not the figure.

Guide curve extent follows the opposite rule. The `IfcOffsetCurveByDistances` specification states that if the defined offset values do not span the full extent of the basis curve, the offsets implicitly continue with the same value toward the head and tail. For guide curves this means a constant offset stringline — such as the edge of a uniform-width shoulder — can be expressed with a single `IfcPointByDistanceExpression` without requiring offset values at every cross-section position. The two behaviors are independent: guide curves extend implicitly because they control interpolation *within* the surface or solid's defined span, not because they extend that span.

Disagreement Between Section Endpoint and Guide Curve

When a cross-section endpoint is authored at a specific position and a guide curve with a matching tag passes through a different position at that same distance along the directrix, the specification provides no guidance on which governs. Implementers should treat the authored section positions as exact and design guide curves to be consistent with them at all defined section positions. The behavior of a geometry kernel when a section endpoint and its guide curve disagree is implementation-defined.

Profile Orientation Documentation Error

The specification for `IfcSectionedSolidHorizontal` states:

> The profile X axis is the direction of `RefDirection` from `IfcAxis2PlacementLinear`, and the profile Y axis is the direction of `Axis`.

This is incorrect. The profile is defined in a 2D XY plane and has a normal — the axis perpendicular to that plane — which determines how the profile is oriented in 3D space when swept along the directrix. It is that normal, not the profile X axis, that is the direction of `RefDirection`. When `RefDirection` is omitted from `IfcAxis2PlacementLinear` it defaults to the curve tangent direction. If the profile X axis were to align with the tangent, the profile plane would lie parallel to the sweep direction rather than perpendicular to it, producing degenerate geometry. The correct behavior is that the profile normal aligns with `RefDirection` (or the curve tangent when `RefDirection` is absent), placing the profile face perpendicular to the sweep path.

The specification for `IfcSectionedSurface` states this correctly: "the profile normal is derived from the associated `IfcAxis2PlacementLinear`." The `IfcSectionedSolidHorizontal` documentation should be read consistently with the `IfcSectionedSurface` text. This error is documented in buildingSMART issues IFC4.x-IF #147 and IFC4.x-development #1010.

Validation Service: IfcOffsetCurveByDistances Must Be Referenced by a Rooted Entity

The buildingSMART validation service enforces rule IFC105, which requires all resource entities to be referenced by a rooted entity. `IfcOffsetCurveByDistances` is a resource entity. Although guide curves are geometrically referenced by a surface or solid, which in turn is referenced by a rooted entity, the validation service does not trace this indirect path and flags guide curves not directly associated with a rooted entity.

In practice, each `IfcOffsetCurveByDistances` guide curve must be directly associated with an `IfcAlignment`. If the model does not already contain an alignment whose geometry includes the guide curve naturally, a placeholder alignment is required. Each such placeholder alignment must also have stationing defined, or additional validation warnings will be raised.

todo:

- Review this section - it is completely generated

Section 11 - LandXML to IFC Conversion 11.0 Introduction

LandXML 1.x is the dominant legacy format for civil alignment data; IFC 4x3 is the modern target. The goal of this chapter is to guide implementers through the non-obvious decisions and pitfalls that arise during conversion, drawn from practical experience.

11.1 Schema Conceptual Comparison

The following table provides a high-level mapping of the two data models.

LandXML	IFC 4x3
`<Alignments>` / `<Alignment>`	`IfcAlignment` nested under `IfcProject` via `IfcRelAggregates`
`<CoordGeom>`	`IfcAlignmentHorizontal` nested via `IfcRelNests`
`<Profile>` / `<ProfAlign>`	`IfcAlignmentVertical` nested via `IfcRelNests`
`<Cant>`	`IfcAlignmentCant` nested via `IfcRelNests`
`<StaEquation>`	`IfcReferent` with stationing attributes
`<Units>`	`IfcUnitAssignment` in `IfcProject`

Table 10.2-1 — LandXML to IFC schema conceptual comparison

The key conceptual difference between the two schemas is that LandXML describes geometry procedurally — PI-based for vertical alignment, point-to-point for horizontal — whereas IFC describes segments declaratively as typed business objects with a start point, radii, and length.

11.2 Units

LandXML carries explicit unit declarations (`<Metric>` , `<Imperial>`) for linear and angular measure. IFC requires all geometry in SI base units (metres, radians) or with an explicit `IfcConversionBasedUnit` .

Read `<Units>` before processing any geometry — all coordinate values depend on it.

Linear units. Convert foot and US Survey foot to metres as needed. Note that the US Survey foot and the international foot are not the same:

$$1 \text{ US Survey foot} = \frac{1200}{3937} \text{ m} \approx 0.3048006 \text{ m}$$

$$1 \text{ international foot} = 0.3048 \text{ m (exact)}$$

Using the wrong conversion factor introduces a systematic error that compounds over long alignments.

Angular units. LandXML directions can be expressed in decimal degrees, grads, or radians.
`IfcAlignmentHorizontalSegment.StartDirection` is always in radians. Apply the appropriate conversion before writing IFC output.

Bearing convention. LandXML directions are azimuths measured clockwise from North. IFC plane angles are measured counter-clockwise from the positive X-axis (East). The conversion is

$$\theta_{IFC} = \frac{\pi}{2} - \theta_{bearing}$$

where $\theta_{bearing}$ is in radians.

11.3 Horizontal Alignment 11.3.1 LandXML Horizontal Element Types

The LandXML `<CoordGeom>` element contains a sequence of `<Line>`, `<Curve>`, and `<Spiral>` child elements. Each maps to an `IfcAlignmentSegment` nested under `IfcAlignmentHorizontal`.

11.3.2 Direction from Geometry

LandXML lines and spirals often omit explicit direction attributes (`Dir`, `DirStart`) and rely on the geometry — start/end or start/PI points — to imply direction. `IfcAlignmentHorizontalSegment.StartDirection` is required. When the direction attribute is absent, compute it from the available geometry:

$$\theta = \arctan 2(\Delta y, \Delta x)$$

then apply the bearing-to-IFC plane angle conversion from Section 11.3.

11.3.3 Circular Curve (`<Curve>` → `CIRCULARARC`)

`<Curve>` provides center, start, end, and radius. IFC requires the start point, the start direction (the tangent direction, not the radial direction), the signed radius, and the arc length.

Compute the tangent direction from the radial line by rotating 90°:

$$\theta_{tangent} = \arctan 2(y_{start} - y_{center},\ x_{start} - x_{center}) + \text{sign} \cdot \frac{\pi}{2}$$

where $\text{sign} = +1$ for counter-clockwise (`rot="ccw"`) and $\text{sign} = -1$ for clockwise (`rot="cw"`). The signed radius passed to `IfcAlignmentHorizontalSegment` follows the same sign convention.

11.3.4 Spiral Type Mapping

The following table maps LandXML `spiType` values to `IfcAlignmentHorizontalSegmentTypeEnum`.

LandXML `spiType`	**IFC Type**	**Notes**
`clothoid`	`CLOTHOID`	Direct mapping
`bloss`	`BLOSSCURVE`	Direct mapping
`biquadratic`	`HELMERTCURVE`	Direct mapping
`biquadraticParabola`	`HELMERTCURVE`	Equivalent to biquadratic
`cubic`	`CUBIC`	Direct mapping
`cubicParabola`	`CUBIC`	Equivalent to cubic
`cosine`	`COSINECURVE`	Direct mapping
`sinusoid`	`SINECURVE`	Direct mapping
`sineHalfWave`	`COSINECURVE`	Approximation — loss of fidelity, see note below
`japaneseCubic`	`VIENNESEBEND`	Direct mapping
`revBiquadratic`	—	No IFC equivalent, log warning and skip
`revBloss`	—	No IFC equivalent, log warning and skip
`revCosine`	—	No IFC equivalent, log warning and skip
`revSinusoid`	—	No IFC equivalent, log warning and skip
`radioid`	—	No IFC equivalent, log warning and skip
`weinerBogen`	—	No IFC equivalent, log warning and skip

Table 10.4.4-1 — LandXML spiType to IFC type mapping

> **Note on `sineHalfWave`.** The LandXML specification describes the sine half-wavelength spiral as an approximation of a cosine spiral. Mapping it to `COSINECURVE` is therefore reasonable but not exact. Implementations should log a warning when this substitution is made.

11.3.5 Radius Sign Convention

LandXML uses `rot="cw"` / `rot="ccw"` to indicate curve direction. IFC encodes direction in the sign of `StartRadius` and `EndRadius`: a positive value indicates a left turn (counter-clockwise) and a negative value indicates a right turn (clockwise).

$$\text{sign} = \begin{cases} -1 & \text{if } rot = \texttt{cw} \\ +1 & \text{if } rot = \texttt{ccw} \end{cases}$$

Apply this sign to both `StartRadius` and `EndRadius`.

11.3.6 Infinite Radius

LandXML represents an infinite start or end radius by omitting the attribute or by supplying `DBL_MAX`. IFC uses `0.0` in `IfcAlignmentHorizontalSegment.StartRadius` or `EndRadius` to indicate an infinite radius. Substitute `0.0` whenever the LandXML value is absent or exceeds a practical threshold.

11.3.7 Unhandled Horizontal Element Types

`<Chain>` and `<IrregularLine>` elements have no direct equivalent in the IFC alignment semantic model. Log a warning when these elements are encountered and skip them. If approximate geometric fidelity is acceptable, a `<Chain>` (polyline) could be decomposed into a sequence of `LINE` segments.

11.4 Vertical Alignment 11.3.1 LandXML Vertical Data Model

LandXML represents vertical geometry as a sequence of PVI-based elements under `<ProfAlign>`: `<PVI>`, `<ParaCurve>`, `<UnsymParaCurve>`, and `<CircCurve>`. This is fundamentally different from IFC's segment-based model, which requires an explicit start station, start elevation, start grade, end grade, and length for each segment.

11.3.2 PVI-Based to Segment-Based Conversion

The core translation challenge is that LandXML locates vertical curves by their PVI station and curve length, while IFC needs each segment's start station explicitly. The conversion procedure is:

1. Compute the entry grade from the PVI station and the previous PVI or curve endpoint.
2. The start station of a symmetric parabolic curve is $\text{PVI station} - L/2$.
3. The start elevation is computed by projecting back from the PVI along the entry grade: $e_{start} = e_{PVI} - g_1 \cdot L/2$.
4. Gradient segments fill any gap between the end of one curve and the start of the next.

11.3.3 Symmetric Parabolic Arc (`<ParaCurve>` → `PARABOLICARC`)

This is the standard vertical curve case. Given PVI station s_{PVI}, PVI elevation e_{PVI}, curve length L, entry grade g_1, and exit grade g_2:

$$s_{start} = s_{PVI} - \frac{L}{2}$$

$$e_{start} = e_{PVI} - g_1 \frac{L}{2}$$

Map to `IfcAlignmentVerticalSegment` with `PredefinedType = PARABOLICARC`, `StartDistAlong = s_{start} - s_{alignment\_start}`, `StartHeight = e_{start}`, `StartGradient = g_1`, `EndGradient = g_2`, `HorizontalLength = L`.

11.3.4 Asymmetric Parabolic Arc (`<UnsymParaCurve>` → Two `PARABOLICARC` Segments)

IFC has no asymmetric vertical curve type. The `<UnsymParaCurve>` must be split into two abutting `PARABOLICARC` segments. Given entry length L_{in}, exit length L_{out}, entry grade g_1, and exit grade g_2:

Compute the vertical offset from the composite curve to the PVI:

$$h = \frac{L_{in} \cdot L_{out} \cdot (g_2 - g_1)}{2(L_{in} + L_{out})}$$

The transition point between the two sub-curves occurs at the PVI station with elevation:

$$e_{transition} = e_{PVI} + h$$

The intermediate grade at the transition point is:

$$g_{mid} = g_1 + \frac{2h}{L_{in}}$$

Create two `PARABOLICARC` segments: the first from the curve start to the PVI station with grades g_1 and g_{mid}, and the second from the PVI station to the curve end with grades g_{mid} and g_2.

11.3.5 Circular Vertical Curve (`<CircCurve>` → `CIRCULARARC`)

LandXML provides the radius and arc length. IFC `IfcAlignmentVerticalSegment` for `CIRCULARARC` accepts the radius directly in the `RadiusOfCurvature` attribute. Compute the start station and elevation the same way as for a symmetric parabolic arc, using $L/2$ as the half-length. Note that this is an approximation — the true tangent length of a circular vertical curve is $T = R \tan(\Delta/2)$ where $\Delta = L/R$, which differs from $L/2$ except for small deflection angles.

11.3.6 Start Station Offset

LandXML stations are absolute. `IfcAlignmentVerticalSegment.StartDistAlong` is measured from the start of the alignment. Subtract the alignment start station from every LandXML station:

$$\text{StartDistAlong} = s_{LandXML} - s_{alignment_start}$$

The alignment start station is found in `<Alignment staStart="...">` and must be read before processing any profile elements.

11.5 Cant 11.3.1 LandXML Cant Model

LandXML `<Cant>` uses `<CantStation>` points and `<CantSegment>` elements with left and right cant values at the start and end of each segment, along with a curve type designation.

11.3.2 Mapping to `IfcAlignmentCant`

Each `<CantSegment>` maps to an `IfcAlignmentCantSegment` with `StartCantLeft` , `StartCantRight` , `EndCantLeft` , `EndCantRight` , and `PredefinedType` . The cant segment type mapping follows the same spiral type table as horizontal alignment (Section 11.4.4).

11.3.3 Rail Head Distance

`IfcAlignmentCant.RailHeadDistance` is required and represents the distance between rail heads (track gauge). LandXML may carry gauge information in a `<Roadway>` or project-level element, or it may need to be supplied as an external parameter. The conversion must account for this value explicitly — it is used in the Viennese Bend cant factor calculation described in Section 2.

11.6 Stationing and Station Equations 11.3.1 Start Station

The LandXML `<Alignment staStart="...">` attribute gives the station at the beginning of the alignment. Map this to an `IfcReferent` with `PredefinedType = REFERENCEMARKER` at `DistanceAlong = 0.0` , with the station value stored as a property in an associated property set.

11.3.2 Station Equations (`<StaEquation>`)

LandXML `<StaEquation>` records station breaks where the back station and ahead station differ. Each equation maps to an additional `IfcReferent`. The relevant attributes are:

LandXML Attribute	Meaning
`staBack`	Station value before the equation (incoming)
`staAhead`	Station value after the equation (outgoing)
`staInternal`	Continuous internal station (no breaks)

Table 10.7.2-1 — LandXML station equation attributes

The `staInternal` value should equal `staAhead` in a well-formed LandXML file. Log a warning if `staInternal != staAhead`, as this indicates an unusual stationing condition that may require manual review.

11.7 IFC Project Structure

LandXML has no concept of project hierarchy. IFC requires a minimum entity graph regardless of what is in the LandXML file. The required structure is:

```
IfcProject
  └─ IfcRelAggregates
       ├─ IfcSite
       └─ IfcAlignment
            └─ IfcRelNests
                 ├─ IfcAlignmentHorizontal
                 │    └─ IfcRelNests → IfcAlignmentSegment (×n)
                 ├─ IfcAlignmentVertical
                 │    └─ IfcRelNests → IfcAlignmentSegment (×n)
                 └─ IfcAlignmentCant (if cant data present)
                      └─ IfcRelNests → IfcAlignmentSegment (×n)
```

The `IfcProject` must include an `IfcUnitAssignment` derived from the LandXML `<Units>` element (Section 11.3). The `IfcSite` provides the spatial context and local placement for the alignment.

11.8 Known Limitations and Unresolved Issues

The following issues are known limitations of the LandXML-to-IFC conversion and should be documented for users of any converter implementation.

Unhandled spiral types. The LandXML spiral types `radioid`, `weinerBogen`, `revBiquadratic`, `revBloss`, `revCosine`, and `revSinusoid` have no IFC equivalent. Segments of these types are skipped with a warning.

Unhandled horizontal element types. `<Chain>` (polyline) and `<IrregularLine>` elements are skipped. A future implementation could decompose them into sequences of `LINE` segments.

`sineHalfWave` **approximation.** Mapped to `COSINECURVE` as an approximation. The two curves are similar but not identical.

Circular vertical curve half-length approximation. The start station of a `<CircCurve>` is approximated as $s_{PVI} - L/2$. The exact value depends on the tangent length, which requires iteration for large deflection angles.

Asymmetric parabolic arc split. The split of `<UnsymParaCurve>` into two `PARABOLICARC` segments is geometrically correct but results in two IFC segments where LandXML had one. Downstream tools must be able to handle this.

Imperial units. Imperial linear unit conversion (foot, US Survey foot) must be applied to all coordinate values. Partial implementations that read the unit declaration but do not scale the coordinates will produce incorrect geometry.

No support for LandXML surface or roadway data. `<Roadway>`, `<Survey>`, `<Surfaces>`, and other LandXML elements beyond alignment geometry are out of scope for an alignment-focused converter.

Section 12 - Precision and Tolerance 12.0 Introduction

`IfcCurveSegment.Transition` defines the intended transition from the end of one segment to the start of the next. The options are:

Transition Value	Description
`CONTINUOUS`	The segments join but no condition on their tangents is implied.
`CONTSAMEGRADIENT`	The segments join and their tangent vectors or tangent planes are parallel and have the same direction at the joint; equality of derivative is not required.
`CONTSAMEGRADIENTSAMECURVATURE`	For a curve, the segments join, their tangent vectors are parallel and in the same direction and their curvatures are equal at the joint: equality of derivatives is not required. For a surface this implies that the principle curvatures are the same and the principle directions are coincident along the common boundary.
`DISCONTINUOUS`	The segments do not join. This is permitted only at the boundary of the curve or surface to indicate that it is not closed.

Figure 11.0-1 illustrates the various transition types.

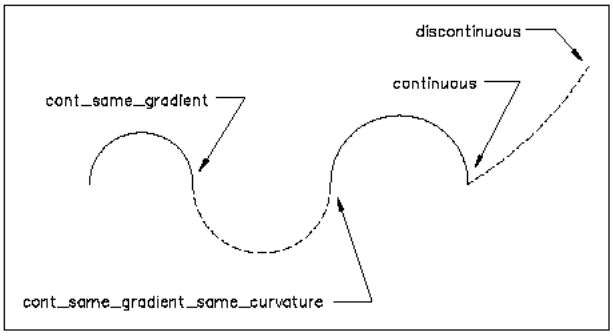


Figure 11.0-1 - Illustration of segment transition types

Adjacent segment end and start points may not exactly line up in position, gradient, or curvature for several reasons:

- **Rounded design parameters.** Designers routinely round curve parameters — radius, arc length, tangent bearing — to convenient values. Geometry computed from those rounded values does not land exactly on the next segment's stored start point.
- **Independent segment computation.** Each segment's coordinates are often computed independently from its own parameters rather than chained forward from the previous segment's computed end. Floating-point rounding differences between two independent computations of the same nominal point produce small gaps or overlaps at joints.
- **Numerical integration of transition curves.** The x/y ordinates at the end of a spiral are transcendental integrals — Fresnel integrals for Clothoids, analogous forms for Bloss, Cosine, and the other spiral types — that different software approximates with different precision. The same nominal parameters can yield slightly different endpoint coordinates depending on the tool that produced them.
- **Unit conversion.** Converting between feet and meters, or between degrees/minutes/seconds and decimal degrees, introduces rounding that propagates to all downstream coordinates.
- **Legacy source fidelity.** Alignments derived from historical plan sheets, right-of-way plats, or field surveys carry the measurement uncertainty inherent in the original source material.

`Pset_Tolerance` is used to indicate the acceptable tolerance under which the calculated end point of a preceding segment would be considered geometrically continuous with the provided start point of the following segment. Similarly, `Pset_Uncertainty` is used to indicate the uncertainty inherent in historical information gleaned from plan sets and other historic sources. As a fallback if `Pset_Tolerance` is not provided, tolerance can be taken from `IfcGeometricRepresentationContext.Precision`.

todo:

- Develop this Section - the idea is to describe the content of the example folders developed by Andres
- Examples were developed by Andres (P - last name) as part of the buildingSMART Germany chapter's BIM Fit Check competition.
- need to update the examples, there arent any zero length segments
- there are missing cases so those examples need to be developed
-

Section 13 - Examples and Validation Data 13.0 Introduction